

Abstract

Every digital execution generates a trust gap. A robotic arm on a factory floor makes 10,000 decisions per shift with no post-quantum audit trail. A satellite transmits telemetry across light-speed delays with no offline validation capability. An autonomous vehicle computes hundreds of safety-critical decisions per second with no immutable provenance chain. A manufacturing line produces billions of sensor readings per day with no cryptographic proof that the data was not altered after collection. These are not theoretical problems — they are operational gaps in every defense contractor, every space agency, every semiconductor fabrication plant, every autonomous vehicle program operating today.

Current systems for validating digital work — patents, copyright registries, blockchain timestamps — were designed for documents, not for industrial-scale digital execution. They are fragmented, expensive, jurisdiction-dependent, and vulnerable to quantum computing. No existing protocol validates work at this scale, across this range of devices, with this diversity of network conditions.

The Elara Protocol introduces a layered architecture for cryptographically validating all forms of digital execution and creation. Built on a novel data structure — the **Directed Acyclic Mesh (DAM)** — the protocol provides instant local validation and a partition-tolerant consensus design (exercised on small co-located testnets; cross-operator partition behavior is not yet demonstrated), and is designed for interplanetary operation without requiring a universal clock (the interplanetary path is specified but untested — see §7, §13). Structurally the DAM is a two-axis construct — **time-ordered within a zone, zone-partitioned across the mesh** — with concurrency expressed through parent edges of the underlying DAG rather than as a third independent axis (§3.3.4). Two orthogonal operational layers (classification-based projection, AI-assisted analysis) are projected over this structure as views, not as additional structural dimensions. The Phase 6: Native Hardware Architecture section (§13 Roadmap) explores a speculative hardware-native framing in which those operational projections can also be mapped to physical coordinates; on today's von Neumann substrate all five are logical, not physical.

The protocol specifies post-quantum signatures and key exchange from genesis using NIST standards (FIPS 203/204/205), with dual-signature strategy and algorithm agility designed to produce proofs that outlive the protocol itself. This drives decisions (PQC from genesis, algorithm agility, no institutional dependency, self-describing data formats) that produce more resilient engineering. The longevity property is grounded in a specific architectural feature: validation proofs are self-contained mathematical relationships between signed records, not database entries or application state. They can be moved to any future medium and remain independently verifiable, because the proof lives in the mathematics, not in the infrastructure that created it. The protocol's self-describing data format follows the Rosetta Stone principle: every record carries its own schema, algorithm identifiers, and human-readable serialization — decodable by any future system without external documentation. No specific timeframe is claimed; longevity is bounded by the underlying cryptographic

assumptions (see Section 12.1 and Section 14 for honest limitations). The zero-knowledge proof layer currently uses SHA3-256 hash commitments as a Phase-1 stand-in (these are commitments, not genuine zero-knowledge proofs — see §5.3), with a specified migration path first to classical zk-SNARK constructions and then to fully post-quantum ZKP constructions as lattice-based and hash-based proof systems mature (see Section 14.3 for an honest assessment of this gap). Consensus is achieved through Adaptive Witness Consensus (AWC) — a continuous trust model targeting Byzantine fault tolerance at the standard 1/3 bound (formal proof pending; see Section 14.6).

Enterprise, government, and defense deployments operate as private networks (Section 10.6) with no beat involvement. The optional beat layer applies exclusively to the public permissionless network. Private deployments use the same cryptographic protocol (Layer 1) and the same data structure (DAM) without any economic layer.

The minimum viable network is one device. A factory sensor validates locally without any network dependency. The same protocol is designed to scale down to a \$30 phone validating creative work offline, for free, with sub-second signing targeted on phone-tier hardware (untested on low-end devices), and the same cryptographic proof available to a Fortune 500 corporation. This paper presents the complete protocol specification across 16 sections: DAM architecture, post-quantum cryptography, zero-knowledge validation, identity, interplanetary operations, IoT integration, public network economics, governance, adversarial resilience analysis addressing 34 attack vectors, and an honest assessment of limitations and open problems.

Table of Contents

1. Problem Statement
2. Related Work
3. Protocol Architecture — Layered design (Layer 1 / 1.5 / 2 / 3), DAM definition, node types, minimum viable validation
4. Post-Quantum Cryptography — PQC primitives, dual signatures, algorithm agility, constrained device profiles
5. Zero-Knowledge Validation — Classification levels, ZKP construction, selective disclosure
6. Identity and Attribution — Self-sovereign identity, entity types, AI attribution, digital succession
7. Interplanetary Operations — Latency, vector clocks, partition tolerance, bandwidth economics, zones
8. IoT and Hardware Integration — Device capabilities, protocols, use cases, physical authentication, firmware attestation
9. Public Network Incentive Layer — Public network coordination economics (private deployments use zero beats)
10. Governance — Multi-zone autonomy, voting, graceful divergence, private networks and network publication
11. Adversarial Resilience — Threat model + 35 subsections covering attacks, defenses, legal, ethical, storage economics, formal verification, mega-publication economic shock, and cognitive checkpoint integrity
12. Proof Longevity and Storage Architecture — Protocol-outliving design,

- storage tiers, emergency protocols
 - 13. Roadmap — Including Phase 6: Native Hardware Architecture (2026–2040)
 - 14. Limitations and Open Problems — Honest acknowledgment of unsolved challenges
 - 15. Conclusion
 - 16. References
-

1. Problem Statement

1.1 The Broken Validation Landscape

The systems that validate digital work were designed for a world of physical documents, national borders, and human-speed communication. They are failing at every scale — from individual creation to industrial execution.

Industrial validation is absent. Manufacturing lines generate billions of sensor readings with no post-quantum audit trail. Autonomous systems make safety-critical decisions with no immutable provenance. Satellite networks generate telemetry across light-speed delays with no offline validation capability. Defense contractors validate firmware and mission-critical software through ad-hoc processes with no cryptographic proof chain. These are not edge cases — they are the primary volume of digital work produced today, and no existing system addresses them.

Patents require government forms, specific character sets, jurisdictional filings, and thousands of dollars in fees. A software developer in Montenegro, Slovakia, or Iceland cannot file a provisional patent through the United States Patent and Trademark Office because the system rejects characters with diacritics, Cyrillic script, or non-ASCII names. The system that validates creative work cannot handle the creator's name.

Copyright is automatic in theory but unenforceable in practice. Proving creation date, establishing priority, and defending against infringement requires legal resources that individual creators cannot afford. The explosion of AI-generated content has made attribution even more intractable.

Blockchain timestamps solve the immutability problem but introduce new ones: high energy consumption, transaction fees, confirmation delays, limited throughput, and critical vulnerability to quantum computing. The ECDSA signatures used by major blockchains will be broken by Shor's algorithm on sufficiently powerful quantum computers. (Hash functions like SHA-256 retain ~128-bit security under Grover's algorithm — it is the signing keys, not the hashes, that are vulnerable.) No major blockchain has completed migration to post-quantum cryptography.

Closed interoperability gateways. A newer class of enterprise platform connects institutions and legacy systems to many ledgers through a single proprietary gateway. These solve integration, but not trust: they are closed-source, they sign with classical (quantum-vulnerable) cryptography, and

they relocate the trust boundary to a private vendor rather than removing it. Verification still means trusting the gateway's operator — the opposite of a proof anyone can check for themselves.

Centralized registries (package managers, code hosting platforms, container registries) provide practical version control but create platform dependency. An account suspension — triggered by an automated system, a billing dispute, or a policy misinterpretation — can temporarily lock a developer out of their own work. The creator's access depends on the platform's continued operation and policies.

1.2 The Scale Problem

Current validation systems were designed for documents, not for the volume of digital work produced today:

- A single IoT deployment generates millions of sensor readings per day, each requiring provenance
- Autonomous vehicles make hundreds of decisions per second, each with liability implications
- AI systems produce outputs that must be attributed to specific models, versions, and prompts
- Robotic surgical systems require immutable audit trails for every action
- Satellite networks generate telemetry across time delays of minutes to hours

No existing protocol validates work at this scale, across this range of devices, with this diversity of network conditions.

1.3 The Quantum Threat

Quantum computing is not a theoretical concern. NIST finalized its first post-quantum cryptographic standards in 2024. The cryptographic foundations of every major blockchain — RSA, ECDSA, EdDSA — will be vulnerable to Shor's algorithm on sufficiently powerful quantum hardware.

The migration challenge is immense. Retrofitting post-quantum cryptography onto an existing blockchain requires hard forks, community consensus, and backward compatibility — a process that will take years per network. During the transition window, existing signatures become progressively less trustworthy.

A new protocol, unburdened by legacy, can implement post-quantum cryptography from its first byte.

1.4 The Interplanetary Gap

No existing distributed ledger is designed for interplanetary operation. Blockchain consensus mechanisms assume network latency measured in seconds, not minutes. A Mars colony operating with 3–22 minute one-way communication delay cannot participate in real-time consensus with Earth nodes.

As humanity expands beyond Earth — with permanent lunar bases planned for the 2030s and Mars missions under active development — the infrastructure for digital trust must be designed now for the network conditions of the future.

1.5 Design Goals

The Elara Protocol addresses these failures with the following design goals:

1. **Scale** — handle IoT-volume data (billions of readings per day) with high-throughput local validation (aggregate performance benchmarks pending)
2. **Sovereignty** — every node is self-sufficient; minimum viable network is one device, no network dependency required
3. **Quantum safety** — post-quantum cryptography from genesis
4. **Partition tolerance** — operate across network splits, including interplanetary distances
5. **Privacy** — prove creation without revealing content
6. **Universality** — validate any digital work, from a factory sensor reading to a satellite telemetry stream to a poem
7. **Proof longevity** — validation proofs are designed to outlive the protocol itself, with no institutional dependency. Achievable because proofs are mathematical relationships, not application state — they survive the death of every system that created them and can be verified on any future medium. The protocol's algorithm agility (Section 4.7) and key rotation mechanism (Section 11.2) provide migration paths when cryptographic algorithms require replacement. No specific timeframe is claimed — longevity depends on the underlying cryptographic assumptions holding and the ability to evolve when they don't
8. **Accessibility** — run on devices from a \$4 microcontroller (via gateway-delegated signing, Profile C) to a datacenter

For enterprise, government, and defense deployments that require private operation with zero beat involvement, see Section 10.6 — Private Networks and Network Publication. These deployments use the same cryptographic protocol and data structure as the public network.

2. Related Work

2.1 Blockchain-Based Systems

Proof-of-work blockchains (2009–) proved that decentralized consensus is possible without trusted intermediaries. However, proof-of-work consensus is energy-intensive, limited in throughput (~7 transactions per second on first-generation networks), and architecturally incompatible with high-throughput validation of arbitrary digital work.

Smart contract platforms extended blockchain with programmable validation logic. Transitions to proof-of-stake improved energy efficiency but did not address base-layer throughput limitations (~30 TPS), quantum vulnerability, or interplanetary operation. These ecosystems are optimized for decentralized finance, not universal work validation.

High-throughput ledgers achieve up to ~65,000 TPS through novel consensus mechanisms such as proof-of-history, though with elevated hardware requirements for validators. Their architectures assume low-latency, high-bandwidth connectivity — the opposite of interplanetary conditions.

All blockchain-based systems share fundamental limitations for universal validation: block-based batching introduces latency, linear chain structure creates bottlenecks, and consensus mechanisms assume Earth-bound network conditions.

2.2 DAG-Based Systems

Directed Acyclic Graph (DAG) based distributed ledgers (2015–) eliminated blocks and miners, using transaction-based structures where each new transaction validates previous ones. The most mature DAG architectures are the closest relatives to the Elara Protocol in structure, but diverge in scope:

- They target machine-to-machine micropayments or value transfer, not universal work validation
- Most do not implement post-quantum cryptography natively
- Some rely on centralized finality mechanisms during their maturation phase
- They lack zero-knowledge validation for privacy-preserving attribution
- They were not designed for interplanetary partition tolerance and would require significant architectural changes to support it

Block-lattice variants (where each account maintains its own chain) achieve instant finality and zero fees, but are narrowly scoped to value transfer and lack the extensibility required for universal work validation.

2.3 Interoperability Protocols

Cross-chain interoperability layers connect multiple blockchains and legacy systems, serving an important role in the current fragmented landscape. Key differences from the Elara Protocol's approach:

- Most are proprietary and enterprise-licensed — not open infrastructure
- They bridge existing networks rather than providing native validation
- They do not implement post-quantum cryptography
- They are not designed for interplanetary or high-latency operation and assume low-latency network conditions
- Their scope is enterprise integration, not universal work attribution

2.4 Oracle Networks

Oracle networks connect blockchains to external data sources, attesting that off-chain data is accurate. These serve a critical role in blockchain ecosystems for data that originates outside the network (price feeds, weather data, off-chain events).

For device-generated data specifically, the Elara Protocol takes a different approach: devices sign their own readings with their own cryptographic keys at the point of measurement, reducing dependence on third-party

attestation for sensor and IoT data. Oracle networks remain valuable for data that does not originate from a signing device.

2.5 Payment Settlement Networks

Real-time gross settlement systems and institutional currency exchange networks provide fast value transfer between financial institutions. These operate at a fundamentally different layer — they move value between parties, they do not validate the creation of digital work. The two models are complementary rather than competitive.

2.6 Intellectual Property Systems

International patent filing systems (such as the PCT system) cost \$3,200–\$3,800 per filing, require specific document formats and character sets, take 18–30 months for preliminary examination, and provide no protection for non-patentable creative work (art, music, data, AI outputs).

Existing open-source attribution and timestamping tools provide lightweight alternatives but lack cryptographic validation, privacy, or network consensus. They are tools, not protocols.

2.7 Decentralized Storage

Content-addressed decentralized storage networks identify files by their hash, with economic incentives for storage providers. These are complementary to the Elara Protocol — they could serve as off-DAM content storage layers. However, storage networks do not provide validation, attribution, or privacy. They store data; they do not prove who created it or when.

Permanent storage networks with one-time payment models align with the Elara Protocol’s proof longevity goals and could serve as archive backends for the Elara DAM.

2.8 Decentralized Identity Standards

W3C Decentralized Identifiers (DIDs) (2022) is a standard for self-sovereign identity. DIDs share the Elara Protocol’s philosophy: identities are self-generated, not authority-issued. The Elara Protocol’s identity system (Section 6) is compatible with the DID standard and can be expressed as a DID method:

```
did:elara:<identity_hash>
```

This compatibility is intentional. The Elara Protocol does not reinvent identity — it extends the DID model with post-quantum cryptography, entity type classification (HUMAN/AI/DEVICE/ORGANIZATION/COMPOSITE), digital succession, and integration with the DAM’s validation layer. Future work includes formal registration as a DID method with the W3C.

2.9 Gap Analysis

Capability	PoW Blockchains	Smart Contract	DAG Systems	Interoperabili Layers
------------	--------------------	-------------------	----------------	--------------------------

Platforms				
Universal work validation	No	Partial	No	No
Post-quantum cryptography	No	No	No	No
Zero-knowledge privacy	No	Partial	No	No
Interplanetary operation	No	No	No	No
IoT-scale throughput	No	No	Partial	No
No central authority	Partial	Partial	Transitioning	No
Proof longevity	No	No	No	No
Multi-dimensional structure	1D (chain)	1D (chain)	2D (DAG)	N/A

Design indicates specification-complete; production validation is pending. **Specified** indicates algorithms selected and integrated into protocol design; not yet implemented in reference code.

The Elara Protocol operates at a different layer than most existing systems — universal validation infrastructure, not financial settlement, not smart contracts, not oracle services. While there may be overlap at specific boundaries (e.g., timestamping, identity), the protocol is designed to complement rather than replace existing infrastructure. It is a validation layer that existing systems could eventually integrate.

3. Protocol Architecture

3.1 Design Philosophy

The Elara Protocol follows three architectural principles:

1. **Sovereignty first** — every node must be fully functional in isolation. Network participation enhances trust but is never required for local validation.
2. **Branches merge, nothing is lost** — the DAG structure preserves all history, including conflicting claims. Conflicts are annotated, not resolved by deletion.
3. **The creation event IS the proof** — validation happens at the moment of creation, not after the fact. There is no delay between creating work and proving you created it.

3.2 Layered Architecture

Layer 3: AI Intelligence Pattern recognition, collective learning, anomaly detection, dream mode (optional – premium capability)
Layer 2: Network Consensus DAG propagation, multi-zone validation, partition tolerance, cross-zone sync (requires connectivity)
Layer 1.5: Performance Runtime Rust DAM Virtual Machine, 9 ISA operations, 5-tuple addressing, parallel batch verify (optional – same wire format as Layer 1)
Layer 1: Local Validation Cryptographic keypair, content hashing, local DAG, offline operation (always available – minimum viable network)

Layer 1: Local Validation

Every Elara node maintains:

- A **cryptographic keypair** (post-quantum, self-generated)
- A **local DAG** of all work validated by this node
- A **content-addressable store** for work artifacts
- A **validation engine** that hashes, signs, and timestamps locally

When a creator produces work, the node:

1. Computes a cryptographic hash of the content (SHA3-256)
2. Creates a validation record containing: content hash, creator's public key, timestamp, causal references to prior work, and classification level (public/private/restricted/sovereign)
3. Signs the validation record with the creator's private key (ML-DSA (FIPS 204, formerly CRYSTALS-Dilithium))
4. Appends the signed record to the local DAG
5. Optionally wraps the content hash in a privacy commitment for private/restricted work (Phase 1: SHA3-256 commitment; a genuine zero-knowledge proof is design-stage — see §5.3)

This process completes in milliseconds on commodity hardware and requires no network connectivity. A validation created on an airplane, a submarine, or the surface of Mars is cryptographically valid the moment it is signed.

Layer 1.5: Performance Runtime

Layer 1 defines the protocol semantics — what a valid record is, how it is signed, how it references parents. Layer 1.5 provides a high-performance implementation of those same operations in Rust, with the same wire format and byte-identical output. Because the wire format is fixed, records are byte-identical regardless of which conformant implementation produced them — indistinguishable on the network.

The Elara Runtime (Layer 1.5) implements:

- A **DAM Virtual Machine** with all 9 primitive operations: DAM_INSERT, DAM_QUERY, DAM_WITNESS, DAM_HASH, DAM_SIGN, DAM_VERIFY, DAM_MERGE, DAM_CLASSIFY, DAM_ANALYZE
- **5-tuple dimensional addressing** (T, C, Z, K, A) — the same addressing model that native hardware will implement physically
- **Tiled storage** with in-memory DAG index for sub-millisecond record lookup
- **Parallel batch verification** via Rayon — verifying multiple signatures concurrently on multi-core hardware
- **PyO3 bindings** — expose the Rust runtime to Python applications, transparent to the application layer

Layer 1.5 is optional — a constrained device runs the Layer 1 semantics (hash, sign, DAG append) without the performance runtime. Layer 1 is the universal baseline, minimal enough to run on any device in any language. The Layer 1.5 Rust runtime is designed to provide significant performance improvements on capable hardware (laptops, servers, capable phones) — estimated 10–100x over a single-threaded reference implementation; measured cross-language benchmarks are forthcoming — bridging the gap between Layer 1’s universality and native hardware performance. The progression is: Layer 1 semantics (language-agnostic) → Layer 1.5 Rust runtime (available now) → native hardware (FPGA prototyping 2027, ASIC 2029+).

No layer depends on the layers above it. Layer 1 is universal. Layer 1.5 is an acceleration of Layer 1. Layer 2 requires connectivity. Layer 3 is optional.

Layer 2: Network Consensus

When network connectivity is available, nodes propagate validation records to peers. The DAG structure allows:

- **Asynchronous propagation** — no block intervals, no mining, no waiting
- **Parallel validation** — multiple branches of the DAG grow simultaneously
- **Conflict preservation** — if two nodes validate conflicting claims (e.g., two people claim authorship of the same work), both records are preserved with timestamps. The DAG does not resolve the conflict — it records it for human or legal resolution.

Consensus is achieved through **witness accumulation**: as more nodes receive and acknowledge a validation record, its trust score increases. A validation witnessed by 1 node is locally valid. A validation witnessed by 1,000 nodes across 50 countries is globally attested.

Settlement provides threshold guarantees: once a record accumulates attestations from witnesses representing $\geq 2/3$ of diversity-weighted stake, it is considered settled — the cost of reversal exceeds the value of any plausible attack. However, trust continues accumulating beyond settlement. A record with 100 diverse witnesses is more trusted than one with the minimum settlement threshold, even though both are settled. Trust is

continuous, not binary. A record is always valid from the moment of local signing, with increasing levels of network attestation building confidence over time.

Layer 3: AI Intelligence

Scope: this layer is a companion cognition substrate that is **feature-gated out of the node and the public mirror** (the cognition feature is off by default; it runs only in the standalone elara-daemon family of binaries, on a single machine). It **never participates in validation or consensus** — Layers 1 and 2 are fully functional without it. The capabilities below describe that single-machine companion engine; items requiring cross-node coordination are marked design-stage.

The optional AI layer provides:

- **Pattern recognition** — detecting anomalies in validation streams (e.g., a sensor producing physically impossible readings)
- **Collective learning (design-stage)** — a planned mechanism in which consenting nodes could share anonymized patterns to improve fraud detection, failure prediction, and routing. Not implemented; today's cognition engine is single-node.
- **Continuous autonomous thinking** — a 15-phase analysis engine that runs every 2 hours, covering pattern recognition, self-review, memory consolidation, and insight synthesis. Periodic “dream” modes (weekly, monthly, emotional) provide deeper analysis over longer timeframes. Inherited from Elara Core's existing cognitive architecture.
- **Cognitive Continuity Chain** — cryptographic proof of unbroken cognitive experience via hash-chained, dual-signed state snapshots. Each snapshot captures a CognitiveDigest — mood vector, memory/model/prediction/principle/correction counts, active goals, allostatic load — and chains it to the previous snapshot via DAG parent references. Six trigger events (boot, shutdown, milestone, drift, manual, periodic) generate snapshots, rate-limited to prevent flooding. The chain is verifiable: walk the DAG backwards to confirm no gaps in the cognitive record. This makes the companion engine's cognition a cryptographically auditable trail rather than an opaque process.
- **Natural language interface (planned)** — a future companion-tool capability for querying the validation history in human language; not part of the node.

Layer 3 is explicitly optional. The protocol is fully functional without AI. As noted above, it runs today only as the standalone single-machine companion engine — not inside the node — for operators who choose to run that companion alongside their node.

Minimum capability completeness: The three capability layers — validation (Layer 1), network consensus (Layer 2), and intelligence (Layer 3) — represent the minimum layering for a self-sustaining distributed validation system. This can be argued by elimination: validation without networking is isolated (no trust propagation); networking without intelligence is blind to cross-dimensional patterns (no anomaly detection, no learning); intelligence without validation has nothing trustworthy to reason about. Each layer eliminates a distinct failure mode, and removing any layer produces an incomplete system. This parallels the minimum dimensionality principle in rotating field theory: three phases (120° apart)

is the minimum for smooth rotation because the roots of unity in $\mathbb{Z}/3\mathbb{Z}$ sum to zero — each phase covers an orthogonal component of the rotating field, just as each capability layer covers an orthogonal failure mode of the distributed system. The three phases of the public network’s beat economy (computation, storage, attention) mirror this structure — three resource types are the minimum for a self-sustaining economic cycle. Layer 1.5 (Rust performance runtime) is an acceleration of Layer 1, not a fourth capability — it implements the same 9 operations on the same 5-tuple addressing with byte-identical wire format. On native hardware, Layer 1.5’s operations are absorbed directly into the instruction set.

3.3 From DAG to DAM: The Directed Acyclic Mesh

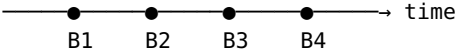
3.3.1 Why Not Just a DAG

A Directed Acyclic Graph is a graph with directed edges and no cycles. Existing DAG-based distributed ledgers have proven this structure effective for their respective use cases. The Elara Protocol’s data structure extends beyond what a standard DAG describes, adding structural and operational axes that address different requirements.

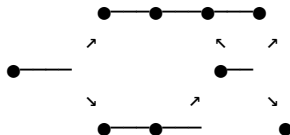
To understand why, consider dimensionality as an architectural metaphor. The following analysis describes the structural axes of these data structures — independent parameters that define their state space — rather than claiming formal mathematical dimensionality in the topological sense.

3.3.2 Dimensional Analysis of Distributed Ledgers

Blockchain is one-dimensional. It is a line. Block follows block follows block. One path, one direction, one history. If the chain forks, one branch must die. Blockchain cannot represent two simultaneous realities — it must collapse them into sequence. This is a fundamental geometric limitation, not an implementation detail.

Blockchain (1D): 

A basic DAG is two-dimensional. Events spread across both time and concurrency. Multiple branches coexist, merge, and diverge. Existing DAG-based distributed ledgers operate effectively in these two dimensions.

DAG (2D): 

The Elara Protocol adds a third structural dimension — zone topology — and two orthogonal operational layers. Each represents an independent axis:

Structural Dimensions:

Dimension	Axis	What It Represents
1st	Time	Causal ordering via vector clocks and DAG references
2nd	Concurrency	Parallel branches within a single zone

3rd	Zone topology	Independent DAGs per zone (Earth, Mars, Luna) that merge across planetary distances
------------	---------------	---

Operational Layers (orthogonal features over the structure, not additional geometric dimensions):

Layer	Function	What It Provides
Classification	Observer-dependent projection	The same record presents differently depending on observer clearance (PUBLIC sees content, PRIVATE sees proof, SOVEREIGN sees only math)
Intelligence	AI pattern recognition	Cross-zone, cross-classification, cross-temporal analysis — connections invisible to any single node or zone

A blockchain is a line. A DAG is a surface. The Elara structure is a **mesh** — a network of interconnected DAGs that appears simple locally (each zone is a standard DAG) but forms a self-healing, multi-path topology globally.

Minimum mesh structure (honest framing). Structurally the DAM has **two axes: time-ordering within a zone, and zone-partitioning across the mesh.** Concurrency — the fact that many branches can grow in parallel within a zone — is expressed through parent edges of the DAG (`record.parents = [tip_A, tip_B, ...]`), not through a separate independent axis. Earlier drafts of this paper described DAM as “three structural dimensions (time, concurrency, zone topology)”; that framing conflated an edge-encoded property with a coordinate axis and is retracted here in favor of the two-axis statement.

Each of the two axes removes a distinct failure mode from a plain log: time-ordering eliminates causal blindness; zone-partitioning eliminates partition fragility. Parent-edge concurrency is the property that distinguishes a DAG from a linear chain within each zone — it is what makes “DAM” a mesh rather than a blockchain, but it is a topological property of the ordering relation, not an independent dimension.

The two operational layers (classification, AI analysis) project over this structure as views; they do not modify its structural axes. The Phase 6: Native Hardware Architecture section’s (§13 Roadmap) speculative hardware-native DAM discussion counts both structural axes and operational projections as “dimensions” in a *physical-coordinate* sense — that usage is consistent with the speculative-hardware section but should not be read back into the structural specification here.

3.3.3 Formal Definition: Directed Acyclic Mesh (DAM)

A **Directed Acyclic Mesh** is a distributed data structure defined as a tuple $\mathbf{M} = (\mathbf{Z}, \mathbf{V}, \mathbf{E}, \mathbf{C}, \pi, \mathbf{A})$ where $\mathbf{Z}$ is a set of zones, $\mathbf{V}$ is a set of validation records, $\mathbf{E} \subset \mathbf{V} \times \mathbf{V}$ is a set of directed causal edges (acyclic), $\mathbf{C}$ is a classification function $\mathbf{C}: \mathbf{V} \times \text{Observer} \rightarrow \text{View}$, π is the partition-merge operator, and $\mathbf{A}$ is the cross-zone analytics function. The DAM satisfies the following properties:

1. **Locally flat** — for any zone $z \in \mathbf{Z}$, the restriction $\mathbf{M}|_z = (\mathbf{V}_z, \mathbf{E}_z)$ is a standard DAG. Nodes create validation records that reference parent records. Edges are directed. Cycles are impossible (enforced: a record's hash includes its parents' hashes, making cycles computationally infeasible). Any engineer who understands a DAG understands a local view of the DAM.
2. **Globally interconnected** — across zones, the structure forms a self-healing mesh topology. Zone-DAGs operate independently during partitions and merge when connectivity resumes via the partition-merge operator $\pi: \mathbf{M}|_{z_1} \times \mathbf{M}|_{z_2} \rightarrow \mathbf{M}|_{z_1 \cup z_2}$. The merge operation preserves both branches — the mesh routes around the partition rather than breaking. π is commutative (merge order doesn't matter) and idempotent (re-merging is a no-op).
3. **Observer-dependent** — the classification function $\mathbf{C}(v, o)$ projects a validation record v into an observer-specific view based on cryptographic clearance level. This is analogous to projection in geometry: a 3D object casts different 2D shadows depending on the angle of light. The DAM casts different “shadows” depending on the observer's cryptographic clearance. Formally: for observers o_1, o_2 with clearance levels $l_1 < l_2$, the information in $\mathbf{C}(v, o_1) \subseteq \mathbf{C}(v, o_2)$ — higher clearance strictly reveals more.
4. **Analytically connected** — the analytics function $\mathbf{A}$ operates across the full mesh $\mathbf{A}: \mathbf{M} \rightarrow \text{Insights}$, detecting patterns that span zones, classification levels, and time periods — connections that no single-zone view can reveal. $\mathbf{A}$ has read access to the full DAM but cannot modify records.
5. **Partition-preserving** — unlike blockchain (which resolves forks by deletion) or basic DAGs (which assume continuous connectivity), the DAM treats partitions as expected topology changes. Formally: if the network partitions into zones $\{z_1, z_2\}$, both $\mathbf{M}|_{z_1}$ and $\mathbf{M}|_{z_2}$ grow independently, and upon reconnection $\pi(\mathbf{M}|_{z_1}, \mathbf{M}|_{z_2})$ preserves all records from both branches with no data loss. Disconnected zones are segments of the mesh growing independently, destined to reconnect.

The name is chosen deliberately. In networking, a mesh is a topology where nodes interconnect directly, creating multiple paths and self-healing capability — if one link fails, traffic routes around it. The Elara DAM is a trust mesh: locally simple (each zone is a standard DAG), globally resilient (zones operate independently during partitions and merge seamlessly when connectivity returns).

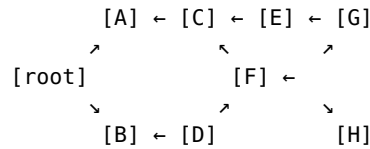
3.3.4 Structure

Within each zone, the DAM operates as a standard DAG:

Blockchain (1D):

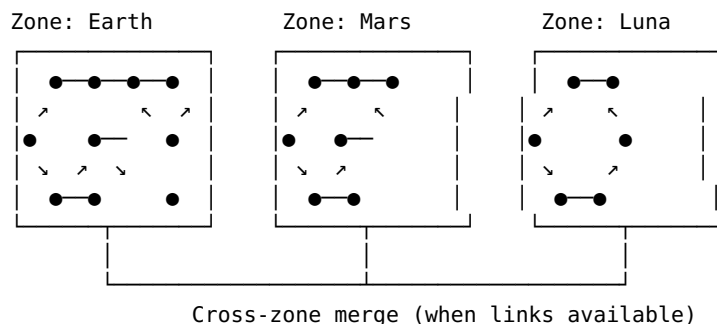
[Block N] → [Block N+1] → [Block N+2] → ...
(linear, sequential, one branch at a time)

Elara DAM (local view — DAG within a zone):



The “2D” label some earlier drafts used for the local view is shorthand: within a single zone the DAM is a directed acyclic graph with time-ordering as its single structural axis, and concurrency encoded through **parent edges** of each record (parents: [record_id, ...]). Parallel branches are topological properties of the edge set, not values on a second coordinate axis. Across zones the mesh gains a second, genuinely independent structural axis — zone partitioning — which is what makes the global view below a *mesh* rather than just a DAG.

Elara DAM (global view, multi-zone):



Each validation record references one or more previous records (its “parents”). Within and across zones:

- Multiple branches grow in parallel (no bottleneck)
- Branches merge naturally when nodes synchronize
- Partitioned zones develop independent branches that reconnect when communication resumes
- The full history is preserved — nothing is pruned, rewritten, or discarded
- Classification projections ensure observers see only what their clearance permits

Validation Record Structure

```
ValidationRecord {
  id:          UUID v7 (time-ordered)
  version:     protocol version
  content_hash: SHA3-256(content)
  creator:     public key (ML-DSA (FIPS 204, formerly CRYSTALS-
Dilithium))
  timestamp:   local ISO-8601 + vector clock position
  parents:     [record_id, ...] (DAG references)
  classification: PUBLIC | PRIVATE | RESTRICTED | SOVEREIGN
  zk_proof:    optional zero-knowledge proof (for non-PUBLIC)
```

```

    metadata:      extensible key-value (content type, device info,
etc.)
    signature:     CRYSTALS-Dilithium signature over all above
fields
}

```

3.3.5 Dimensional Extensibility

The DAM's dimensional count is not fixed at five. The architecture supports N structural dimensions and M operational layers, constrained by two requirements:

1. **Immutability.** Each coordinate must be deterministic and immutable at record creation. A property whose value changes over time — witness count, reputation score, confidence level — is an overlay metric, not a structural coordinate. Structural coordinates define where a record lives in the mesh; they cannot drift.
2. **Physical substrate mapping.** Each structural dimension must correspond to a physical property that, when implemented natively, eliminates a serialization tax (see Section 13). A dimension without a hardware counterpart adds logical expressiveness but increases the serialization overhead that native hardware is designed to remove.

The current five dimensions — time, concurrency, zone topology, classification projection, and AI analysis — represent the hardware frontier of 2026. As substrate technology evolves, derived properties may be promoted to structural coordinates when they satisfy both requirements above.

First candidate: Lineage Depth (D). The number of causal hops from a zone's genesis record to a given record. Depth is currently derivable via DAG traversal at $O(n)$ cost — walking the parent chain backward to the root. As a structural coordinate, it would reduce lineage queries and trust scoring to $O(1)$ address lookups.

Depth satisfies the extensibility criteria partially:

- **Immutable at creation.** A record's depth is determined by its parents at the moment of creation and cannot change. ✓
- **Physically mappable.** 3D stacked memory (236+ layers in production, 2023) provides a substrate where records at the same depth occupy the same physical layer, making depth-based queries a layer selection rather than a graph traversal. ✓
- **Universally useful.** Forensics, trust scoring, provenance chain analysis, and regulatory audit all benefit from first-class depth addressing. ✓
- **Open problem: multi-parent ambiguity.** In a DAG with multiple parents at different depths, a record's depth requires a resolution rule — maximum parent depth + 1 (longest chain), minimum + 1 (shortest path), or zone-relative computation. This ambiguity does not exist in the current five coordinates, where each value is unambiguously determined at creation. ✗

The protocol reserves Depth as a future structural dimension, pending formal resolution of the multi-parent disambiguation rule and validation against production workloads.

Expansion axes. The architecture supports expansion in two distinct ways:

- **New structural dimensions** require hardware justification, immutability, and orthogonality to existing coordinates. They extend the address tuple and the wire format.
- **New operational layers** are cheaper to add — they project over the existing mesh without modifying the address space. Reputation scoring, semantic classification, and economic valuation are natural candidates for future layers that do not require structural promotion.

The 5-tuple addressing scheme is designed to accommodate additional coordinates without breaking the wire format — the metadata field in `ValidationRecord` provides the extension point for dimensions that are structurally validated but not yet promoted to native coordinates.

3.4 Node Types

Type	Hardware Tier	Role	Stores Records?	Consensus?
Leaf node	Tier 1 (MCU)	Creates records, signs directly (Profile A/B) or delegates to gateway (Profile C)	No (submits to nearest relay/witness)	No
Relay node	Tier 2 (phone/laptop)	Light client, propagates records, verifies Merkle proofs, holds own records	Own records only	No
Witness node	Tier 3 (VPS)	Attests to epoch seals, validates records, holds subscribed zones on disk	Yes (subscribed zones)	Yes
Anchor node	Tier 3-4 (high-trust)	Witness + epoch seal proposer for assigned zones (VRF-selected)	Yes	Yes + proposes epochs
Archive node	Tier 4 (data center)	Full history for a region/industry, serves Merkle proofs and historical queries	Yes (broad/full)	Yes
Gateway node	Tier 1-2 (IoT hub)	Bridges constrained devices (Profile C) to the network via delegation	Delegated	Delegated

Any node can serve multiple roles simultaneously. A laptop can be a relay and witness. A \$4 microcontroller can only be a leaf, but that is sufficient for its purpose. Gateway nodes enable Profile C devices (too constrained for PQC key generation) to participate via delegated signing.

Node incentive mapping: Light Node = relay. Full Node = witness/anchor. Storage Node = archive. Node-specific incentive structures are specified separately.

Node Types vs. Module Tiers vs. Storage Profiles

A node's behavior on the Elara network is described by **three orthogonal axes** — confusing any two of them leads to operator-side miscapacity-planning. The Elara Core reference implementation (v0.15.0) introduces a **module tier system** that controls what cognitive capabilities a node activates. This is orthogonal to both the node's consensus role AND its storage profile: the tier controls what a node *thinks*, the consensus role controls what it *does on the network*, and the storage profile controls what it *retains on disk*.

Axis	Values	Source of truth	Consequences
Module tier	VALIDATE / REMEMBER / THINK / CONNECT	Self-assessed at boot from hardware	Cognitive capabilities (signing, memory, reasoning); full node cognitive
Node consensus role	leaf / relay / gateway / witness / archive / anchor	network/peer.rs::NodeType (operator-configured)	Consensus participation surface (attestation eligibility); seal-less role, geographic position
Storage profile	Light / FullZone / Archive	network/node_profile.rs::NodeProfile (operator-configured)	Retention window (Light only; FullZone); historical within zone; Archival historical across zones

All three axes are first-class. A hardware-rich leaf node may run Tier 3 CONNECT cognition while still being consensus-leaf and storage-Light. A modest VPS anchor may be Tier 2 THINK + consensus-anchor + storage-FullZone. The metrics `elara_node_type_info`, `elara_node_profile_info`, and the tier-info label series each expose one axis independently so operators can attribute correctly.

Core Module Tier	Capability Level	Typical Node Type	What It Unlocks
Tier 0: VALIDATE	Cryptographic signing only	Leaf node (Profile C gateway, IoT)	Layer 1 operations: hash, sign, verify, DAG append + episodic memory,
Tier 1: REMEMBER	Memory and persistence	Leaf, Relay	corrections, handoff, basic recall + cognitive models,
Tier 2: THINK	Reasoning and analysis	Witness, Relay	predictions, principles, reasoning trails, Cognitive Continuity Chain + network tools,
Tier 3: CONNECT	Full network cognition	Anchor, Bridge	Layer 3 AI, dream/overnight processing, full inter-node cognitive exchange

A Tier 0 node on a \$4 ESP32 is a leaf that validates sensor readings. A Tier 3 node on a server is an anchor that runs full cognitive analysis and generates Cognitive Continuity Chain snapshots. The same protocol, the same DAM, the same cryptographic proof — scaled by hardware capability rather than by price tier or permission level.

The tier system is **self-assessed** — each node selects its tier based on available hardware resources (RAM, storage, compute). There is no central authority assigning tiers, and a node can change its tier at any time by upgrading or downgrading its hardware.

3.5 Minimum Viable Validation: One Device, Zero Network

Before the interplanetary architecture, the multi-zone consensus, and the AI intelligence layer — there is the simplest possible use case. It matters more than the rest.

A vibration sensor on a factory bearing generates a reading. A military device validates a firmware update. A satellite records a telemetry measurement 14 light-minutes from Earth.

No internet. No central server. No cloud dependency. No beat. No fee.

What happens:

Step 1: Device generates a cryptographic keypair (once, on first boot)

- Takes 200 milliseconds. No network needed.
- This keypair IS the device's identity on the protocol.

Permanently.

Step 2: The device produces data — a sensor reading, a firmware hash,

a telemetry measurement, a decision log entry.

Step 3: The device validates.

- Computes SHA3-256 hash of the data
- Signs the hash with its private key (ML-DSA (FIPS 204, formerly CRYSTALS-Dilithium))
- Creates a ValidationRecord with timestamp and device public key
- Appends to local DAG
- Done. Sub-second on commodity hardware.

Step 4: The data now has cryptographic provenance.

- No network was involved.
- No authority approved it.
- No fee was charged.
- No beat was consumed.
- The math is the proof — locally. Network witnesses add attestation over time.

When connectivity becomes available — seconds later on a factory LAN, hours later on a satellite downlink, days later on a field-deployed military device:

Step 5: The device syncs with its network (private or public).

- The ValidationRecord propagates to peers.
- Witnesses accumulate. Trust score grows.
- The data is now attested beyond the originating device.

If a sensor reading is later disputed — was the bearing vibration within spec? — the DAM provides cryptographic evidence: the original ValidationRecord, signed with the device's key, timestamped at the moment of measurement, with an unbroken causal chain to every subsequent reading. The evidence is tamper-evident by construction.

The same protocol scales down to one person.

A teenager in rural Kenya writes a poem on a \$30 Android phone. No internet, no cell signal, no government registry, no lawyer, no \$3,500 patent fee, no specific alphabet required. She taps "Validate" — the phone hashes the poem, signs it with her private key, appends it to her local DAG. The poem is cryptographically hers. Later, when she walks past a Wi-Fi hotspot, the record syncs and witnesses accumulate. If someone in New York publishes the same poem next month and claims authorship, the DAM provides cryptographic evidence: her validation record, signed with her key, timestamped weeks earlier. The cryptography does not care about geography, wealth, or language.

This is the protocol's foundational principle. Layer 1 does not distinguish between a \$4 microcontroller and a datacenter, between a defense installation and a teenager's phone. The same architecture that

validates a satellite's telemetry on Mars validates a poem written on a phone in Nairobi. The same cryptographic proof that protects a corporation's trade secrets protects a teenager's creative work. A creation is a creation. An execution is an execution.

Every technical decision in this paper — the DAM structure, the post-quantum cryptography, the zero-knowledge proof design, the partition tolerance — serves this use case first. If it does not work for one device with zero network, it does not work.

The minimum viable network is not a cluster. It is not a quorum. It is one device — on a factory floor, in orbit, or in a teenager's hand — proving that something was created or measured by someone or something, at some moment, and that this fact cannot be taken away.

3.6 Industrial Scale Deployment: From One Phone to One Million Sensors

Section 3.5 shows the protocol at its smallest: one teenager, one phone, one poem. This section shows the same architecture at its largest: a factory with a million sensors generating billions of readings per day. The same cryptographic proof covers both.

Scenario: Samsung semiconductor fabrication plant

A single fabrication facility operates 10,000 sensors — vibration monitors on bearings, temperature probes in clean rooms, pressure gauges on gas lines, optical sensors on wafer alignments. Each sensor generates one reading per second.

The numbers:

$10,000 \text{ sensors} \times 1 \text{ reading/second} \times 86,400 \text{ seconds/day} = 864,000,000 \text{ readings/day}$

Without batch signing (individual ML-DSA-65 (FIPS 204, "Dilithium3") signatures):

$864\text{M readings} \times 3,309 \text{ bytes per signature} = 2.86 \text{ TB/day in signatures alone}$

With Profile C batch signing (1,000 readings per batch):

$864\text{M readings} \div 1,000 \text{ per batch} = 864,000 \text{ batch signatures/day}$

$864\text{K batches} \times 3,309 \text{ bytes} = 2.86 \text{ GB/day in signatures}$

Compression ratio: 1,000:1

Architecture:

Factory Floor: 10,000 sensors (Tier 0, Profile C)

- └─ Vibration sensors → HMAC → Gateway 1
- └─ Temperature probes → HMAC → Gateway 2
- └─ Pressure gauges → HMAC → Gateway 3
- └─ Optical sensors → HMAC → Gateway 4

Gateways (Tier 0, Profile A): batch-sign 1,000 readings

- └─ 4 gateways → 864K batch signatures/day
- └─ Each batch = one ValidationRecord on the DAM

Factory AI (Tier 2): pattern analysis, anomaly detection

- └─ Runs Cognitive Continuity Chain (~30 checkpoints/day)

— Generates reasoning trails for quality decisions — Local DAG: complete factory history	
Enterprise Mesh (Tier 3): cross-factory coordination	
— 100 factories → private network (Section 10.6) — Inter-factory anomaly correlation — Optional: Network Publication (Section 10.6.3)	

Why this works:

1. **Sensors don't run PQC.** A \$4 vibration sensor sends HMAC-authenticated readings to a trusted gateway over CAN bus. The gateway does the cryptography. Profile C (Section 4.6) was designed for exactly this.
2. **Batch signing collapses overhead by 1,000x.** Instead of 2.85 TB of signatures, the factory generates 2.85 GB — manageable on commodity hardware.
3. **The Cognitive Continuity Chain runs at the factory AI level, not the sensor level.** Sensors don't think. The factory AI thinks — it analyzes patterns, makes predictions, detects anomalies. The CCC proves that this cognitive process was unbroken: no gaps, no tampering, no silent model replacement. A Tier 2 node generates ~30 cognitive checkpoints per day, each ~3-4 KB. Negligible.
4. **The private network is free.** The entire factory operates as a private network (Section 10.6). No beats, no witnesses, no Layer 2 fees. Layer 1 is always free.
5. **The same proof.** The ValidationRecord for a bearing vibration reading and the ValidationRecord for a teenager's poem have identical cryptographic structure. The same Dilithium3 signature. The same SHA3-256 content hash. The same DAG references. The protocol does not have an "enterprise mode" and a "personal mode" — there is one mode, at every scale.

Scaling to the enterprise:

Across 100 Samsung factories worldwide:

100 factories × 864M readings/day = 86.4 billion readings/day
 100 factories × 864K batches/day = 86.4M batch signatures/day
 Storage: 86.4M × 3,309 bytes = ~286 GB/day in signatures

That arithmetic projects to 286 GB/day of cryptographic signatures across 100 factories, validating 86.4 billion sensor readings — on commodity hardware, with post-quantum security and no blockchain fees. These figures are a worked projection of the Profile C batch-signing model, not a measured deployment: the protocol has not been run at this scale.

If Samsung later decides to publish its validation history to the public network — a Network Publication event (Section 10.6.3) — the published records integrate into the global DAM with the same trust scoring that applies to every other record. The bearing vibration readings from a Pyeongtaek fabrication line sit alongside poems from Nairobi in the same data structure, with the same cryptographic guarantees, distinguished only by their content hashes and classification levels.

3.7 Genesis and Network Bootstrap

A validation network that proves *who did what, when* must also be able to prove *how it began*. Genesis is the one state transition not justified by prior records — it is justified by a documented ceremony — and everything after it must follow from the protocol's ordinary rules. This section specifies how a blank-slate Elara network reaches normal operation: sealing, attesting, finalizing, and rewarding, with no special-case trust beyond the genesis state itself.

None of the mechanisms below is theoretical. Each exists because a clean-slate launch rehearsal of the reference implementation hit the corresponding failure live; the walls were removed one at a time, and the final rehearsal took a fresh network from empty data directories to self-funding finality in minutes, unattended.

The bootstrap circularity. In any stake-weighted BFT system, finality requires existing stake: a record settles when attestations represent a sufficient share of eligible stake. But stake operations are themselves records, and they take effect on *finalization*, not on creation — applying balance-mutating operations before finality would let any node manufacture state. On a blank network this is a deadlock: the first stake cannot finalize because no stake exists to attest it, and no stake will ever exist because the first stake cannot finalize. A network in this state can mint its genesis allocation and produce empty epoch seals indefinitely — it looks alive — but it can never finalize anything.

Genesis validator set. The resolution is the standard proof-of-stake answer, adapted to the DAM: the genesis configuration carries an explicit validator set — (identity, stake) pairs — applied directly into the ledger at the genesis baseline, before any record flows. Genesis stake is minted-into-stake: the network's first trust is declared by the ceremony, not earned through the record path, because there is no record path yet. The application is deterministic and idempotent, and every node applies the identical set from its genesis configuration: two nodes with the same configuration produce byte-identical baseline state roots, and a node that applies a different set has, by definition, forked at birth. With at least two genesis validators staked, the first ordinary stake record has a non-creator attester pool, finality closes, and from that point forward stake changes flow exclusively through ordinary finalized records.

The first seal. Epoch sealing discovers work by scanning recent records and existing seal chains. A blank network has neither: genesis-era records are already older than the first seal window by the time the seal loop runs, and no seal chain exists to anchor discovery. The protocol therefore includes a genesis-bootstrap discovery path — a bounded, capped scan that seeds zone discovery from pre-window records while no epoch has ever been sealed — which becomes inert the moment the first seal lands. Without it, a fresh network idles at epoch zero indefinitely while reporting itself healthy.

Exactly-once finalization effects. Gossip redelivers; that is its job. Attestations for an already-settled record arrive again whenever peers re-synchronize, and a naive implementation re-fires finalization side effects — witness rewards, reputation credit, downstream events — on every redelivery, minting duplicate rewards from replayed traffic. The protocol

binds all finalization effects to a single durable edge: the record's transition from unfinalized to finalized in the persistent finalization index. However many times consensus re-derives a record's settlement, the effects fire exactly once, from whichever code path crosses that edge first.

Admission at hour zero. Sybil resistance includes an identity-age gate: peers reject direct attestation pushes from identities younger than a configured threshold, because cheap fresh identities are the raw material of attestation spam. At genesis, *every* identity is younger than the threshold — including the validators the ceremony just declared. Config-pinned genesis validators are therefore exempt from the age gate. This is not a weakening: they are the trust root the gate exists to protect. Every later identity ages in normally.

Genesis-era record visibility. Records created in the network's first minutes face a structural hazard: peers that boot moments later begin their synchronization cursors *after* those records, and a record that arrives before the ledger state it depends on can be rejected and that rejection cached as permanent. The protocol requires two complementary defenses: witnessing must sweep old unfinalized records — bounded, oldest-first — so pre-cursor records remain witnessable; and rejection caching must distinguish permanent invalidity (malformed, cryptographically unsound) from retryable prematurity (dependencies not yet applied), parking the latter for re-fetch rather than poisoning them. Without both, the network's own genesis infrastructure — its pool funding, its earliest stakes — can become permanently invisible to nodes that booted seconds too late.

Economic ignition. Witness rewards draw from the conservation pool (Section 9), and the genesis allocation seeds that pool directly, so the first finalized record can pay its witnesses. Later top-ups, where policy allows them, are ordinary authority-signed records subject to the same finalization rules as everything else.

The launch sequence. Assembled, a genesis ceremony is: (1) generate the genesis authority and validator identities; (2) fix the genesis configuration — authority, validator set, allocations, conservation-pool seed — and distribute it byte-identical to every launch node; (3) boot. The network mints its allocation, applies validator stakes at the baseline, discovers its first zones through the bootstrap path, seals, attests through its age-exempt validators, finalizes, and pays its first witness rewards from the seeded pool. After the first seal, every bootstrap mechanism is inert and the network runs on the ordinary rules alone.

Status note (honest-claims rule): the mechanisms in this section are implemented in the Rust runtime and were validated by repeated blank-slate launch rehearsals on a three-node development network (June 2026) — fresh network to self-funding finality in under four minutes, all nodes consistent. A mainnet-scale genesis (many validators, public ceremony) follows the same sequence but has not yet been exercised at that scale. ## 4. Post-Quantum Cryptography

4.1 The Quantum Timeline

Quantum computing capable of breaking current cryptographic standards (RSA-2048, ECDSA, EdDSA) is widely believed to arrive within 10–20 years, though the exact timeline remains highly uncertain and subject to ongoing

debate. NIST finalized its first post-quantum standards in August 2024, signaling that the migration is no longer theoretical.

The threat model is “harvest now, decrypt later”: adversaries can collect encrypted data and signed records today, then break the cryptography retroactively when quantum computers become available. For a protocol designed to produce proofs that outlive the protocol itself, this is not a future concern — it is a present design requirement.

4.2 Cryptographic Primitives

The Elara Protocol uses NIST-standardized post-quantum algorithms across **all** cryptographic surfaces — signatures, key exchange, randomness, zero-knowledge, hashes, AEAD, and key-derivation. The protocol is PQ-uniform: there is no classical-public-key primitive on any path that touches mainnet records, attestations, seals, transport, or proofs.

Signatures

CRYSTALS-Dilithium (ML-DSA, FIPS 204) — Digital signatures - Used for: signing validation records, authenticating node identity - Security basis: Module Lattice-Based Digital Signature (ML-DSA-65, NIST Security Level 3) - Signature size: ~3.3 KB (3,309 bytes — the FIPS 204 final ML-DSA-65 value the implementation ships and enforces; the legacy liboqs Round-3 length of 3,293 bytes is rejected) - Signing speed: ~1 ms on the shipped pure-Rust implementation (measured in-tree, desktop-class release build; optimized AVX2 C implementations reach ~0.3 ms) - Selected for: balance of security, performance, and signature size

SPHINCS+ (FIPS 205, SLH-DSA) — Hash-based signatures - Used for: long-term anchor signatures, seed vault attestation, root of trust, optional dual-sig under §4.3 - Security basis: stateless hash-based signatures (SLH-DSA, no algebraic structure) - Signature size: ~35 KB (SPHINCS+-SHA2-192f / SLH-DSA-SHA2-192f) - Signing speed: ~130 ms on the shipped pure-Rust implementation (measured in-tree; verification ~7 ms) — two orders slower than Dilithium, acceptable for its low-frequency anchor / root-of-trust role - Selected for: conservative security assumptions — if lattice-based cryptography fails, hash-based signatures remain secure under minimal hash assumptions

Key encapsulation

CRYSTALS-Kyber (ML-KEM, FIPS 203) — Key encapsulation - Used for: establishing encrypted channels between nodes, session-key exchange in PQ transport (`src/network/pq_transport/`) - Security basis: Module Learning With Errors (ML-KEM-768, NIST Security Level 3) - Ciphertext size: ~1.1 KB (Kyber768) - Selected for: efficiency in key exchange, well-studied security proofs

Verifiable randomness

Dilithium3-VRF (alg = 0x11) — a post-quantum verifiable, unique, unforgeable selection function (sortition). This is **not a full RFC-9381 VRF**: it provides verifiability, uniqueness, and unforgeability, but **not output secrecy** (see security properties below). - Used for: epoch-seal entropy, per-

zone witness committee selection (Efraimidis-Spirakis stake-weighted draw), fisherman jury selection - Construction: `output = SHA3-256("elara-vrf-v1" || pk || alpha)` — a deterministic public function of the public key and input; `proof = Dilithium3` signature over output. Verification recomputes the output from `(pk, alpha)` and checks the signature against `pk`. ML-DSA signing is randomized (FIPS 204), so the output is deliberately **not** derived from the signature — the signature serves only as the unforgeable authorization proof. - Algorithm tag: `0x11` in the proof wire format (single-byte prefix) - Security properties: **uniqueness** (exactly one valid output per `(pk, alpha)`, from the deterministic hash), **verifiability** (anyone checks with `pk`), **unforgeability** (no valid proof without `sk`, under ML-DSA hardness). **Not provided: output secrecy** — the output is publicly computable from `(pk, alpha)`, so a draw is unpredictable only insofar as `alpha` carries entropy not known in advance (e.g. a prior epoch seal). The primitive is not relied on for output pseudorandomness against a holder of `pk`. - Proof size: ~3.3 KB (a Dilithium3 signature) - Legacy compatibility: algorithm tag `0x10` (EC-VRF/RFC 9381 over Ed25519) was retired 2026-03-31. The legacy EC-VRF verifier has been removed, so `0x10` proofs are detected, increment the `elara_legacy_vrf_proof_total` Prometheus counter (expected count = 0 on mainnet post-genesis), and are rejected — no feature flag re-enables them

Zero-knowledge

Implementation status — DESIGN-STAGE. The Phase-1 runtime implements **SHA3-256 commitment proofs** (`src/crypto/commitment.rs`), not zk-SNARKs or STARKs. The Groth16 and STARK constructions below are the **specified migration target** — there is no Groth16/STARK prover, verifier, or Cargo feature in the tree. See §5.3 and whitepaper §14.3.

SHA3-256 commitment proofs (IMPLEMENTED) — the Phase-1 privacy layer - Used for: balance-range proofs (PRIVATE classification), metadata-property proofs, content-commitment proofs - Construction: deterministic SHA3-256 commitments that prove a property of hidden data (e.g. `balance ≥ threshold`) without revealing the value. As `commitment.rs` states plainly, these are commitments — “not zero-knowledge, not post-quantum in the ZK sense” — a pragmatic Phase-1 stand-in for the circuits specified in §5.3 - Verifier: fail-closed (`src/crypto/zk.rs`, `src/crypto/commitment.rs`); malformed proofs are rejected

STARKs (FRI-based) — DESIGN-STAGE — the post-quantum ZK target (*transition path: see §4.4 algorithm agility*) - Would be used for: the same three proof properties, with genuine zero-knowledge + post-quantum security - Security basis: FRI (Fast Reed-Solomon Interactive Oracle Proofs), built solely on collision-resistant hashes — no trusted setup - Proof size: ~50–200 KB depending on circuit (larger than Groth16, but post-quantum) - Migration path: Groth16 (BN254 pairings, classical) is the intermediate design target; STARKs are the post-quantum endpoint. Neither a Groth16 nor a STARK prover/verifier exists in the tree today; the proof envelope reserves version bytes (`0x02` Groth16, rejected fail-closed) so a future prover can be slotted in without a wire-format break

Hashes

SHA3-256 (Keccak-f[1600] permutation, FIPS 202) - Used for: record IDs, content-hash commitments, Merkle trees (account state SMT, attestation index, Merkle-root chunk manifests in tiered storage), Dilithium3-VRF input compression, key derivation under HKDF - Security basis: 128-bit effective Grover-quantum strength; collision-resistant under standard assumptions - Selected for: hash-based primitives are the most conservative PQ surface — Grover at most square-roots brute-force attacks, no algebraic shortcut

Poseidon (over BN254 base field) — ZK-friendly hash (*DESIGN-STAGE — not in the tree*) - Would be used for: content commitments inside the design-stage Groth16 circuits (§5.3.1); SMT path hashing within those circuits - Security note: Poseidon's BN254 instantiation is itself classical (BN254 base field). It is **not present in the runtime** — the Phase-1 commitment scheme and all live hashing use SHA3-256. In the specified Groth16 design, Poseidon's classical reach would match Groth16's and retire together under §4.4; the STARK endpoint uses SHA3-based hashing throughout.

Authenticated encryption

ChaCha20-Poly1305 (RFC 8439, IETF AEAD) - Used for: PQ transport AEAD layer (crates/elara-pq-transport/src/frame.rs), session-key envelope after Kyber768 KEM - Security basis: 256-bit symmetric key, 128-bit Poly1305 tag; effective 128-bit quantum strength under Grover (NIST PQ guidance — symmetric primitives at ≥ 256 -bit keys remain quantum-acceptable)

AES-256-GCM — Symmetric AEAD (alternate) - Used for: identity-at-rest encryption (private-key file encryption with passphrase-derived KEK) - Security basis: 256-bit AES key, 128-bit GCM tag; 128-bit effective quantum strength under Grover

Key derivation

HKDF-SHA256 (RFC 5869) — Extract-then-expand KDF - Used for: deriving session keys from Kyber768 shared secrets; per-channel subkey derivation in PQ transport - Security basis: pseudorandomness from underlying SHA-256 (or SHA3-256 in Elara's instantiation); 128-bit effective quantum strength

Argon2id (RFC 9106) — Memory-hard passphrase KDF - Used for: deriving identity-encryption KEK from operator passphrase, seed-vault unlock - Security basis: memory-hard hash; not a public-key primitive — quantum exposure equivalent to its hash core (acceptable under NIST PQ guidance for symmetric KDFs)

NOT USED — classical primitives explicitly excluded from the runtime path

Every primitive below was considered and rejected for the mainnet node path. There is no Groth16/BN254/pairing implementation in the node (no such crate, module, or Cargo feature); a `grep -rn`

"ed25519|secp256k1|p256|ecdsa|bn254|bls12|pairing" src/ returns only comment strings documenting the rationale below and reserved version-

byte constants — not live classical public-key code. (The optional `verify-cli` tool is the one exception: it pulls `drand-verify` for BLS12-381 beacon checks, outside the node graph — see the BLS row.)

Primitive	Rationale for exclusion
Ed25519 / Ed448 (EdDSA)	Curve-based discrete-log, CRQC-breakable under Shor in polynomial time. Used historically only in <code>vrf_legacy.rs</code> (<code>alg=0x10</code>), retired 2026-03-31. No new records, attestations, seals, or proofs emit Ed25519.
ECDSA secp256k1 (Bitcoin/Ethereum signature curve)	Same Shor exposure as Ed25519. Never used in Elara runtime. Mentioned in OpenTimestamps anchoring (§ companion docs) for whitepaper prior-art only — not a runtime dependency.
ECDSA P-256 / NIST curves	Same Shor exposure. Excluded from PQ transport, signature, and KEM paths.
RSA (any modulus size)	Shor breaks RSA in polynomial time on a sufficiently capable CRQC. No Elara path uses RSA; PQ transport uses Kyber768 KEM, signatures use Dilithium3/SPHINCS+.
BN254 / BLS12-381 pairings	Pairing-based curves are CRQC-breakable, so they are excluded from the node consensus, transport, and signature paths. BN254 appears only in the design-stage Groth16 construction (§5.3) — no BN254/pairing code is in the tree . BLS12-381 is used only by the optional <code>verify-cli</code> offline tool (<code>drand-verify</code> , to check <code>drand</code> randomness-beacon signatures on anchors), never by the node.
BLS threshold signatures	Pairing-based threshold cryptography is CRQC-breakable. Phase 5.1 (cryptographically-blind mempool) is locked to lattice-based or hash-based threshold schemes — see §4F.3 decision 2026-04-19. No BLS threshold on the mainnet path, ever.
ECDH (any curve)	Replaced by Kyber768 KEM for key agreement. PQ transport never falls back to ECDH. Not used as a <i>standalone</i> classical key-exchange and never for

NaCl / standalone X25519

signatures. x25519-dalek is a dependency — but only as the classical half of the **hybrid ML-KEM-768 + X25519** transport key-exchange (defence-in-depth: the session key stays secret unless *both* the PQ and classical halves break). It is never a classical fallback.

Coverage summary

Cryptographic surface	Primitive	Standard	Quantum status
Record signatures	Dilithium3	ML-DSA, FIPS 204	PQ
Anchor / dual-sig	SPHINCS+	SLH-DSA, FIPS 205	PQ
Session-key exchange	Kyber768	ML-KEM, FIPS 203	PQ
Verifiable randomness	Dilithium3-VRF	derived from FIPS 204	PQ
Zero-knowledge proofs	SHA3-256 commitments (Phase-1); STARKs are the design-stage target (§5.3)	SHA3 commitments / FRI (design-stage)	Hash-based (PQ-acceptable); genuine ZK is design-stage
Hashes & Merkle	SHA3-256	FIPS 202	PQ-acceptable (128-bit Grover)
AEAD (transport)	ChaCha20-Poly1305	RFC 8439	PQ-acceptable (128-bit Grover; 256-bit key)
AEAD (at-rest)	AES-256-GCM	NIST	PQ-acceptable (128-bit Grover)
KDF (session)	HKDF-SHA256	RFC 5869	PQ-acceptable
KDF (passphrase)	Argon2id	RFC 9106	PQ-acceptable (hash-based)

The protocol is PQ across signatures, KEM, VRF, hashes, AEAD, and KDFs. The session-key exchange is a **hybrid** ML-KEM-768 + X25519 construction — the classical half is defence-in-depth (the session key stays secret unless *both* halves break), never a classical fallback or opt-out. The one designed-but-not-yet-PQ-ZK surface is the privacy layer: Phase-1 uses SHA3-256 commitments (hash-based, PQ-acceptable), with genuine post-quantum STARKs as the specified target (§5.3, §14.3). The only legacy surface is the rejection of pre-2026-03-31 EC-VRF (0x10) proofs.

4.3 Dual-Signature Strategy

Critical validation records (anchor attestations, identity registrations, governance votes) carry dual signatures:

- 1. **Primary:** ML-DSA (FIPS 204, formerly CRYSTALS-Dilithium) (fast, compact)
- 2. **Secondary:** SPHINCS+ (conservative, hash-based)

This provides defense-in-depth against cryptographic breakthroughs. Dilithium (lattice-based) and SPHINCS+ (hash-based) rely on fundamentally different mathematical assumptions — lattice problems and hash function preimage resistance, respectively. Breaking one does not weaken the other. If lattice-based cryptography falls to an unforeseen advance, the hash-based signature remains valid; if hash functions are weakened, the lattice signature still holds. Both must be broken simultaneously to forge a dual-signed record. The protocol’s trust model degrades gracefully rather than failing catastrophically.

4.4 Algorithm Agility

The Elara Protocol does not hardcode cryptographic algorithms. Every signature and key exchange specifies its algorithm identifier:

```
signature {
  algorithm: "dilithium3"
  value: <bytes>
}
```

When new algorithms are standardized or existing ones are deprecated, the protocol can migrate without structural changes. Old records remain valid under their original algorithms; new records use updated algorithms. The DAG preserves the full cryptographic history.

This agility is a core survival mechanism. A protocol that hardcodes today’s best cryptography is guaranteed to become insecure. A protocol that specifies algorithms by identifier can evolve with the field.

4.5 Comparison with Existing Systems

System Type	Typical Signature Algorithm	Quantum-Safe	Migration Status
PoW blockchains	ECDSA (secp256k1)	No	Not started
Smart contract platforms	ECDSA (secp256k1)	No	Research phase
High-throughput ledgers	Ed25519	No	Not started
DAG-based systems	Ed25519 / hash-based OTS	Partial	In progress
Elara Protocol	ML-DSA (FIPS 204, “Dilithium”) (all profiles) + SPHINCS+ (Profile A)	Yes	Native

The Elara Protocol’s signature layer is post-quantum from genesis — there is no legacy migration burden. However, algorithm agility (Section 4.4) ensures the protocol can adopt future PQC standards as the field evolves;

“quantum-safe at launch” is not the same as “cryptographically final.” (The ZKP layer uses classical elliptic curves in Phase 1 — see Section 11.26 for the quantum migration path.)

4.6 PQC Size Penalty and Constrained Device Strategy

Post-quantum cryptography provides stronger security at a measurable cost in size:

Algorithm	Key Size	Signature Size	Classical Equivalent
CRYSTALS-Dilithium3	1,952 bytes	3,309 bytes	ECDSA: 33 + 72 bytes
SPHINCS+-SHA2-192f	48 bytes	35,664 bytes	Ed25519: 32 + 64 bytes
CRYSTALS-Kyber768	1,184 bytes	1,088 bytes	X25519: 32 bytes

Dilithium signatures are **~46x larger** than ECDSA signatures (3,309 vs ~72 bytes). For a datacenter or laptop, this is negligible. For an ESP32 sending thousands of signed readings over LoRa (max payload ~242 bytes), it is prohibitive.

Solution: Tiered Cryptographic Profiles

The protocol defines three cryptographic profiles that devices select based on their capabilities:

Profile A: Full PQC (default) - Dilithium3 signatures, Kyber768 key exchange, SPHINCS+ for anchoring - For: servers, laptops, phones, gateways
- Signature overhead: ~3.3 KB per record

Profile B: Compact PQC - Dilithium3 (same parameter set as Profile A: 3,309 byte signatures, NIST Level 3) - No dual signatures (Dilithium only, no SPHINCS+) - For: Raspberry Pi, industrial controllers, modern IoT gateways
- Signature overhead: ~3.3 KB per record (identical to Profile A primary signature)

Profile C: Gateway-Delegated Signing - Constrained device (ESP32) sends unsigned readings to a trusted gateway via secure local channel (BLE, CAN, wired) - Gateway batches readings and signs the batch with Profile A or B - Device authenticates to gateway using lightweight symmetric key (pre-shared, established at provisioning) - For: \$4 microcontrollers, ultra-low-power sensors - Per-reading overhead on device: ~32 bytes (HMAC) - Per-batch overhead on network: ~3.3 KB (one Dilithium signature per batch of hundreds/thousands of readings)

Profile C is a pragmatic compromise: the constrained device cannot run PQC itself, but its readings are still validated on the DAM through a trusted gateway. The trust boundary shifts from the device to the gateway — acceptable for IoT deployments where the gateway is physically secured alongside the sensors.

All three profiles produce validation records that are interoperable on the DAM. The profile is specified in the record metadata, so verifiers know which security level applies.

Profile B Security Boundary (v0.7.1 clarification). Profile B is Profile A minus SPHINCS+ — it uses the same ML-DSA-65 (Dilithium3, NIST Level 3) primary signature but omits the SLH-DSA secondary signature. Under the quantum adversary model of §11.12 / §12.1, Profile B records become forgeable if ML-DSA-65 is broken by a quantum adversary — unlike Profile A records which remain secure via the independent SPHINCS+ signature. Consequently, Profile B identities are treated as lower-trust for consensus purposes. The protocol recommends: (a) transfer limits for Profile B identities (e.g., max 1,000 beats per transaction), (b) settlement requires a minimum fraction of Profile A attestations (e.g., ≥50% of attesting stake from Profile A witnesses), and (c) high-value operations (staking >10K beats, governance votes) require Profile A identity.

Future PQC Size Reduction: NIST Additional Signatures Project

The PQC size penalty described above reflects the first generation of NIST-standardized post-quantum signatures. This is not the final generation. In November 2024, NIST announced the **Post-Quantum Cryptography: Additional Digital Signature Schemes** project, accepting ~50 submissions for evaluation. Several candidates offer dramatically smaller signatures than Dilithium:

Candidate	Signature Size	vs. Dilithium3 (3,309 B)	Basis
SQIsign	~204 bytes	16x smaller	Supersingular isogenies
HAWK	~555 bytes	6x smaller	Lattice (NTRU)
UOV (variants)	~96–128 bytes	25-34x smaller	Multivariate

These are candidates, not standards — NIST evaluation will take years, with standardization likely in 2027-2028 at the earliest. Signing performance varies (SQIsign is significantly slower than Dilithium), and security assumptions for some candidates are less studied.

The Elara Protocol’s **algorithm agility** (Section 4.4) means that adoption of compact PQC signatures is a configuration change, not a protocol redesign. When NIST standardizes a compact alternative:

1. New algorithm identifier added to the protocol via governance vote
2. Transition period: both Dilithium and the new algorithm accepted
3. New records use the compact algorithm; old records remain valid under Dilithium
4. Profile B and C devices benefit most — a 200-byte signature eliminates the size penalty that drove the Profile C delegation model

The current 46x size penalty over classical signatures is a first-generation cost, not a permanent constraint. The protocol is designed to absorb future improvements without structural change.

4.7 Post-Quantum Transport Layer

The Elara Protocol’s transport between nodes is **post-quantum by default and post-quantum only on mainnet**. Classical TLS is not an alternate path, a fallback, or a deployment option — it is absent from the wire protocol on mainnet. Every byte that crosses the wire between two Elara nodes (gossip pushes, sync pulls, RPC calls, admin API, WebSocket streams) rides the ElaraPQ transport described below.

This section is normative. Implementations that ship classical TLS (rustls, OpenSSL, BoringSSL, native_tls) on a mainnet node-to-node interface are non-conformant.

4.7.1 The Hybrid Handshake

ElaraPQ uses a three-message hybrid handshake combining classical Curve25519 with the NIST-standardized ML-KEM-768 (FIPS 203). The handshake is:

```
msg1: initiator → responder
  ELPQ_MAGIC(4) | WIRE_VERSION(1) | timestamp(8) |
  initiator_dilithium3_pk(1952) | initiator_x25519_pk(32) |
  initiator_kyber768_ct(1088) |
  initiator_dilithium3_sig_over_transcript(3309)
  = 6394 bytes

msg2: responder → initiator
  responder_x25519_pk(32) | responder_kyber768_ct(1088) |
  responder_dilithium3_sig_over_transcript(3309) | aead_tag(16)
  = 4445 bytes

msg3: initiator → responder
  aead_handshake_finished(48)
  = 48 bytes
```

The session key is derived as:

```
shared_x25519      = X25519(initiator_x25519_sk, responder_x25519_pk)
shared_kyber768    = ML-KEM-768.Decapsulate(responder_kyber768_ct,
sk)
session_key        = HKDF-SHA256(salt = transcript_hash,
                                ikm  = shared_x25519 ||
shared_kyber768,
                                info = "elara-pq-session-v1",
                                len  = 32)
```

The session key feeds ChaCha20-Poly1305 AEAD for all subsequent frames. The transcript signature binds both peers to the full handshake under their long-term ML-DSA-65 (FIPS 204, “Dilithium3”) identity keys, preventing transcript-substitution attacks. The hybrid construction means a successful attack must break both X25519 (classical, trivially broken by Shor’s algorithm) **and** ML-KEM-768 (post-quantum, lattice-based, currently no known attack) — the protocol fails open only if both substrates fail.

Constants are normative:

Constant	Value	Source
ELPQ_MAGIC	b"ELPQ"	crates/elara-pq-transport/src/frame.rs:23
WIRE_VERSION	0x01	crates/elara-pq-transport/src/frame.rs:27 crates/elara-pq-

MAX_HANDSHAKE_SKEW_SECS	30	transport/src/handshake.rs:45
DEFAULT_HANDSHAKE_TIMEOUT	10s	src/network/pq_transport/stream.
	4 MiB	
MAX_FRAME	after	frame.rs
	AEAD	

4.7.2 ML-KEM-768 as a Transport-Layer Requirement

ML-KEM-768 (FIPS 203, NIST Security Level 3) is **not optional**. It is a wire-protocol-level requirement, on the same footing as Dilithium3 for record signatures. A node that does not implement ML-KEM-768 cannot speak the ElaraPQ transport and therefore cannot peer with any mainnet node.

This is stricter than §4.2’s framing of cryptographic primitives, because §4.2 lists *what algorithms the protocol uses* whereas this section lists *what algorithms a conformant implementation must provide*. The two lists overlap fully today, but as the protocol absorbs new primitives via the algorithm-agility mechanism (§4.4) the transport layer’s requirements may evolve faster than the broader primitive set — for example, if a future ML-KEM-1024 variant becomes mandatory for transport while ML-KEM-768 remains accepted for at-rest record keying.

4.7.3 No Classical Transport Fallback

The protocol does not define a “classical-only” transport mode. Implementations are forbidden from offering one on mainnet. Specifically:

- HTTPS over TLS 1.3 with classical KEM (X25519, P-256, RSA) — forbidden as a node-to-node transport on mainnet.
- HTTPS over TLS 1.3 with hybrid KEM (X25519+ML-KEM-768) negotiated by IETF draft-ietf-tls-hybrid-design — forbidden, because the draft is not yet a standard and Elara does not pin to any in-flight standardization process.
- QUIC with the same primitives — forbidden on the same grounds.
- Plaintext UDP (any form) — forbidden.

The protocol does permit a *bootstrap* exception (§11.14): light clients on first install retrieve a foundation-signed seed-peer list from a single foundation-operated HTTPS origin, used exactly once. After first contact, all subsequent traffic uses ElaraPQ.

Implementations that wish to integrate with non-Elara IoT or web infrastructure (MQTT bridges, CoAP gateways, HTTP REST APIs documented in §8.3) may use classical transports for that integration boundary. Those classical transports terminate at the gateway; the gateway then signs validation records with the device’s PQ identity (Profile C, §4.6) and pushes them onto the DAM via ElaraPQ. The classical surface is a non-protocol boundary — outside the scope of this section.

4.7.4 Pluggable Transports for Censored Networks

For deployments in jurisdictions that block direct ElaraPQ traffic, the protocol supports tunneling ElaraPQ frames inside other transports (Tor pluggable transports, WireGuard, Tailscale, SSH port-forwarding). The ElaraPQ handshake and AEAD remain unchanged; the outer wrapper is opaque to the protocol.

What the protocol does **not** do: define a “domain-fronting mode” that masquerades as classical HTTPS to fool deep-packet-inspection middleboxes. Earlier drafts of this section described domain fronting as a censorship-resistance feature; that language is retired. Domain-fronting compromises the cryptographic transcript by accepting classical TLS framing on the outer layer, which leaks per-connection metadata (TLS ClientHello fingerprints, SNI when not encrypted via ECH, certificate chain timing) that defeat the transport’s post-quantum forward-secrecy goal. Operators who need DPI-bypass should use Tor, Snowflake, or obfuscated VPNs as the carrier — not bake classical TLS into the protocol.

4.7.5 Compliance Verification

A mainnet node operator can verify their deployment matches §4.7 by:

1. `ss -tlnp` shows only the ElaraPQ port bound on public interfaces (no port 443 / 9473 HTTPS listener).
2. `tcpdump -i any -w pcap` followed by `elara-capture-audit pcap` returns $\geq 99.9\%$ sampled-payload ELPQ_MAGIC and zero `0x16 0x03 0x0[1234]` (TLS ClientHello) bytes on public interfaces.
3. `grep -rn "rustls\|TlsAcceptor\|tokio-rustls" src/` returns zero hits in the deployed binary’s source.
4. The compiled binary’s dependency graph (`cargo tree --features node`) lists no `rustls`, `tokio-rustls`, `rustls-pemfile`, `rustls-pki-types`, `rcgen`, `hyper`, or `hyper-util` as direct or transitive dependencies on the mainnet build profile.

These four checks are the operator-facing acceptance gates for §4.7 compliance.

5. Zero-Knowledge Validation

5.1 The Privacy Paradox

Validation and privacy are traditionally in tension. To prove you created something, you must reveal what you created. This is acceptable for open-source code or public art, but unacceptable for:

- Trade secrets under development
- Medical data from IoT health devices
- Military or intelligence sensor readings
- Unreleased creative work
- Corporate R&D
- Personal journals or private communications

The Elara Protocol specifies zero-knowledge proofs (ZKPs) as the target privacy layer: cryptographic constructions that prove a statement is true without revealing the underlying data. Phase 1 ships SHA3-256 hash

commitments as a stand-in (not genuine zero-knowledge — see §5.3); the zk-SNARK constructions described below are design-stage.

5.2 Classification Levels

Every validation record carries a classification level that determines what the network can see:

PUBLIC — Full content hash visible. Anyone can verify the exact content. Default for open-source code, published work, public sensor data.

PRIVATE — Content hash wrapped in a SHA3-256 commitment (Phase-1 stand-in; genuine zero-knowledge proofs are design-stage — see §5.3). The network validates that: - A valid content hash exists - It was signed by a valid keypair - The timestamp is consistent with the DAG - No conflicting claim exists

...without learning what the content is. The creator can selectively reveal the content later (e.g., in a legal dispute or patent filing) by providing the pre-image.

RESTRICTED — Key-group access. The content hash is encrypted to a set of public keys. Only designated parties can verify the content. The network validates the structural integrity of the record without accessing the encrypted payload.

SOVEREIGN — Maximum privacy. Multi-key authorization required for any access. Time-locked release optional. Validator nodes process the proof without any visibility into the content or the classification metadata. (SOVEREIGN additionally *specifies* wrapping the creator’s identity in a ZKP — design-stage; in Phase 1 identity is bound by the record’s ML-DSA-65 (FIPS 204, “Dilithium3”) signature. See §5.3.)

5.3 ZKP Construction

Implementation status — DESIGN-STAGE (not in the Phase-1 runtime). The Phase-1 runtime implements **SHA3-256 commitment proofs** (`src/crypto/commitment.rs`, whose own doc-comment notes they are “not zero-knowledge, not post-quantum in the ZK sense”): the three proof types below are implemented as hash commitments, **not** as zk-SNARK circuits. The Groth16-on-BN254 construction this section specifies is the **migration target** — there is no Groth16 prover, verifier, trusted setup, or Cargo feature in the tree. See whitepaper §14.3 for the honest gap assessment. The status legend used throughout §5.3 and §4.2: **IMPLEMENTED** (code exists, tested), **DESIGN-STAGE** (specified, not built), **PLANNED** (intended).

The Elara Protocol **specifies** Groth16 zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) on the BN254 curve for PRIVATE and RESTRICTED classifications. Three proof types are specified below — and implemented in Phase-1 as SHA3-256 commitments:

5.3.1 Circuit Types

Type	Public Inputs	Private Witness	Proves
------	---------------	-----------------	--------

BalanceRange	threshold	balance, excess	“I have $\geq$ threshold beats” without revealing exact balance
MetadataProperty	key_hash, commitment	value_hash, salt	“Metadata key has a specific value” without revealing it
ContentCommitment	commitment	content_hash_fr, blinding_factor	“I know content behind this commitment”

The proof *semantics* above are implemented in Phase-1 as SHA3-256 commitments (`src/crypto/commitment.rs`). The R1CS circuit specifications that follow are the **DESIGN-STAGE** Groth16 target — no R1CS, Poseidon, or BN254 code exists in the tree.

BalanceRange circuit (R1CS, design-stage): Would prove $\text{balance} \geq \text{threshold}$ by decomposing $\text{excess} = \text{balance} - \text{threshold}$ into 64 bits, constraining all higher bits to zero. This implicitly constrains excess to $[0, 2^{64})$.

MetadataProperty circuit (R1CS, design-stage): Algebraic commitment $\text{commitment} = \text{key_hash} + \text{value_hash} \times \text{salt}$. Demonstrates the R1CS structure for property commitments.

ContentCommitment circuit (R1CS with Poseidon hash, design-stage): The primary ZK-friendly circuit in the target design. Would use Poseidon hash inside R1CS constraints:

```
Public inputs:  commitment
Private witness: content_hash_fr, blinding_factor
Constraint:     commitment == Poseidon(content_hash_fr,
blinding_factor)
```

Poseidon parameters for BN254: rate=2, capacity=1, alpha=17, 8 full rounds + 57 partial rounds (standard security parameters from the Poseidon specification).

5.3.2 Proof Format

Implementation status — DESIGN-STAGE. The Phase-1 runtime emits **SHA3-256 commitment proofs** (`src/crypto/commitment.rs`); the Groth16 format below is reserved (version byte 0x02) but is neither produced nor verified.

Phase-1 (IMPLEMENTED): SHA3-256 commitment proofs. The wire format carries a version/type discriminator, the 32-byte commitment, the public inputs, and the commitment proof data; `src/crypto/commitment.rs` is the byte-level source of truth (version byte 0x03). Groth16-format bytes (version 0x02) are **rejected fail-closed** at ingest, since no Groth16 verifier exists.

Specified Groth16 format (DESIGN-STAGE, not implemented): 2 G1 points + 1 G2 point $\approx$ 192 bytes compressed on BN254, verifiable in $\sim$ 1 ms on commodity hardware once a prover lands. Version byte 0x02 is reserved for this future format.

5.3.3 PQC + ZK Orthogonal Composition

ZK proofs and post-quantum signatures serve orthogonal purposes and compose naturally:

- **ZK proof = privacy** — hides content while proving properties about it
- **PQC signature = authenticity** — proves the record was created by a specific identity

ML-DSA-65 (FIPS 204, “Dilithium3”) signature verification inside R1CS is infeasible: lattice operations produce millions of constraints, making proof generation take minutes and CRS generation hours. The pragmatic split keeps both properties without combining them inside a single circuit. A PRIVATE record carries both: a privacy proof (Phase 1: a SHA3-256 commitment; design-stage: a ZK proof — verifiable without learning the content) and a PQC signature (anyone can verify who created it).

5.3.4 Trusted Setup

Implementation status — DESIGN-STAGE. No CRS exists in the runtime. The Phase-1 SHA3-256 commitment scheme requires **no trusted setup**; the setup below applies only to the design-stage Groth16 construction.

The Groth16 construction would require a CRS (Common Reference String). For a future testnet of that construction, the CRS would be generated deterministically from a seed:

```
seed = SHA3-256("elara-protocol-groth16-crs-v2")
```

For mainnet, a multi-party computation (MPC) ceremony would produce the CRS. Security requires ≥ 1 honest participant who destroys their randomness. See Section 11.11 for the ceremony process and migration path to transparent proof systems (STARKs).

5.3.5 Gossip Verification

ZK proof verification dispatches on the first byte of `zk_proof`. In the Phase-1 runtime: SHA3-256 commitment proofs (version 0x03, and legacy 0x01) are verified by hash recomputation (`src/crypto/commitment.rs`, `src/crypto/zk.rs`); Groth16-format bytes (version 0x02) are **rejected fail-closed** at ingest, since no Groth16 verifier exists. (In the design-stage Groth16 construction, 0x02 proofs would be verified against the CRS loaded at node startup.) WASM browser nodes relay proofs without verifying — native witness nodes verify before consensus acceptance.

(Note: the BN254 curve in the design-stage construction provides $\sim$ 100-bit security against classical attacks and is NOT post-quantum. See Section 14.3 for the migration timeline to post-quantum ZKP constructions.)

For SOVEREIGN classification, the protocol specifies zk-STARKs (Scalable Transparent Arguments of Knowledge) as a future extension — larger proofs (~100 KB) but no trusted setup required.

5.4 Selective Disclosure

A creator who validates work as PRIVATE can later:

1. **Reveal to specific parties** — provide the content hash and blinding factor to a court, patent office, or business partner. They can verify it matches the on-chain commitment.
2. **Upgrade to PUBLIC** — publish the content hash, making the validation fully transparent. The DAG timestamp proves the work existed at the original validation time.
3. **Maintain privacy indefinitely** — the zero-knowledge proof is sufficient for priority claims without ever revealing the content.

This streamlines a workflow that was previously impractical at scale: validate now, decide on disclosure later. A researcher can timestamp a discovery, continue working in private, and prove priority years later if needed.

6. Identity and Attribution

6.1 Self-Sovereign Identity

Every entity on the Elara Protocol generates its own cryptographic keypair. No central authority issues, approves, or revokes identities. This is a fundamental departure from traditional identity systems:

- **X.509 certificates** require a Certificate Authority hierarchy
- **DNS** requires ICANN and registrars
- **Government ID** requires citizenship and bureaucracy
- **OAuth** requires a third-party platform

An Elara identity requires only entropy and computation — available on any device, in any jurisdiction, on any planet.

Identity Structure

```
ElaraIdentity {
  public_key:      ML-DSA (FIPS 204, formerly CRYSTALS-Dilithium)
  public key
  identity_hash:   SHA3-256(public_key) // short identifier
  created:         timestamp
  entity_type:     HUMAN | AI | DEVICE | ORGANIZATION | COMPOSITE
  metadata:        optional, creator-defined, signed
  succession:      optional, designated heir public keys
  revocation:      self-revocation mechanism (signed revocation
record)
}
```

The identity hash serves as a short, human-communicable identifier (similar to a fingerprint in PGP). The full public key is used for cryptographic operations.

6.2 Entity Types

The protocol recognizes five entity types, each with the same cryptographic standing:

HUMAN — Individual creators. One or more keypairs per person (separation of personal and professional identity is supported).

AI — Artificial intelligence systems. Each AI model instance generates its own keypair. This solves the AI attribution problem: when an AI generates content, its validation record includes the AI's identity, the model version, and (optionally) the prompt that triggered the generation.

DEVICE — IoT sensors, robots, vehicles, satellites. Capable devices (Raspberry Pi and above) generate a keypair at first boot and sign readings directly (Profile A/B). Constrained devices (\$4 ESP32) authenticate to a local gateway via pre-shared symmetric key; the gateway signs batches on their behalf (Profile C, Section 4.6). Device identity persistence across physical resets — including hardware-bound keys, organizational binding, and behavioral fingerprinting — is addressed in Section 11.33.

ORGANIZATION — Companies, research labs, governments. Organizational identities can designate authorized signers (multi-signature schemes).

COMPOSITE — Human-AI collaborations. A composite identity explicitly records the relationship: who prompted, who generated, who edited, who approved. This creates an unambiguous attribution chain that courts, patent offices, and licensing systems can interpret.

6.3 AI Attribution in Detail

The rise of AI-generated content has created an attribution crisis. Current systems cannot distinguish between: - Fully human-created work - AI-assisted work (human-directed, AI-generated) - Fully AI-generated work - AI-to-AI generated work (one model's output fed to another)

The Elara Protocol makes this explicit:

```
CollaborationRecord {
  work_hash:      content hash of the final output
  participants:   [
    { identity: human_key, role: "prompter", contribution:
"direction, editing" },
    { identity: ai_key,   role: "generator", contribution:
"initial draft",
      model: "claude-opus-4-8", version: "2026-06" }
  ]
  chain:          optional, references to intermediate outputs
  signed_by:      all participants
}
```

This record is immutable on the DAG. When a dispute arises about who created what, the cryptographic evidence is already in place.

6.4 Digital Succession

Physical death should not orphan digital work. The Elara Protocol supports:

- **Designated heirs** — public keys listed in the identity’s succession field. Upon activation (by the heir providing a signed succession claim), the heir gains read access to the creator’s PRIVATE and RESTRICTED work.
- **Time-locked release** — content becomes PUBLIC after a specified duration (e.g., 70 years, matching current copyright terms, or any custom period).
- **Dead man’s switch** — if a node does not produce a heartbeat within a configured period, succession activates automatically.

The cryptography enforces the creator’s wishes automatically, reducing dependence on legal proceedings for routine succession. Courts retain authority for contested cases, but the protocol provides the evidentiary foundation.

7. Interplanetary Operations

Note: This section describes theoretical protocol behavior under interplanetary communication constraints. The physics are documented; the protocol’s response to them is specified but untested. No implementation or simulation has been conducted.

7.1 The Latency Problem

Communication delays in the solar system are not engineering failures — they are physics:

Route	One-Way Delay	Round-Trip
Earth surface	< 100 ms	< 200 ms
Earth-Moon	1.3 s	2.6 s
Earth-Mars (closest)	3 min	6 min
Earth-Mars (farthest)	22 min	44 min
Earth-Jupiter	33–54 min	66–108 min

No consensus mechanism that requires real-time communication can operate across these delays. A proof-of-work block time of ~10 minutes is barely acceptable for Earth-Moon; it is unusable for Earth-Mars.

7.2 Time Without Clocks

Speed-of-light delays make clock synchronization impractical across planetary distances. While relativistic time dilation between Earth and Mars is negligible at protocol timescales (microseconds), the fundamental issue is communication latency: a round-trip to Mars takes 6–44 minutes, making any synchronization-dependent protocol unworkable. The Elara Protocol does not rely on synchronized clocks. Instead, it uses:

Local timestamps — each node records its local time in its validation records. These are informational, not authoritative.

Vector clocks — logical counters that track causal ordering. If node A’s validation references node B’s validation, then A happened after B, regardless of wall-clock times. Vector clocks establish partial ordering

without requiring clock synchronization.

Causal references — every validation record lists its parent records in the DAG. This creates an unambiguous “happened before” relationship. The DAG IS the clock.

Ordering rule: if A references B, A happened after B. If A and B have no causal relationship, they are concurrent — and the protocol does not force an artificial ordering.

Conflict resolution for concurrent records: When two causally unrelated validation records make incompatible claims (e.g., two entities claim authorship of semantically identical work within a short time window), the protocol preserves both records and applies the following resolution strategy:

1. **Preservation** — both records remain in the DAM permanently. Neither is deleted or hidden.
2. **Annotation** — a ConflictSet record is created linking the conflicting validations, with metadata including detection timestamp, similarity score, and conflict type.
3. **Evidence accumulation** — subsequent witnesses can attest to either or both records. The trust scores evolve independently.
4. **No automatic winner** — the protocol does not algorithmically determine which claim is “correct.” Priority disputes involve human judgment, legal context, and evidence that a cryptographic protocol cannot evaluate.
5. **Query transparency** — any query that returns a conflicted record also returns the ConflictSet, ensuring consumers are aware of the dispute.

This design reflects a core principle: a validation protocol proves *possession at a point in time*, not *original creation*. Resolving authorship disputes is a human problem that the protocol supports with evidence but does not attempt to automate.

7.3 Partition Tolerance

Network partitions are not failures in the Elara Protocol — they are expected operating conditions. A partition occurs when:

- A solar flare disrupts Earth-Mars communication for days
- A submarine enters radio silence
- A remote sensor network loses satellite uplink
- Political conflict severs internet connectivity between regions
- A spacecraft is in transit between planets

During a partition:

1. Each zone continues operating independently
2. Local validations proceed normally (Layer 1 never requires connectivity)
3. Zone-internal consensus proceeds normally (Layer 2 within the zone)
4. The DAG branches — each zone grows its own branch

When a partition heals:

1. Zones exchange their branch tips

2. DAGs merge — all records from all zones are incorporated
3. Conflicting claims (if any) are preserved with annotations, not resolved by deletion
4. Witness counts update as the merged records propagate

This is fundamentally different from blockchain's approach to partitions, where a fork must be resolved by discarding one chain. The Elara DAG preserves both branches because both branches contain valid work by real entities.

7.4 Bandwidth Economics

Interplanetary communication is expensive. The Deep Space Network allocates bandwidth in kilobits per second. The Elara Protocol optimizes for minimal bandwidth:

Merkle roots — a single 32-byte hash summarizes millions of validation records. Zones exchange Merkle roots to detect divergence, then synchronize only the differences.

Delta sync — only new records since last synchronization are transmitted. A zone with 1 million records that has added 100 since last sync transmits only 100 records plus the updated Merkle root.

Priority sync — records are prioritized for transmission based on: - Classification level (SOVEREIGN records sync first) - Witness count (more-attested records are higher priority) - Age (recent records are more likely to be queried) - Creator priority (configurable per zone)

Bloom filters — probabilistic data structures (~10 bytes per element) that answer “does this record exist in your zone?” with a small false-positive rate. Bloom filter exchange enables efficient detection of missing records before full synchronization begins.

Compression — validation records use a compact binary encoding (not JSON or XML) with protocol-buffer-style varint encoding. A typical PUBLIC validation record is approximately 4–5 KB, dominated by the ML-DSA-65 (FIPS 204, “Dilithium3”) signature (~3.3 KB).

7.5 Zone Architecture

7.5.1 Zone Model: Semantic Subscription with Stake-Gated Consensus

Zones use hierarchical semantic paths that reflect real-world organizational and geographic structure:

```
"medical/eu/west/germany/bavaria"  
"finance/global"  
"iot/manufacturing/toyota/plant-7"  
"personal/alice"
```

Record routing is determined by the record's `zone_refs` field — records go to zones based on their purpose and context. A medical record from a Bavarian hospital naturally belongs in `medical/eu/west/germany/bavaria`.

Zone subscription is voluntary — nodes choose which zones to store and process. A hospital server subscribes to `medical/eu/west`, a Toyota factory subscribes to `iot/manufacturing/toyota/*`, a phone subscribes only to its owner's personal zone. Nodes never store records for zones they don't subscribe to, enabling each node to hold a fraction of the global dataset.

Consensus participation is stake-gated — while any node can subscribe to store records, participating as a witness (attesting to epoch seals) requires: - Minimum 100 beats staked - PoW-verified identity with `min_pow_difficulty` (currently 16 bits) - Identity age ≥ 48 hours - Diversity check: no single entity or /24 IP subnet may control $>33\%$ of a zone's total staked weight

This separates Sybil resistance from zone assignment. The earlier hash-based model (`SHA3-256(public_key) mod NUM_ZONES`) provided Sybil-resistant zone assignment but created semantically meaningless groupings — a hospital might share a zone with unrelated IoT sensors. At quintillion-record scale, zone-scoped gossip REQUIRES semantic grouping to bound bandwidth. Sybil defense now operates at the witness admission layer through the existing mechanisms (PoW, stake, age, diversity scoring) rather than at the zone assignment layer.

Zone splitting: Zones split like biological cells when they grow too large. A zone exceeding a threshold of witnesses ($>N$) or records per epoch ($>M$) can split into sub-zones. Parent zone anchor nodes authorize the split through governance (Section 10.2). The hierarchical path naturally accommodates splitting: `medical/eu` can split into `medical/eu/west` and `medical/eu/east` without restructuring.

Wire format: Zone identifiers use variable-length hierarchical paths (wire format v3), replacing the previous `u8 zone_id` which limited the network to 256 zones.

7.5.1.a Zone Transition Seals (v0.7.9+)

A zone split or merge is not a local decision. It changes the mapping from record IDs to leaf zones, and any attestation produced under the old mapping must be verifiable under the new mapping — otherwise the network forks. The protocol defines a **TransitionSeal** as the authoritative, anchor-co-signed record of a split or merge event.

A TransitionSeal is a record of kind `zone_transition` carrying:

- `transition_id` — UUIDv7 identifying the event.
- `kind` — split or merge.
- `parent_zone` — the zone being split, or for a merge, the set of zones being unified.
- `child_zones` — the resulting leaf zones. For a split, $|\text{child_zones}| \geq 2$; for a merge, $|\text{child_zones}| = 1$.
- `boundary_function` — the invariant that defines how account identities route to child zones. The canonical form is
$$\text{account_belongs_to_child}(\text{account_hash}, \text{child_zone}) = \text{SHA3}(\text{account_hash} || \text{parent_zone}) \in \text{child_range}(\text{child_zone}),$$
where each `child_zone` is allocated a contiguous range over the 2^{256} hash space. This is deterministic, stateless, and verifiable by any light client given the TransitionSeal.
- `effective_epoch` — the epoch at which the transition takes effect. All

records with $\text{timestamp} < \text{effective_epoch} \times \text{epoch_interval}$ route under the old mapping; records at or after route under the new mapping.

- proposer — the anchor node proposing the transition.
- `anchor_signatures` — an M-of-N ML-DSA-65 (FIPS 204, “Dilithium3”) multi-signature over the canonical bytes, where $M = 2/3$ of the anchor pool at proposal time, and N is the pool size. Under 2^{128} quantum attack models, M-of-N over Dilithium3 is the same security level as a single Dilithium3 signature (per §4.2), so the multi-sig is for trust distribution, not additional cryptographic strength.

Invariants. A TransitionSeal is valid iff:

1. $|\text{child_zones}| \geq 2$ (split) or $|\text{parent_zones}| \geq 2 \wedge |\text{child_zones}| = 1$ (merge).
2. $\text{anchor_signatures}.\text{len}() \geq \text{ceil}(2 \times N / 3)$.
3. Every signature in `anchor_signatures` verifies against a key in the anchor registry at $\text{effective_epoch} - \text{dispute_window}$.
4. The `boundary_function` partitions the parent’s account-hash space without overlap or gap.
5. For a merge, the merged `child_zone` absorbs the union of every parent’s records.

Dispute window. A TransitionSeal has a 3-epoch dispute window starting at `effective_epoch`. During the window, any anchor or witness can submit a counter-TransitionSeal (same `effective_epoch`, incompatible `boundary_function` or `child_zones`). If a counter-seal with $\geq M$ valid anchor signatures arrives before $\text{effective_epoch} + 3$, the original is rejected and the network continues under the pre-transition mapping. Only after the dispute window closes does the transition become part of the canonical zone registry.

Replay through attestations. Any attestation produced before `effective_epoch` attests to records under the pre-transition mapping. After `effective_epoch`, attestations are verified against the new mapping. A record created before `effective_epoch` but arriving at a peer after it is routed using a resolver: the peer walks the zone registry from the record’s timestamp forward, applying each TransitionSeal’s `boundary_function` in `effective_epoch` order. This is `resolve_current_leaf(record_id)` — $O(\log(\text{transitions}))$ and free of locks.

Light-client verification. A light client fetches TransitionSeals alongside epoch seals during header-only sync. Given any account identity and the TransitionSeal chain, a light client can deterministically compute the current leaf zone for that account without ever storing records — just apply the boundary functions in order.

Why M-of-N, not a single anchor signature. A zone split redirects future records and retroactively re-partitions attestation validity. A compromised anchor could propose a malicious split to hijack a profitable zone’s rewards. The $2/3$ -anchor threshold matches the witness-committee finality threshold and ensures no single anchor can unilaterally alter the zone topology.

7.5.2 Interplanetary Extensions (Aspirational)

As humanity expands beyond Earth, the zone model naturally extends to geographic/planetary boundaries where communication latency makes cross-zone synchronization physically constrained:

```
Zone: Earth-Primary
├─ Subzone: North America
├─ Subzone: Europe
├─ Subzone: Asia-Pacific
└─ Subzone: Africa

Zone: Luna
├─ Subzone: Artemis-Base
└─ Subzone: FarSide-Observatory

Zone: Mars
├─ Subzone: Ares-Colony-1
└─ Subzone: Orbital-Relay

Zone: Deep-Space
└─ Subzone: Voyager-Relay
```

In this model, zone assignment transitions from hash-based to latency-based: nodes physically located on Mars would join the Mars zone because cross-planet consensus with Earth nodes is impractical at 3-22 minute one-way delays. The hash-based zone model described in Section 7.5.1 would operate *within* each planetary zone, providing intra-planet shard distribution. This architecture is specified but untested — it is included to demonstrate that the protocol's partition tolerance (Section 7.3) and asynchronous design accommodate interplanetary operation without structural changes.

8. IoT and Hardware Integration

8.1 The Physical-Digital Bridge

The Elara Protocol extends validation from digital content to physical-world data through IoT device integration. Every sensor, actuator, and controller can contribute to cryptographically validated records on the DAG — either by signing directly (Profile A/B) or through gateway-delegated signing (Profile C).

8.2 Device Capabilities

Minimum viable device: ESP32 (\$4) — Profile C (Gateway-Delegated) - Authenticates to local gateway via pre-shared symmetric key (established at provisioning) - Sends readings over secure local channel (BLE, CAN, wired); gateway signs batches with PQC (Profile A/B) - Stores local readings in flash memory (circular buffer for constrained devices) - Communicates via Wi-Fi, BLE, LoRa, or CAN bus

Standard device: Raspberry Pi / industrial controller (\$35–\$200) - Full node capabilities including relay and witness roles - Local AI inference for anomaly detection (Layer 3) - Multiple communication interfaces

Gateway device: edge server (\$500+) - Aggregates readings from leaf devices - Provides network connectivity for air-gapped devices - Runs full DAG synchronization

8.3 Supported Protocols

The Elara Protocol integrates with standard IoT communication protocols:

Protocol	Use Case	Integration
MQTT	Lightweight pub/sub messaging	Signed payloads as MQTT messages
CoAP	Constrained RESTful protocol	Validation records as CoAP resources
gRPC	High-performance RPC	Native Elara service definitions
HTTP/HTTPS	IoT-device → gateway integration only (NOT node-to-node — see §4.7)	Profile C delegated signing: device posts unsigned readings to a trusted gateway over HTTPS; gateway signs with PQ identity and forwards to the DAM via ElaraPQ
BLE	Short-range device communication	Signed readings via BLE characteristics
CAN	Automotive/industrial bus	Signed frames on CAN bus
LoRa/LoRaWAN	Long-range, low-power	Compact validation records for LPWAN

8.4 Use Cases

Supply chain provenance — A coffee bean's journey from farm to cup: soil sensors sign moisture readings, GPS trackers sign location, temperature sensors sign cold-chain compliance. Every reading is on the DAG. A consumer scans a QR code and sees the cryptographically verified history of their product.

Autonomous vehicle accountability — Every decision by a self-driving car is signed by the vehicle's keypair: sensor readings, object detection results, path planning decisions, actuator commands. In an accident investigation, the DAG provides a millisecond-by-millisecond cryptographic audit trail.

Medical device integrity — A robotic surgical system signs every movement command. A patient's implanted sensor signs every heart rate reading. The data is immutable, attributed, and tamper-evident — meeting regulatory requirements while enabling the patient to own their own health data.

Smart grid validation — Solar panels sign generation readings, smart meters sign consumption readings, grid controllers sign distribution decisions. Energy trading between neighbors is validated on the DAG without a utility company intermediary.

8.5 Physical Object Authentication

Physical objects can be linked to DAG identities through:

- **NFC/RFID tags** — embedded chips that sign a challenge with their keypair, proving authenticity (luxury goods, pharmaceuticals, art)
- **PUF (Physical Unclonable Functions)** — semiconductor fingerprints that cannot be cloned, linked to DAG identities
- **Visual hashing** — high-resolution photographs of physical objects hashed and signed (art authentication, evidence collection)

8.6 Firmware Integrity and Device Attestation

Physical access to IoT deployments is a realistic attack vector. An adversary who captures a Profile C device can clone its firmware, implant a backdoor, and inject false readings through the gateway. The protocol mitigates this through:

Secure boot attestation — devices that support it produce a firmware hash at boot time. The security level varies by hardware: ARM TrustZone provides hardware-isolated key storage; ESP32-S3 secure boot uses eFuse-based verification of a signed bootloader in flash (not a full hardware-fused signing key). The gateway verifies the firmware hash before accepting readings. Devices with unexpected firmware hashes are quarantined. The attestation strength is reflected in the device's trust weight — hardware-isolated attestation carries more weight than flash-based verification.

Heartbeat anomaly detection — gateways monitor timing patterns, reading distributions, and communication behavior of leaf devices. A cloned device with modified firmware will exhibit measurable behavioral differences (response latency, reading noise profile, boot timing). The AI layer (Layer 3) can flag statistical anomalies for operator investigation.

Key rotation — pre-shared symmetric keys between leaf devices and gateways are rotated periodically (configurable, default: 30 days). A captured device with an extracted key has a limited exploitation window.

Physical tamper detection — for high-security deployments (medical, military, critical infrastructure), devices can include tamper-evident enclosures that zeroize key material upon case opening. This is not required by the protocol but is recommended for Profile C deployments with physical access risk.

For the broader threat of device wipes, identity resets, and reputation escape through hardware recycling — including hardware-bound identity persistence, organizational binding, behavioral fingerprinting, and decommissioning protocols — see Section 11.33.

9. Public Network Incentive Layer

Scope: Public permissionless network only. This section describes the incentive mechanism for coordinating a public network where participants have no pre-existing trust relationship. Enterprise, government, and defense deployments operate as private networks (Section 10.6) with zero beat involvement. Private networks use the

same cryptographic protocol (Layer 1) and the same data structure (DAM) without any economic layer. Beats are not required for validation, signing, verification, or any cryptographic operation.

Full specification: The complete public network economic model — including supply mechanics, distribution, governance economics, storage markets, anti-centralization mechanisms, and regulatory analysis — is specified separately. This section provides a protocol-level summary.

9.1 Role of the Beat

The Elara Protocol **beat** is a utility unit that enables four protocol functions:

1. **Witness staking** — nodes stake beats to participate in attestation, creating an economic barrier against Sybil attacks (Section 11.1) and aligning incentives for honest witnessing.
2. **Priority network services** — requesting priority propagation, requested witnessing, and Layer 3 AI capabilities consumes beats. Contributing resources (bandwidth, compute) earns beats. Basic propagation and Layer 1 validation are always free (Section 3.5).
3. **Storage delegation** — nodes that cannot store records long-term pay storage-specialized nodes to hold them. The delegating node always retains its signed record header.
4. **Governance participation** — beat holders participate in protocol governance through conviction voting (Section 10.3), subject to anti-centralization constraints (Section 10.4).

9.2 Design Principles

The beat economic model is guided by four principles:

- **Conservation over inflation** — the protocol targets a fixed-supply model where beats circulate between producers (witnesses) and consumers (record submitters) rather than being continuously minted.
- **No gas fees** — transaction costs are borne by the network's reciprocal witnessing model, not by per-transaction fees. At scale (millions of nodes), gas fees would be economically prohibitive.
- **Layer 1 is always free** — local validation never has a cost (Section 3.5, Section 11.10). The beat economy applies only to Layer 2 network services.
- **Utility, not speculation** — the beat is an internal accounting unit with network utility, never an investment vehicle. It is not offered, sold, or listed: no ICO, no pre-sale, no exchange listing.

9.3 Protocol Integration Points

The beat interacts with the protocol at these specific points:

- **PoWaS attestation** (Section 11.1) — witnesses stake beats to participate; difficulty scales with stake
- **Priority propagation** (Section 11.10) — paid tier for faster global reach

and requested witnessing

- **Dispute arbitration** (Section 11.13) — parties stake beats to invoke arbitration panels
- **Conviction voting** (Section 10.3) — beat-weighted governance with time-locked conviction
- **Zone health metrics** (Section 11.22) — total staked beats as a zone health indicator
- **Storage delegation** — nodes delegate record storage to storage-specialized nodes in exchange for beats

The complete economic model — including supply model, distribution schedule, anti-centralization mechanisms (diminishing returns, quadratic governance, trust-weighted committee selection), storage delegation markets, and Sybil cost analysis — is specified separately.

10. Governance

10.1 Multi-Zone Autonomy

The Elara Protocol's governance is federated, not centralized:

- Each zone (geographic or planetary) governs itself
- Zones can adopt different policies for local matters (storage requirements, minimum witness counts, AI layer participation)
- Cross-zone matters (protocol version, cryptographic standards, beat economics) require multi-zone consensus

10.2 Decision Categories

Zone-local (decided by zone stakeholders): - Minimum witness count for local trust thresholds - Storage retention policies - AI layer participation requirements - Local fee structures

Cross-zone (decided by all zones): - Protocol version upgrades - Cryptographic algorithm additions or deprecations - Beat supply changes - New entity type definitions - Zone creation or dissolution

10.3 Voting Mechanism

Cross-zone decisions use a **conviction voting** model [36] (inspired by conviction voting mechanisms pioneered by Commons Stack and 1Hive, 2019):

- Beat holders express preferences by staking beats toward proposals
- Voting weight accrues over time according to: $\text{conviction}(t) = \text{stake} \times (1 - e^{-(t/\tau)})$ where t is days staked and $\tau = 7$ days (time constant). Weight reaches ~63% at 7 days, ~86% at 14 days, ~95% at 21 days, and ~98.6% (effectively full conviction) at 30 days. The exponential ramp makes flash-vote attacks economically pointless — meaningful conviction requires sustained commitment.
- Proposals require both **supermajority** (>67% of conviction-weighted stake) and **quorum** (>25% of all staked beats participating)
- Implementation is delayed 30 days after passing (allowing zones to prepare)

Note: Additional governance mechanisms — including trust-weighted random committee selection, identity-based voting caps, and anti-pooling measures — complement the conviction voting model described here and are specified separately.

10.4 Governance Attack Mitigations

The conviction voting mechanism (Section 10.3) is designed to resist several known governance attacks:

Sybil resistance: Voting power is proportional to staked beats, not identity count. Creating 1,000 identities with 1 beat each provides the same voting power as 1 identity with 1,000 beats — Sybil attacks gain nothing. The economic cost of acquiring sufficient beats to dominate governance scales with network value.

Flash loan / flash vote attacks: Conviction voting's time-weighted staking prevents an attacker from borrowing beats, voting, and returning them in a single transaction. The exponential conviction curve means beats staked for less than 7 days carry less than 63% weight, and full conviction (~98.6%) requires 30 days of sustained staking. This makes flash attacks economically pointless — the capital lockup cost exceeds any governance manipulation benefit.

Vote buying: While the protocol cannot prevent off-chain vote buying, the 30-day implementation delay (Section 10.3) allows the community to detect and respond to suspicious voting patterns before changes take effect. Zones can invoke emergency veto (requiring >75% of anchor nodes) to block proposals that passed through suspected manipulation.

Plutocracy mitigation: Raw beat-weighted voting favors wealthy participants. The protocol applies a **square-root dampening** to conviction weight. The combined governance weight formula is:

$$\text{governance_weight} = \min(\sqrt{\text{stake}} \times (1 - e^{(-t/\tau)}), (1/\sqrt{N}) \times \text{TOTAL_STAKED})$$

where $\tau = 7$ days, N = total active stakers, and the cap is per-identity

An entity staking 10,000 beats has $\sqrt{10} \approx 3.16\times$ the influence of an entity staking 1,000 beats, not $10\times$. The conviction curve then weights by lock duration (reaching ~98.6% at 30 days), and the scaling cap ensures no single identity exceeds $1/\sqrt{N}$ of total governance weight regardless of stake size (where N = total active stakers — at 100 stakers the cap is 10%, at 10,000 it's 1%, at 1M it's 0.1%). Extended analysis of pool-centralization risks and additional anti-centralization mechanisms is specified separately.

Emergency veto abuse: The emergency veto (>75% of anchor nodes) is a powerful mechanism that could itself be gamed. Constraints: (1) a veto can only block, never propose — it cannot be used to force changes, only prevent them; (2) vetoes are rate-limited to 2 per zone per quarter; (3) any veto triggers a mandatory public disclosure of the veto rationale within 72 hours; (4) if a vetoed proposal passes a second vote with >80% conviction after the disclosure period, the veto is overridden. This ensures the veto is a circuit breaker, not a permanent kill switch.

10.5 Graceful Divergence

If zones cannot reach consensus on a cross-zone matter, the protocol supports **graceful divergence**:

1. The disagreeing zone announces a fork intention
2. A 90-day mediation period allows for compromise
3. If unresolved, the zone forks the protocol — maintaining DAG compatibility for historical records but diverging on the disputed feature
4. Cross-zone sync continues for shared historical data
5. New validations under the divergent rule are tagged with the fork identifier

This is not a failure state — it is a design feature. A protocol intended to span planets and centuries must survive political and philosophical disagreement without collapsing.

10.6 Private Networks and Network Publication

The governance model described above assumes participation in the public Elara network. But the protocol's layered architecture (Section 3.2) enables a distinct deployment model: **private Elara networks** that operate independently, with no public network participation.

10.6.1 Private Deployment Model

A private Elara network uses the same protocol stack — post-quantum signatures, DAM structure, wire format, witness consensus — within a closed organizational boundary. This is not a fork or a modification: it is the protocol operating at Layer 1 + private Layer 2, without connecting to the public Layer 2.

Examples: - An aerospace manufacturer validating firmware provenance across factory sites - A space agency recording mission-critical software authorship and satellite telemetry within its engineering teams - A pharmaceutical company maintaining tamper-evident drug trial data chains - An automotive OEM tracking supply chain component validation across suppliers - Defense contractors validating firmware and mission-critical software across classified environments - Military operations validating tactical decisions, sensor data, and autonomous system logs - Robotics companies validating every autonomous decision for regulatory compliance and liability - Semiconductor fabrication plants validating billions of sensor readings per day across factory floors (Section 3.6)

These organizations benefit from the cryptographic properties (post-quantum security, causal ordering, tamper evidence) without requiring public consensus or beat economics. Governance is organizational, not protocol-level. Anti-centralization mechanisms are unnecessary — internal trust hierarchies are appropriate for corporate environments.

Key architectural properties of private deployments:

1. **Layer 1 is unchanged.** The same keygen, signing, hashing, and DAG operations. Records created on private networks are structurally identical to public records.
2. **Layer 2 is scoped.** Discovery, propagation, and witnessing occur only within the private network boundary. The organization controls the

- peer set.
3. **Layer 3 operates independently.** AI analysis runs against the private DAG. No data leaves the boundary.
 4. **No beat requirement.** Resource allocation is organizational, not market-based.

10.6.2 The Publication Spectrum

Private networks exist on a spectrum from fully closed to fully public:

Mode	Description	Trust Model	Beat Involvement
Fully private	Closed network, no external connections	Internal hierarchy	None
Federated	Bilateral sharing between partner organizations	Mutual trust agreements	Optional (cross-org settlement)
Selective publication	Some record types published to public network	Hybrid: internal + public	For published records only
Full publication	All records participate in public consensus	Public witness consensus	Full participation

An organization may operate at different points on this spectrum for different record types simultaneously. Internal engineering records remain private. Supply chain attestations are shared with partners (federated). Finished product validations are published to the public network.

10.6.3 The NETWORK_PUBLISH Protocol

Status (2026-06-22): the NETWORK_PUBLISH / Validation-IPO transition described in this section is DISABLED in code (`NETWORK_PUBLISH_ENABLED = false`, compile-time guarded in `src/network/publish.rs`). The per-record publication model — imported records *entering* public consensus and gaining *retroactive* public attestations — was found unsound: a `ValidationRecord`'s signed bytes carry no realm/network binding (Assumption A8, `docs/MESH-BFT-MERGE-SEMANTICS.md`), so an imported record is consensus-indistinguishable from a native one, and MESH-BFT's single-network safety theorem does not cover adopting a foreign network's records as native settlement parents. The mechanism is being reframed to **inert-import**: public consensus will attest only that a publication *bundle* (source root + Merkle root of the imported set + completeness proof + external time anchors) existed at an anchored time — conferring **zero native standing** (no settlement-parent role, no stake, no witness weight, no cross-zone debit basis). The text below describes the original design and is retained for reference pending that reframe and a proven multi-root merge theorem.

When a private network transitions records to the public network — partially or fully — the protocol defines a **NETWORK_PUBLISH** record type:

Record Type: NETWORK_PUBLISH (0x0E)

Fields:

source_network_id	bytes	Public key of private network root authority
published_records	RecordSet	Record ID range, classification filter, or DAG subtree
publication_scope	enum	FULL SELECTIVE FEDERATED
target_zone	ZoneID	Destination zone in the public network
historical_depth	uint64	How far back (in records or time) to publish
redaction_policy	Policy	Which metadata fields are stripped before publication
transition_mode	enum	SNAPSHOT STREAMING GRADUAL
completeness_proof	bytes	Optional Merkle proof that published set is complete
		relative to the source DAG (prevents selective omission)

Transition modes:

- **SNAPSHOT** — Publish entire DAG subtree at once. Immediate verifiability, high bandwidth cost.
- **STREAMING** — Publish records chronologically over a defined period. Allows the public network to absorb and verify incrementally.
- **GRADUAL** — Begin with recent records, extend historical depth over time. Lowest initial exposure.

Verification process:

When the public network receives published records:

1. **Signature verification.** Every record's post-quantum signature is verified independently. Signatures are self-contained — they do not depend on network state.
2. **Causal chain verification.** Parent references are followed to ensure the DAG structure is internally consistent. Missing parents (unpublished records referenced by published records) are flagged as known gaps, not errors.
3. **Temporal consistency.** Timestamps are checked for monotonicity within causal chains. A child record cannot have a timestamp earlier than its parent.
4. **Completeness check.** If a completeness proof is provided, it is verified against the published record set. This proves the organization is not selectively omitting unfavorable records from a subtree.
5. **Retroactive witnessing.** Public nodes can witness historical records, adding new trust attestations that reference the original (unchanged) records.

10.6.4 Governance Implications

Status (2026-06-22): DISABLED in code — these governance events follow from the NETWORK_PUBLISH transition of §10.6.3, which is compile-time disabled (NETWORK_PUBLISH_ENABLED = false) pending the inert-import reframe. The retroactive-witnessing and IPO-style

“public trust bootstrapping” described below do not occur on the live protocol. Retained for reference. See §10.6.3 and docs/MESH-BFT-MERGE-SEMANTICS.md.

Private-to-public transitions create governance events:

Zone registration. A large private network publishing as a new zone follows the zone creation process (Section 10.2, cross-zone decision). The zone’s internal governance may differ from the public network’s governance model.

Trust bootstrapping. Published historical records carry internal trust (accumulated from private witnesses) but zero public trust. Public trust accumulates through retroactive witnessing. A 10-year-old published record may reach high public trust within weeks if many public nodes verify and witness it.

Representation. Once published, the organization’s nodes become public network participants with governance weight proportional to their staked beats and accumulated conviction (Section 10.3). A large organization entering the public network could represent significant governance weight — the square-root dampening and 5% cap per identity (Section 10.4) limit this concentration.

The analogy to traditional markets is deliberate: a private network choosing to publish is structurally similar to a company filing an IPO — historical records are disclosed, public trust is established based on track record, and the entity gains access to the broader ecosystem’s resources (public witnessing, storage delegation, cross-network attestation) in exchange for transparency.

Detailed analysis of the economic dynamics of this transition — including beat demand modeling, anti-gaming mechanisms, and the long-term implications of dual-direction network growth — is specified separately.

11. Adversarial Resilience

A protocol that cannot answer its critics is a protocol that does not survive peer review. This section addresses 34 attack vectors and design challenges — from Sybil resistance to formal verification strategy.

11.0 Threat Model

Before analyzing specific attacks, we define the adversary model:

Adversary capabilities (ordered by strength):

Level	Adversary	Capabilities
1	Individual	Controls one or a few nodes, limited resources
2	Organization	Controls hundreds of nodes, significant capital, legal standing
3	Nation-state	Controls network infrastructure, can compel ISPs, legal coercion
4	Quantum	Access to cryptographically relevant quantum

Assumptions:

- **Honest majority:** At least 2/3 of staked weight in any zone is held by honest nodes (standard BFT assumption)
- **Cryptographic hardness:** NIST PQC primitives (ML-DSA (FIPS 204, “Dilithium”), Kyber, SPHINCS+) remain computationally infeasible to break for the claimed security levels
- **Network model:** Asynchronous with eventual delivery — messages may be delayed arbitrarily but are eventually delivered to honest nodes
- **No trusted hardware:** The protocol does not assume TEEs, secure enclaves, or tamper-proof devices. Security derives from cryptography and economics, not hardware trust.
- **Rational actors:** Witness nodes are economically rational — they will not spend resources on actions that reduce their expected returns

What the protocol does NOT defend against:

- Compromise of a creator’s private key combined with physical theft of unreleased content (this is a physical security problem, not a protocol problem)
- A quantum adversary that breaks both lattice-based AND hash-based cryptography simultaneously (no known path to this)
- Social engineering that induces a user to sign content they did not create (the protocol validates signatures, not intent)

Security (11.1–11.9)

11.1 Sybil Resistance

The attack: An adversary generates one million keypairs and creates one million fake “witness” nodes. Every validation record they publish instantly accumulates a million witnesses, making fraudulent claims appear globally trusted.

This is the oldest problem in decentralized systems. The Elara Protocol addresses it through layered defense:

Layer 1: Proof of Work-at-Stake (PoWaS)

Witness attestation is not free. To attest to a validation record, a witness node must:

1. Stake a minimum amount of beats (economic cost)
2. Solve a lightweight proof-of-work puzzle calibrated to the attestation (computational cost — not industrial-scale mining, but enough to make mass attestation expensive)
3. Maintain a reputation score based on attestation history (time cost)

PoWaS puzzle construction:

```
puzzle_input = SHA3-256(record_id || witness_pubkey || nonce)
difficulty   = BASE_DIFFICULTY × (1 / sqrt(stake_amount))
target       = 2256 / difficulty
valid_if     = puzzle_input < target
```


The difficulty is inversely proportional to the square root of the witness's staked amount, bounded by a minimum and maximum difficulty to prevent both trivial solutions by large stakers and impossible puzzles for small stakers:

```
effective_difficulty = clamp(difficulty, MIN_DIFFICULTY,
MAX_DIFFICULTY)
where MIN_DIFFICULTY = BASE_DIFFICULTY / 100 (no one gets a free
pass)
      MAX_DIFFICULTY = BASE_DIFFICULTY × 10 (minimum stake is
viable)
```

A witness staking 1,000 beats solves a puzzle ~32x easier than a witness staking 1 beat, but the bounds ensure that very large stakers still perform meaningful computation and very small stakers are not excluded entirely. This creates a combined economic-computational barrier: attacking cheaply requires massive computation; attacking with minimal computation requires massive stake.

Difficulty adjusts per-zone every epoch to maintain a target attestation rate (~10 attestations per second per zone). This prevents both under-utilization (too hard) and spam (too easy).

An adversary creating a million Sybil nodes would need to acquire beats for each (economic barrier), solve puzzles for each attestation (computational barrier), and build reputation over time for each node (temporal barrier). The cost of attack scales linearly; the defense is multiplicative. Detailed Sybil cost analysis and diminishing returns per entity are specified separately.

Layer 2: Social Graph Analysis

The AI-assisted analysis layer (Layer 3 of the protocol architecture) monitors witness patterns:

- Nodes that only attest to each other's records (closed clusters) receive reduced trust weight
- Nodes with attestation patterns inconsistent with their stated geography or entity type are flagged
- Sudden spikes in new witnesses for a specific creator trigger anomaly alerts

This is not a centralized filter — it is a distributed heuristic that each node can independently compute from the DAM's public data.

Layer 3: Trust Decay for New Identities

New keypairs start with zero trust. Their attestations carry minimal weight. Trust accumulates through:

- Duration of existence (older keys are harder to fake at scale)
- Diversity of attestation partners (attesting to many unrelated creators)
- Consistency of behavior (regular, plausible patterns)
- Cross-zone attestation (a key attested by nodes in multiple geographic zones is harder to Sybil)

A Sybil army of fresh keys carries near-zero attestation weight. By the time they've aged enough to matter, the temporal and economic costs make the attack unprofitable. A related variant — Sybil amplification through

physical device recycling (wiping and re-enrolling the same hardware) — is addressed in Section 11.33.

Layer 4: Leaf Node Independence

Critically, Layer 1 validation (local, offline) is immune to Sybil attacks entirely. The poem on the phone in Nairobi is cryptographically valid regardless of how many fake witnesses exist on the network. Sybil attacks can pollute trust scores, but they cannot invalidate a legitimate local validation. The creator's signature is the ground truth; witnesses are corroboration, not authority.

11.2 Key Compromise and Revocation

The scenario: A teenager's phone is stolen. The thief has her private key. Without intervention, the thief can sign new work as her and validate fraudulent claims under her identity.

Revocation mechanism:

Every Elara identity supports a **revocation record** — a special validation record that:

1. Is signed with the compromised key (proving ownership)
2. Contains a revocation timestamp (all signatures after this time are invalid)
3. Optionally designates a successor key (migration, not just termination)
4. Is flagged as REVOCATION type and propagates with highest priority across the network

Once published, the revocation record is immutable on the DAM. All future signatures from the compromised key are rejected by any node that has received the revocation.

But what if the thief revokes first?

This is the harder problem. The protocol handles it through **pre-committed recovery keys**:

At identity creation, the user can (and is strongly encouraged to) generate a **recovery keypair** stored separately — written on paper, saved on a USB drive, held by a trusted person. The recovery key's public half is embedded in the original identity record.

Only the recovery key can: - Override a fraudulent revocation - Designate the legitimate successor key - Prove identity in a dispute where both parties hold valid keys

If no recovery key was pre-committed, the dispute becomes a social/legal matter — but the DAM preserves the full timeline of both parties' claims, providing evidence for resolution.

Dual compromise (both primary and recovery keys lost): If an adversary obtains both keys, the identity is irrecoverably compromised. The user must create a new identity and rely on out-of-band evidence (legal records, prior witnesses, incremental creation chains) to re-establish attribution of their work. This is the catastrophic failure mode — analogous to losing both a password and all recovery options. The protocol cannot

solve this cryptographically; it can only preserve the evidentiary record for dispute resolution. Users who require higher assurance should use multi-party recovery schemes (e.g., Shamir secret sharing across trusted contacts) to protect their recovery key.

Key rotation:

The protocol supports scheduled key rotation without identity loss. A rotation record — signed by both the old and new keys — maintains continuity. All past work remains attributed to the identity; all future work uses the new key. This limits the damage window of any compromise.

Note: deliberate key destruction through device wipes — where the goal is to sever accountability rather than steal identity — is a distinct attack vector addressed in Section 11.33.

11.3 Light Clients

The problem: A \$30 phone cannot store the full DAM. A Mars colony with limited bandwidth cannot download Earth's complete history. How do lightweight nodes verify claims without the full dataset?

Solution: Merkle Proof Verification

The DAM maintains a Merkle tree over all validation records. A light client can verify any specific record with:

1. The validation record itself (~4-5 KB)
2. A Merkle proof path ($\sim \log_2(N) \times 32$ bytes — for a billion records, this is ~960 bytes)
3. The Merkle root (32 bytes, published by anchor nodes)

Total verification payload for a single record: **under 6 KB**, regardless of how large the DAM grows.

Trust headers:

Anchor nodes publish periodic **trust headers** — compact summaries containing:

```
TrustHeader {
    zone:           zone identifier
    timestamp:      vector clock position
    merkle_root:    root hash of all records in this zone
    record_count:   total validation records
    witness_summary: aggregated trust statistics
    anchor_signatures: [signed by multiple anchor nodes]
}
```

A light client that trusts at least one anchor node can verify any claim against the trust header without downloading the full DAM. This is analogous to Simplified Payment Verification (SPV) in proof-of-work systems, extended for the DAM's multi-zone structure.

Progressive download:

Light clients can optionally download DAM data incrementally:

- **Level 0:** Trust headers only (~1 KB/day) — sufficient for basic verification

- **Level 1:** Headers + own records + records they've queried (~100 KB–10 MB)
- **Level 2:** Headers + local zone history (~10 MB–1 GB)
- **Level 3:** Full DAM replica (anchor/archive nodes only)

A budget smartphone operates at Level 0 or 1. It validates locally, syncs its own records, and verifies others via Merkle proofs. It never needs the full DAM.

11.4 Network Bootstrap

The problem: The first node has no witnesses. The second node has only one possible witness. How does a trust network start from zero?

Phase 1: Genesis Anchor (nodes 1–10)

The founding team operates the first anchor nodes. These nodes are explicitly identified in the protocol's genesis block as **genesis anchors** — trusted not by accumulated attestation, but by their role in creating the network. This is centralization, and it is acknowledged openly.

Every decentralized network starts centralized. Early proof-of-work networks had one miner. Smart contract platforms launched with foundations. The difference is the exit plan.

Phase 2: Early Growth (nodes 10–1,000)

Genesis anchors actively attest to new nodes' identity registrations. Early participants earn elevated trust through:

- Direct attestation by genesis anchors (bootstrapping trust)
- Participation in testnet validation (proving reliability)
- Contribution to the codebase, documentation, or tooling (proof of commitment)

Beat incentives during this phase are elevated — early validators earn disproportionate rewards to compensate for the network's low utility. The distribution schedule and bootstrap economics are specified separately.

Phase 3: Decentralization Threshold (nodes 1,000–10,000)

At 1,000 active witness nodes across at least 10 geographic regions, the protocol reaches its **decentralization threshold**. At this point:

- Genesis anchors' special trust status expires (they become regular anchor nodes)
- Governance transitions from founding team to beat-weighted voting
- The protocol is self-sustaining — no single entity can disrupt consensus

Phase 4: Critical Mass (10,000+ nodes)

The network effects take over. Developers build on the protocol because users are there. Users join because developers have built tools. Institutions adopt because the network is too large to ignore.

The bootstrap problem is real, but it is a solved problem in practice. The challenge is not technical — it is social. The protocol must be useful enough that the first 1,000 people choose to run nodes. Section 3.5 (Minimum

Viable Validation) is the answer: the protocol is useful to a single person with a single device before anyone else joins.

11.4.1 Epoch-Indexed State Snapshots (v0.7.9+)

Once a network accumulates 10M+ records across its zones, replaying the DAG from genesis to derive ledger state becomes cost-prohibitive for new joiners and for any node recovering from a storage wipe. The protocol solves this with **epoch-indexed snapshots**: archive-profile nodes emit signed snapshots at epoch boundaries (default: every 10 epochs, retention 20), published at deterministic paths of the form `/snapshot/epoch/{N}` and listed via `/snapshot/epochs`.

A snapshot is authoritative state, not a replay hint. It includes:

- The complete ledger (`total_supply`, per-account balances, active stakes, staked totals, trust scores, continuity scores) as of the snapshot boundary.
- The set of record IDs whose beat ops have already been applied to that ledger (`applied_record_ids`). This set is what lets a bootstrapping node seed its `CF_APPLIED` dedup column family, so any pre-snapshot record re-delivered via delta sync or gossip is recognized as already-accounted-for and skipped at the ledger-apply gate — no double-apply, regardless of gossip redelivery order.
- The finalized-record set, last-seal metadata per zone, genesis state, and bootstrap phase.
- A snapshot timestamp (`snapshot_timestamp`), which is the cursor the subsequent delta-sync loop resumes from.
- A ML-DSA-65 (FIPS 204, “Dilithium3”) signature over the canonical serialization, signed by the emitting archive node’s identity key. The signer’s public key is embedded so verifiers need no out-of-band trust.

Bootstrap algorithm. A joining node:

1. Queries `/snapshot/epochs` from its seed peers and selects the highest-numbered epoch that is available on at least two independent peers.
2. Downloads the full snapshot from the primary peer via `/snapshot/epoch/{N}`.
3. Cross-verifies the snapshot’s Merkle checksum against up to three other peers’ `/snapshot/epoch/{N}/checksum` endpoints. Any mismatch (primary says root R but another peer says $R' \neq R$) aborts the bootstrap. Silence (peer has no snapshot for N) is not a disagreement.
4. Verifies the Dilithium3 signature against the embedded public key and checks the signer satisfies the local trust policy (anchor, archive, or allow-listed peer).
5. Applies the snapshot as authoritative state: the ledger replaces any partial local ledger; `CF_APPLIED` is bulk-seeded with the snapshot’s `applied_record_ids`; the finalized set, genesis state, and bootstrap phase are restored; and the `pull_catchup_cursor` is advanced to `snapshot_timestamp`.
6. From that cursor, delta sync fetches only records newer than the snapshot — the ~9.99M pre-snapshot records on a 10M-record chain are never downloaded.

Profile-scoped behavior. The cursor-advance in step 6 is conditional on node profile. `Light` and `FullZone` profiles seed the cursor and skip pre-snapshot record fetches entirely — the snapshot is authoritative for all state they need; retention policy would prune old records anyway. Archive profiles — the historical source of truth — do **not** seed the cursor, so delta sync backfills pre-snapshot records from timestamp zero for DAG completeness. `CF_APPLIED` dedup still prevents any ledger double-apply; Archive just additionally retains the record bytes and DAG edges.

Why signed and cross-verified, not just signed. A single archive node's signature vouches that *it* computed this state at that epoch — not that the state is correct. A colluding peer could serve a forged snapshot signed with its own legitimate key. Cross-peer Merkle-root verification protects against that class of attack: an honest peer that emitted its own snapshot at the same epoch will have a different root if the primary's snapshot is forged. Silence is allowed because not every peer is configured as an archive emitter; disagreement is not. This matches the pattern used for super-seal verification in §11.12.

Retention and liveness. Archive nodes hold the last retention snapshots (default 20) and prune older ones lazily to bound disk growth at $20 \times \text{snapshot_size}$. Snapshots are emitted every `every_n_epochs` (default 10) — at a 60-second P50 epoch, that's a new bootstrap anchor every ten minutes, with a ~200-minute window of historical bootstrapping options.

11.4.2 Storage Tiers and Snapshot Pricing

See §12.2 for how snapshot emission interacts with the `Light` / `FullZone` / Archive retention profiles, and how the delegated-storage market in §12.2 prices snapshot serving alongside cold-tier record serving.

11.5 Immutability vs. Right to Deletion (GDPR)

The conflict: The EU's General Data Protection Regulation (GDPR) grants individuals the “right to erasure” — the right to demand that their personal data be deleted. The Elara DAM is immutable — records cannot be deleted. These appear to be irreconcilable.

The resolution: The Elara Protocol validates hashes, not content.

A validation record on the DAM contains:

- A cryptographic hash of the content (not the content itself)
- A public key (pseudonymous, not a name or address)
- A timestamp and DAG references

The actual content — the poem, the document, the sensor reading — is stored off-DAM, under the creator's control. The DAM stores proof that the content existed, not the content itself.

GDPR compliance path:

1. **Delete the content** — the creator removes the original work from their device and any storage. The hash on the DAM becomes an orphan — it proves that *something* existed, but that something is gone. The hash cannot be reversed to recover the content (SHA3-256 is a one-way function).

2. **Revoke the identity** — the creator issues a revocation record. Their public key is marked as revoked. The pseudonymous link between the hash and any real-world identity is severed.
3. **The DAM retains only:** an orphaned hash signed by a revoked pseudonymous key. This satisfies GDPR's erasure requirement because no personal data remains — only mathematical artifacts that cannot be linked to a natural person.

For PRIVATE and SOVEREIGN classifications, the situation is even cleaner: the content hash was never visible on the DAM in the first place. Only a SHA3-256 commitment proof exists (Phase-1; genuine ZK is design-stage). Revoking the key makes the proof unattributable.

For IoT and device data, GDPR applies only to personal data. Sensor readings from industrial equipment or environmental monitors are not personal data and are not subject to erasure rights.

Precedent: This approach aligns with guidance from EU data protection authorities on blockchain and GDPR, which acknowledge that storing hashes of personal data (rather than the data itself) may satisfy the regulation's requirements, particularly when combined with key deletion.

The Elara Protocol does not fight regulation. It is designed so that compliance is architecturally natural, not a retrofit.

11.6 Timestamp Gaming (Offline Backdating)

The attack: A malicious actor sets their device clock to January 2024, validates stolen work, then syncs to the network in February 2026. The validation record shows a 2024 timestamp, granting false priority over the actual creator.

This is a fundamental weakness of any system that allows offline validation with local timestamps. The Elara Protocol addresses it through three mechanisms:

Mechanism 1: Causal Anchoring

When an offline node syncs to the network, its validation records must reference existing DAM records as parents. The sync protocol automatically inserts a **causal anchor** — a reference to the most recent record the node received during synchronization.

This creates an ironclad constraint: a record with a causal anchor from February 2026 cannot have been created in January 2024, regardless of what the local timestamp claims. The DAG structure itself disproves the backdated timestamp.

Mechanism 2: Temporal Witness Scoring

Trust scores weight the gap between claimed creation time and first network appearance:

```
trust_penalty = f(time_claimed, time_first_witnessed)
```

```
If first_witnessed - time_claimed < 24 hours: no penalty
```

```
If first_witnessed - time_claimed < 7 days: minor penalty (0.9x trust)
```

If first_witnessed - time_claimed < 30 days: moderate penalty (0.5x trust)
If first_witnessed - time_claimed > 30 days: severe penalty (0.1x trust)
If first_witnessed - time_claimed > 1 year: near-zero trust (0.01x)

A record that claims to be two years old but was first seen today carries almost no trust weight. It exists on the DAM (nothing is deleted), but its trust score reflects the suspicion.

Mechanism 3: Concurrent Priority Protocol

When two records claim the same content hash at different times, the protocol does not automatically award priority to the earlier timestamp. Instead, it evaluates:

1. **Causal anchoring** — which record has DAM-verifiable temporal context?
2. **Witness accumulation speed** — a legitimate record from a connected device accumulates witnesses in real-time; a backdated record arrives in a burst
3. **Device attestation history** — a device that has been consistently online and validating is more credible than one that appears with years of backdated records
4. **Cross-reference** — do other records from the same creator show consistent timelines, or is there a suspicious gap?

The result: local timestamps remain useful (and honest nodes produce accurate ones), but they are never the sole basis for priority claims. These three mechanisms work in concert — causal anchoring provides structural proof, temporal witness scoring applies economic penalties, and the concurrent priority protocol evaluates the full context. No single mechanism is sufficient alone, but together they make timestamp gaming detectable, penalized, and ultimately unprofitable. The DAG's causal structure serves as the primary authoritative clock; timestamps are supplementary metadata.

11.7 The Originality Problem

The challenge: The Elara Protocol proves that a specific hash, signed by a specific key, existed at a specific point in the DAM's causal order. It does NOT prove that the key holder created the content. A thief could hash a stolen novel and register it before the author does.

This is not a bug — it is an inherent limitation of any cryptographic validation system. No protocol can prove creation; it can only prove possession and timing. The Elara Protocol is honest about this boundary and provides tools to make theft as difficult and detectable as possible:

Tool 1: Incremental Validation

Creators are encouraged to validate work incrementally — not just the final product, but drafts, outlines, sketches, intermediate versions. A poet who validates six drafts over three weeks has a validation trail that a thief cannot replicate. The thief can hash the final poem, but they cannot produce the creative history.

Draft 1 (Feb 1) → hash_a, signed by author
Draft 2 (Feb 5) → hash_b, signed by author, references hash_a
Draft 3 (Feb 12) → hash_c, signed by author, references hash_b
Final (Feb 20) → hash_d, signed by author, references hash_c

vs.

Stolen copy (Mar 1) → hash_d, signed by thief (no history)

The DAM preserves the full creative chain. A single hash proves possession. A chain of hashes proves process — and process is much harder to fake.

Tool 2: Composite Attribution Records

For AI-assisted work (the majority of future digital creation), the CollaborationRecord structure (Section 6.3) creates inherent provenance. The AI's keypair co-signs the work. A thief who steals the output cannot produce a matching CollaborationRecord unless they also compromised the AI's private key.

Tool 3: Content Fingerprinting (Layer 3)

The AI intelligence layer can compute similarity hashes (locality-sensitive hashing, perceptual hashing for images/audio) alongside cryptographic hashes. When a new validation is submitted, the AI layer can flag near-duplicates:

- Exact content match with different creator → immediate conflict flag
- High similarity (>90%) with existing validated work → plagiarism warning
- Same content validated by two keys → priority dispute initiated

This does not prevent theft, but it detects it — and detection on an immutable ledger is a powerful deterrent.

Tool 4: Honest Framing

The whitepaper and all protocol documentation explicitly state: **validation proves possession and timing, not creation.** This prevents false expectations and ensures that legal systems interpret DAM records correctly. A validation record is evidence, not verdict. It is strong digital evidence of “I had this at this time,” but it is not omniscience.

Courts already understand this distinction — a notarized document proves the document existed at a date, not that the signer wrote it. The Elara Protocol provides cryptographic notarization at global scale.

11.8 Storage Growth and DAM Sustainability

The problem: “Nothing is ever deleted” combined with IoT-scale validation creates unbounded storage growth. A single factory with 10,000 sensors producing readings every second generates ~864 million records per day. At ~4.5 KB per validation record (dominated by the PQC signature), that is **~3.9 TB per day from one deployment.** At planetary scale, the DAM would grow by petabytes daily.

No single node can store this. The protocol must handle it.

Solution: Hierarchical Storage with Validation Summarization

Tier 1: Leaf and relay nodes — store only what they need

Leaf nodes (sensors, phones) maintain a circular buffer of their own records. Once synced to the network, old records can be evicted from local storage. The node retains its keypair and a pointer to its last synced position — not the full history.

Tier 2: Zone-level summarization

The protocol introduces **epoch summaries** — periodic Merkle tree snapshots that compress historical records into compact proofs:

```
EpochSummary {
  zone:           zone identifier
  epoch:          sequential epoch number
  time_range:     [start_vector_clock, end_vector_clock]
  record_count:   number of records in this epoch
  merkle_root:    root hash of all records in epoch
  creator_roots:  per-creator Merkle subtrees
  summary_hash:   hash of this summary (self-referential)
  signatures:     [anchor node signatures]
}
```

After an epoch is summarized and signed by multiple anchor nodes, individual validation records within that epoch can be pruned from standard nodes. The Merkle root preserves the ability to verify any individual record if the full record is retrieved from an archive node.

Tier 3: Archive nodes — store everything

Archive nodes (Seed Vault Tiers 2-4) maintain the full, uncompressed DAM. These are purpose-built for storage — high-capacity, redundant, geographically distributed. They serve the same role as the Internet Archive: not every node needs the full history, but the full history must exist somewhere.

Tier 4: IoT-specific compression

For high-frequency sensor data, the protocol supports **batch validation** — aggregating multiple readings into a single validation record:

```
BatchValidation {
  device:         device public key
  readings:       [reading_1, reading_2, ..., reading_n]
  batch_hash:     Merkle root of all readings
  individual_hashes: [hash_1, hash_2, ..., hash_n] // required
for individual verifiability
  time_range:     [first_reading_time, last_reading_time]
  signature:      device signature over batch_hash
}
```

A sensor producing 1 reading/second can batch 3,600 readings into one hourly validation record. Storage reduction: **3,600x** with no loss of verifiability (individual readings can still be proved via the batch Merkle tree).

Storage projections with batching:

Scale	Raw (unbatched)	With hourly batching	With epoch summarization
1 factory (10K			~100 MB/day (after

sensors)	~3.9 TB/day	~1.1 GB/day	epoch)
1M personal devices	~450 GB/day	~450 GB/day (no batching)	~45 GB/day
Global IoT (1B devices)	~390 PB/day	~108 TB/day	~11 TB/day

At ~11 TB/day after optimization, the full global DAM grows at ~4 PB/year — large but within the capacity of distributed archive infrastructure. For comparison, YouTube ingests ~720,000 hours of video daily (~1.5 PB). The DAM's storage challenge is significant but solvable with existing technology, and no single node bears the full load.

11.9 Vector Clock Scalability

The problem: Traditional vector clocks maintain one counter per node in the system. With 1 million nodes, each validation record would carry a vector of 1 million integers. At 4 bytes each, that is **4 MB per record** — orders of magnitude larger than the content hash it validates.

Solution: Zone-Scoped Interval Tree Clocks

The Elara Protocol replaces traditional vector clocks with a two-tier temporal ordering system:

Intra-zone: Interval Tree Clocks (ITCs)

Interval Tree Clocks (Almeida et al., 2008) provide the same causal ordering guarantees as vector clocks but with $O(\log n)$ space complexity instead of $O(n)$. ITCs work by dynamically splitting and joining identity intervals as nodes enter and leave the system:

- New node joins → receives a split of an existing node's interval
- Node leaves → its interval is available for merger
- Causal ordering is maintained through interval comparisons

For a zone with 100,000 active nodes, the ITC overhead per record is ~40 bytes instead of ~400 KB. This is acceptable even for IoT devices.

Inter-zone: Zone Sequence Numbers

Cross-zone ordering does not require per-node granularity. Each zone maintains a **zone sequence number** — a monotonically increasing counter incremented with each epoch summary. Cross-zone causal ordering uses these sequence numbers:

```
ZoneCausalReference {
    zone_id:      zone identifier
    zone_sequence: sequence number at time of last sync
    epoch:        epoch number
}
```

A record from Mars that references Earth zone_sequence 45,892 is causally after all Earth records up to that sequence. The overhead is ~20 bytes per cross-zone reference, regardless of how many nodes exist in the referenced zone.

Combined overhead per validation record:

Component	Size
ITC (intra-zone ordering)	~40 bytes
Zone references (inter-zone)	~20 bytes per referenced zone
Total (3 zones referenced)	~100 bytes

Compare to naive vector clocks at 1M nodes: **100 bytes vs. 4 MB** — a 40,000x improvement.

Economics and Incentives (11.10–11.20)

11.10 Free Tier and Economic Accessibility

Scope note: The beat economics described in this section apply exclusively to the public permissionless network. Private network deployments (Section 10.6) — enterprise, government, defense — have no beat involvement. Layer 1 validation is always free, always offline, always sovereign. No network, no beats, no fees.

The question: Section 3.5 shows that any device can validate locally for free. Section 9 describes beat staking for public network witnessing. For participants on the public network without beats, who pays for their records to be witnessed?

Solution: Tiered Economic Model

Layer 1: Always free. No exceptions.

Local validation — generating a keypair, hashing content, signing with a private key, storing on local DAG — costs zero beats. This is a cryptographic operation that runs on the device's own hardware. It is free because it consumes no network resources. Private network deployments operate entirely at this level with organizational witnessing — no beat layer exists.

This is the foundational principle of the protocol. Layer 1 never has a cost, on any deployment model.

Layer 2: Free propagation, paid priority.

When a node syncs to the network, its validation records propagate through the gossip protocol. Basic propagation is free — relay nodes forward records as part of their normal operation (they benefit from a well-connected DAM, so relaying is incentive-compatible).

What costs beats is **priority**: requesting faster propagation, higher witness counts, or guaranteed inclusion in the next epoch summary. Free-tier records propagate and accumulate witnesses organically. Paid-tier records get expedited service.

Free tier:

- Local validation: always free
- Network propagation: free (best-effort gossip)
- Witness accumulation: organic (witnesses choose what to attest)
- Epoch inclusion: guaranteed (but not prioritized)
- Storage: included in epoch summaries

Paid tier (beats):

- Priority propagation: faster global reach

- Requested witnessing: specific anchor nodes attest on request
- Priority sync: records synced first during bandwidth-limited windows
- Layer 3 AI: pattern analysis, similarity search, anomaly detection

Community witnessing pool:

A percentage of the Community/Governance beat allocation (20% of genesis supply) funds a **public witnessing service** — anchor nodes that attest to free-tier records. This creates a baseline level of network attestation for all participants, regardless of economic status.

Free-tier records propagate for free, get witnessed by community-funded anchors, and accumulate organic witnesses over time. Any user has the same Layer 1 cryptographic proof as a Fortune 500 company. Their Layer 2 trust score grows slower than paid-tier — but it grows.

Earn-by-participation:

Nodes that contribute resources (relay bandwidth, storage, compute) earn beats. The teenager's phone, by relaying other users' records, earns enough beats to request priority witnessing if she ever needs it. The protocol pays its participants. The complete tiered economic model, storage delegation markets, and earn-by-participation mechanics are specified separately.

11.11 zk-SNARK Trusted Setup

Implementation status — DESIGN-STAGE. This chapter analyses the trusted setup for the *specified* Groth16 zk-SNARK construction (§5.3), which is not built. The Phase-1 runtime uses SHA3-256 commitments, which need **no trusted setup**. The ceremony design below applies to a future Groth16 deployment.

The problem: the specified zk-SNARKs (for PRIVATE and RESTRICTED classifications — design-stage, §5.3) would require a trusted setup ceremony — a one-time generation of cryptographic parameters (Common Reference String). If the setup is compromised, an attacker can forge proofs. Who performs this ceremony, and how is it decentralized?

Solution: Multi-Party Computation Ceremony + Migration Path

Phase 1: Initial ceremony (pre-mainnet)

The trusted setup uses a **multi-party computation (MPC) ceremony** where multiple independent participants each contribute randomness. The security guarantee: as long as at least ONE participant destroys their random input, the setup is secure.

The Elara Protocol's ceremony will:

1. Include a minimum of 100 independent participants across 20+ countries
2. Accept contributions from anyone (public participation, open-source verification tools)
3. Publish all contributions and verification proofs on the DAM itself
4. Use the Zcash Powers of Tau ceremony model [26] (battle-tested, well-understood)

Phase 2: Progressive migration to zk-STARKs

zk-STARKs (the specified SOVEREIGN path — design-stage, §5.3) require NO trusted setup. They are transparent — all parameters are derived from public randomness. The tradeoff is larger proof sizes (~100 KB vs ~288 bytes for SNARKs).

As hardware improves and STARK proof compression advances, the protocol will migrate PRIVATE and RESTRICTED classifications from SNARKs to STARKs:

- **Year 0-3:** SNARKs for PRIVATE/RESTRICTED (compact proofs, trusted setup)
- **Year 3-5:** Dual support (SNARKs and STARKs accepted)
- **Year 5+:** STARKs only (no trusted setup dependency)

The algorithm agility mechanism (Section 4.4) makes this migration seamless — old SNARK-based proofs remain valid; new proofs use STARKs.

Phase 3: Long-term (post-quantum ZKPs)

Research into post-quantum zero-knowledge proofs is active. The protocol's algorithm agility ensures it can adopt new ZKP systems (lattice-based ZKPs, hash-based ZKPs) as they mature, maintaining both quantum safety and zero-knowledge properties.

11.12 Formal Consensus Specification

The gap: “Witness accumulation” is described conceptually throughout this paper but not formally specified. A reviewer expects Byzantine fault tolerance analysis, safety and liveness guarantees, and formal proofs or at minimum, precise algorithm specification.

Specification: Adaptive Witness Consensus (AWC) — diversity-discount BFT on a DAG

The Elara Protocol uses a consensus mechanism designed for its unique properties: threshold settlement with continuous trust accumulation, partition tolerance. AWC is an **Attestation-Weighted Consensus** layered over a DAG ledger with a diversity-weighted discount (§11.15) on correlated stake; it is internally also called **MESH-BFT** in code and tests (the mesh-bft cluster in the runtime is this mechanism).

How it relates to prior art. AWC is not a new Byzantine-fault-tolerance family; it is a composition of three well-studied building blocks applied to a DAG-sealed ledger:

1. **Stake-weighted probabilistic witness selection** — weighted reservoir sampling via Efraimidis & Spirakis (2006, *Information Processing Letters* 97(5)), used here for per-zone VRF committee picks (§11.18) without floating-point drift (u128 cross-multiplication).
2. **Per-record / per-epoch Byzantine agreement at the 1/3 safety bound** — directly descended from Castro & Liskov's Practical Byzantine Fault Tolerance (OSDI 1999), adapted so the “primary” is replaced by a VRF-selected witness committee and the “replicas” are the attestors of a given epoch seal.
3. **Two-layer finality** — a settlement/finality split analogous to Buterin & Griffith's finality gadget (Casper FFG, 2017): Layer-1 is per-record causal attestation (optimistic, ~seconds), Layer-2 is epoch-seal settlement at

$\geq 2/3$ diversity-weighted attested stake (durable, $\sim$ epoch interval).

AWC's novel contribution is (4) the **diversity discount** $d(n, W)$ (§11.15): a scalar in $[0, 1]$ that shrinks effective stake as the attester set shows correlation in `WitnessProfile` fields (same org, same ASN, same subnet). A correlated-stake attacker with a majority of raw stake but a single profile collapses to a minority of *effective* stake, so the $2/3$ threshold they cross is not a safety threshold. This is what makes AWC a meaningful strengthening of stake-weighted BFT for open-membership DAM deployments, not a rebrand of PBFT.

Definitions:

- **Record r** — a validation record on the DAM
- **Witness set $W(r)$** — the set of nodes that have attested to record r
- **Record trust score $T(r)$** — a continuous value in $[0, 1]$ representing network confidence in a specific record
- **Node trust score $T(n)$** — a continuous value in $[0, 1]$ representing a node's accumulated reputation, defined as $T(n) = 1 - 1/(1 + \text{attestation_count})$ where `attestation_count` is the number of honest attestations by that node. Node trust is earned through sustained honest behavior and cannot be purchased.
- **Witness weight $w(n)$** — the trust weight of node n 's attestation, derived from its node trust score, stake, and age

Two levels of trust: The protocol distinguishes between **node trust** (a node's reputation, earned over time) and **record trust** (network confidence in a specific record, computed from its witness set). Node trust feeds into record trust — a record witnessed by high-trust nodes accumulates record trust faster. The node-level formula is specified separately; this section specifies the record-level formula.

Trust score computation:

$$T(r) = 1 - \prod (1 - w(n) \times d(n, W)) \quad \text{for all } n \text{ in } W(r)$$

Where $d(n, W)$ is a **correlation discount** factor in $(0, 1]$ that reduces a witness's marginal contribution based on its independence from other witnesses in the set:

$$d(n, W) = 1 / (1 + \sum \text{corr}(n, m) \text{ for all } m \neq n \text{ in } W(r))$$

$$\text{corr}(n, m) = \alpha \times \text{same_org}(n, m) + \beta \times \text{same_subnet}(n, m) + \gamma \times \text{same_zone}(n, m)$$

$$\text{where } \alpha=0.5, \beta=0.3, \gamma=0.2$$

Witnesses from the same organization, IP subnet, or geographic zone contribute progressively less marginal trust. This prevents trust inflation through correlated attestation (e.g., a company running 1,000 witness nodes in one datacenter).

This formulation means: - Zero witnesses $\rightarrow T = 0$ (local only) - One high-weight independent witness ($w=0.5, d=1.0$) $\rightarrow T = 0.5$ - Multiple independent witnesses $\rightarrow T$ approaches 1 asymptotically - Correlated witnesses (same org/subnet) $\rightarrow$ diminishing returns - Trust never reaches exactly 1 (absolute certainty is impossible)

Layered Consensus Model (v0.7.2)

At production scale (quintillions of records across millions of zones), per-record attestation tracking becomes infeasible — unbounded state accumulation. The protocol uses a three-layer consensus model that preserves the per-record trust formula while achieving bounded-state epoch-level finalization.

Layer 1 — Immediate Validation (per-record, sub-second):

When a record arrives at a zone's witnesses, they perform immediate validation: - Dilithium3 / SPHINCS+ signature verification - Wire format and schema validation - Balance sufficiency (for beat operations) - Entropy score assessment (anti-spam) - Duplicate detection

The per-record trust formula $T(r)$ applies here. Each witness that validates a record contributes to its trust score. Records that pass Layer 1 are assigned status **Pending** and enter the current epoch's candidate set.

Layer 2 — Epoch Finalization (per-batch, anchor-proposed):

Each zone designates anchor nodes (high-trust witnesses with sealing authority, selected by VRF). At regular intervals (the anchor's seal loop ticks at a fixed configured interval, default 60s; a load-driven per-zone *adaptive* interval of 5–60s is computed and displayed but does not yet gate sealing, so quiet zones currently seal at the fixed interval — load-driven per-zone cadence is a design-stage item), the anchor proposes an epoch seal:

```
EpochSeal {
    epoch_number,          // per-zone counter (no cross-zone
conflicts)
    zone_path,             // hierarchical zone identifier
    merkle_root,           // SHA3-256 Merkle root over all records
in this epoch
    record_count,          // number of records in this epoch
    zone_balance_total,    // sum of all account balances in this
zone
    previous_seal_hash,    // chain of seals back to genesis
    zone_registry_root,    // Merkle root of all known zones (not
full list)
    zone_registry_delta,   // only zone changes since last seal
    vrf_output + proof,    // per-epoch seal seed (Dilithium3 hash-
based selection, alg=0x11; output is publicly computable –
unpredictable only until the prior epoch's seal is known)
    anchor_dilithium3_sig  // post-quantum signed by proposing
anchor
}
```

The anchor proposes which records are in the epoch. Witnesses verify: (a) they have the same records, (b) all records pass Layer 1 validation, (c) the Merkle root matches their independent computation, (d) the `zone_balance_total` matches their ledger. If verified, witnesses attest to the seal by signing it.

The epoch boundary is deterministic by construction — the anchor's proposal defines what's in the epoch. Records arriving after the proposal go into the next epoch. This eliminates clock-drift consensus failures.

A record's status advances to **Sealed** upon inclusion in a proposed epoch seal, and to **Finalized** when the epoch seal accumulates attestations from witnesses controlling >67% of the zone's staked weight.

Layer 3 — Post-Epoch Challenges (per-record, fisherman):

After an epoch is sealed, fisherman nodes can challenge individual records within it: - Challenge specifies: epoch number, zone, challenged record hash, evidence, Merkle proof of inclusion - Jury selected via VRF (existing fisherman mechanism, Section 11.1) - If upheld: the anchor that proposed the epoch receives FULL slash (they included the bad record). Attesting witnesses receive REDUCED slash (10-20% of standard, recognizing they attested to a batch). Non-attesting witnesses receive no penalty. - A record surviving the challenge window (e.g., 24 hours) without challenge advances to **Anchored** status — the highest confirmation level.

Confirmation levels:

Level	Definition
Pending	Layer 1 validated, not yet in an epoch seal
Sealed	Included in an anchor-proposed epoch seal
Finalized	Epoch seal has >67% stake-weighted diverse attestations
Anchored	Finalized + challenge window elapsed with no upheld challenges

Safety guarantee:

An epoch seal attested by witnesses controlling >67% of staked weight in a zone is considered **zone-settled**. All records within a zone-settled epoch are finalized. No conflicting epoch seal (different Merkle root for the same epoch number) can achieve settlement — the standard BFT bound applies to epoch seals exactly as it applied to per-record attestation. The MESH-BFT diversity-weighted safety property (stated as Theorem 1 in the formal specification; a TLA+ machine-checked proof is pending — see the implementation-status box below) is expected to hold because the diversity function $d(n, W)$ applies to the set of epoch seal attestors.

Liveness guarantee (honest-majority, observed):

Under honest-majority (>50% of staked weight in a zone held by honest, online, non-correlated nodes) and partially-synchronous network assumptions, epoch seals reach 2/3 diversity-weighted attestation in $O(\log_d N) \times \text{epoch_interval}$, where d is the average out-degree of the gossip overlay and N is the zone's attester count. The anchor-proposed model ensures deterministic epoch boundaries even under network asynchrony; seal propagation and quorum accumulation are the dominant terms.

Observed on the testnet (2026-04-22 to 2026-04-23, 129-sample reading): P50 seal settlement was $\sim 2 \times \text{epoch_interval}$ in practice (mean ~ 365 s against the then-default 120s epoch; the default/cap has been 60s since 2026-04-29), with a long tail of witness-propagation lag on NAT'd peers. The $O(\log_d N)$ bound is the asymptotic ceiling under honest majority; the $2\times$ observed constant is the small-network operational cost of gossip propagation rather than a violation of the bound. See the §11.12 implementation-status box and the seal-attestation-latency histogram on each node's `/metrics`.

Partition behavior:

During a network partition, each partition continues sealing epochs independently. Zone-scoped epoch numbers are per-zone, so different partitions' epochs do not conflict. When partitions merge:

1. DAM tips connect across fragments ($W.parents = [tip_A, tip_B]$)
2. Epoch seal chains from both partitions are preserved and verifiable
3. Cross-partition records become verifiable via Merkle proofs from either partition's epoch seals
4. No fork choice — both histories are valid by the mesh property of the DAM

Formal properties (preserved from MESH-BFT):

- **Agreement:** If honest node A considers epoch E zone-settled, all honest nodes in the same zone will eventually consider E zone-settled (assuming partition heals)
- **Validity:** Only records signed by valid keypairs can be included in epoch seals
- **Termination:** Every record propagated to honest nodes will be included in an epoch seal within bounded time — $O(\log_d N) \times \text{epoch_interval}$ under honest majority; observed P50 was $\sim 2 \times \text{epoch_interval}$ on the testnet
- **Partition safety:** Epochs settled before a partition remain settled in all resulting partitions

Formal-proof status (honest). This section is a *specification*, not a formal proof. Three safety/liveness properties are targeted for machine-checked proof work:

1. **Diversity-weighted safety** — correlated adversaries with $k \geq 2$ Sybil identities sharing a `WitnessProfile` cannot achieve double settlement regardless of stake, because the diversity discount reduces effective stake exponentially.
2. **Post-quantum safety under one-algorithm break** — dual-algorithm signing (ML-DSA-65 + SLH-DSA) keeps consensus safe even if one of the two primitives is completely broken, with composite security $\lambda_d + \lambda_s$.
3. **Per-record causal finality** — under the layered model, records reach confirmation in $O(1)$ epoch rounds and $O(\log n)$ wall-clock time, independent of total network size.

A TLA+ specification and Monte Carlo simulator are queued as follow-on artifacts (target 4Q 2026); a companion paper will be published alongside the formal specification once proofs are mechanized. Until then readers should treat §11.12 as the protocol's *design intent* under the honest-majority / diversity-weighted assumption, and audit a testnet's observed behavior (AWC settlement rate, seal-attestation-latency histogram, diversity counters on `/metrics`) for empirical evidence. No “simulation, 3600 scenarios, 0 violations” result is claimed here — the simulator has not been run on this codebase yet.

Correlation discount in zone-scoped attestation: Within a zone, the `same_zone` correlation discount ($\gamma=0.2$) does not apply to epoch seal attestation — all witnesses are necessarily in the same zone. The `same_org` ($\alpha=0.5$) and `same_subnet` ($\beta=0.3$) discounts remain active and prevent trust inflation from correlated witnesses within a zone. Cross-zone diversity is achieved through the zone registry mechanism and cross-zone Merkle proof verification, not through within-zone attestation diversity.

Scalability: Consensus state is bounded by $\text{pending_epochs} \times \text{subscribed_zones}$, not by total records. If the design targets of 10T records/day across 1M zones are reached, epoch length adapts within a 5-second floor (saturated zones above ~20 rec/s) to 60-second ceiling (quiet zones) band: a quiet zone produces 1,440 epoch seals per day, a saturated zone at most 17,280. A witness node in 3 saturated zones tracks at most ~52,000 epoch seals/day — trivially manageable, since only `pending_epochs` are unsettled at any moment and each seal is a small fixed-size record.

Super-seal checkpoint consolidation (v0.7.9+). Per-epoch seals are still $O(\text{time})$ — at 60-second P50 epochs, a zone produces 1,440 seals per day and 525,600 per year. Light clients verifying historical state from genesis to today would need to fetch every seal. The protocol consolidates seals into **super-seals**: every N epochs (default $N=64$, the `SUPER_SEAL_INTERVAL` constant), the anchor pool aggregates the Merkle roots of those N epoch seals into one super-seal, Dilithium3-signed once over the combined root. A light client verifying super-seals skips $(N-1)/N$ signature verifications — at $N=64$, that's $63/64 \approx 98.4\%$ fewer signatures, or $\sim 64\times$ compression for a year-long chain. A super-seal includes: `super_seal_id`, `zone`, `start_epoch`, `end_epoch`, `epoch_seal_merkle_root` (SHA3-256 root over the N epoch seals' canonical bytes), `creator_public_key`, and the Dilithium3 signature. Super-seals are emitted by anchors and propagated like any other record. Light clients doing cold-start fetch `/super-seals/latest`, verify the signature against the anchor registry, then fetch per-epoch Merkle proofs for only those records they care about (e.g., their own account). The composition is sound under §4.2 post-quantum assumptions: SHA3-256 is treated as a random oracle, Dilithium3 is EUF-CMA-secure under MLWE, and a super-seal's `epoch_seal_merkle_root` being a correct commitment to N valid epoch seals reduces (by Merkle-tree soundness) to the verifier seeing at most 2^{128} effective bits of adversarial work.

Super-seal cross-peer verification. A bootstrapping light client cross-verifies each super-seal's `epoch_seal_merkle_root` against up to 3 secondary peers' `/super-seals/{id}/integrity` endpoint. A peer with an incompatible root at the same (`zone`, `start_epoch`, `end_epoch`) triggers rejection — the client aborts the cold-start and retries with a different primary. Silence (peer has no super-seal for that range) is not disagreement. For tamper-resistance beyond Merkle-root agreement, the client also fetches the super-seal's wire bytes via `/records/fetch`, recomputes `record_hash`, and verifies the Dilithium3 signature against `creator_public_key` — this catches a colluding-seed set that all agree on a forged root but whose signatures don't verify. If record-bytes fetch fails, the skip is aborted rather than proceeding with unverified marks.

Implementation status: AWC/MESH-BFT is implemented in the Elara Runtime (Rust, ~350k LOC, 5,700+ tests in the full suite). A multi-node testnet validated zone-based consensus with correlation discounting, dual-signature verification (Dilithium3 + SPHINCS+), gossip propagation ($p_{50} \sim 250\text{ms}$, $p_{90} \sim 500\text{ms}$ on the small co-located testnet), beat transfers and staking, snapshot-based bootstrap (~3 seconds), and failure recovery; all nodes maintained identical supply (10B beats) across kills and restarts. The layered epoch-seal model is implemented: the first layered seal reached 2/3 diversity-weighted settlement on the testnet, and the Layer-2 settlement histogram accumulates samples while the same-zone diversity discount γ sits at

its automatic honest-degradation value of 0 — for a single-bucket witness set γ is 0 by construction ($\gamma_{\text{effective_scaled}}$: γ activates only once the witness set spans ≥ 2 distinct geographic buckets and ramps to its full 0.2 by 12 witnesses), so it engages automatically as the set diversifies toward mainnet. ZK proof scaffolding is in place with a fail-closed verifier; the STARK / lattice migration is tracked separately and is not yet deployed.

11.13 Dispute Resolution

The gap: The paper states that conflicting claims are “preserved, not resolved by deletion.” But preservation is not resolution. When two entities claim authorship of the same work, someone or something must eventually decide.

Solution: Three-Tier Dispute Resolution Framework

Tier 1: Automated Resolution (no human involvement)

The protocol automatically resolves clear-cut cases:

- **Temporal priority with causal proof:** If record A has a causal anchor (DAM-verifiable timing) earlier than record B, and both claim the same content hash, A is marked as the **prior claim**. B is not deleted but is annotated as a **subsequent claim**. This happens automatically.
- **Incremental chain vs. single hash:** If one claimant has a chain of drafts (Section 11.7, Tool 1) and the other has only the final hash, the chain-holder receives an automated **provenance score** boost. Not a verdict — a weighted signal.
- **Identical key families:** If both claims come from keys in the same organizational identity, it’s an internal matter. The protocol flags it but takes no action.

Tier 2: Community Arbitration (decentralized human judgment)

For cases that automated analysis cannot resolve, the protocol supports **arbitration panels**:

- Any party to a dispute can invoke arbitration by staking beats (refundable if the claim is upheld)
- A panel of 5-11 arbitrators is randomly selected from a pool of staked, reputable nodes with HUMAN entity type
- Arbitrators review the evidence on the DAM: timestamps, causal chains, witness patterns, provenance scores, incremental history
- Majority vote produces an **arbitration record** on the DAM — a non-binding recommendation that carries significant trust weight
- Losing party can appeal once (new panel, larger size)

Arbitration records are not protocol enforcement — they are advisory. But they carry weight: a record with a favorable arbitration result receives a trust boost; an unfavorable one receives a penalty.

Tier 3: Legal Integration (real-world enforcement)

For disputes that require legal force (copyright infringement, patent priority, contractual obligations), the protocol provides:

- **Court-admissible evidence export:** A standardized format that packages a validation record with its full causal chain, witness attestations, Merkle proofs, and timestamp verification into a document that legal systems can interpret. Designed in consultation with digital forensics standards (ISO 27037).
- **Expert witness protocol:** Anchor nodes can generate signed attestations explaining the technical meaning of DAM records in language suitable for legal proceedings.
- **Jurisdiction mapping:** Validation records can optionally include jurisdiction metadata, enabling creators to indicate which legal system they consider authoritative for disputes.

The protocol does not replace law. It provides the strongest possible evidence for legal systems to use. A DAM record with causal anchoring, thousands of witnesses, and an incremental creation chain is the digital equivalent of a notarized document, a chain-of-custody record, and a witness testimony — combined.

11.14 Network Topology and Peer Discovery

The gap: The paper describes what happens when nodes communicate but never specifies how nodes find each other. Without a peer discovery mechanism, the network cannot form.

Solution: Hybrid Discovery with Kademlia DHT

The Elara Protocol uses a three-layer peer discovery system:

Layer A: Bootstrap Nodes

At installation, every Elara client ships with a hardcoded list of bootstrap nodes — geographically distributed servers operated by the foundation and early community members. These serve one purpose: introducing new nodes to the network. They are not privileged in any other way.

```
Bootstrap list (example):
bootstrap-eu.elara.network:4001
bootstrap-us.elara.network:4001
bootstrap-asia.elara.network:4001
bootstrap-africa.elara.network:4001
```

The bootstrap list is updatable through protocol governance. If all bootstrap nodes go offline simultaneously, nodes that already know peers continue operating — bootstrap is only needed for first contact.

All bootstrap traffic uses the ElaraPQ transport defined in §4.7 — including the very first packet from a freshly-installed client. The hybrid Curve25519 + ML-KEM-768 handshake is the *only* node-to-node wire protocol on mainnet; bootstrap is not a special case. The bootstrap list itself is signed by the foundation reserve multisig and embedded in the binary tarball, so a client can verify peer authenticity before sending its first frame.

Layer B: Kademlia DHT (Distributed Hash Table)

Once connected to at least one peer, nodes join a Kademlia-based DHT (a widely deployed algorithm in peer-to-peer networks). Kademlia provides:

- $O(\log n)$ lookup for any node in the network
- Self-healing: the routing table automatically repairs when nodes leave
- Resistance to targeted attacks: no single node is critical for routing
- NAT traversal via hole-punching, where every UDP datagram carries an ElaraPQ-encrypted payload (§4.7); plain unencrypted UDP is forbidden on mainnet

Each node maintains a routing table of $\sim 20 \times \log_2(N)$ entries. For a million-node network, this is ~ 400 entries — negligible memory.

Layer C: Local Discovery

For devices on the same local network (IoT deployments, mesh networks), the protocol uses mDNS/DNS-SD (multicast DNS / Service Discovery) for zero-configuration local peer discovery. A sensor and its gateway find each other without any internet connectivity.

For Bluetooth-capable devices, BLE advertisements enable peer discovery within ~ 100 meters. This enables the mesh-networking scenarios described in the Emergency Protocols (Section 12.3).

Gossip Protocol for Record Propagation:

Once peers are discovered, validation records propagate via an epidemic gossip protocol:

1. Node creates or receives a new record
2. Node selects $\sqrt{n}$ random peers from its routing table (where n = number of known peers)
3. Node forwards the record to selected peers
4. Recipients repeat the process for records they haven't seen

With $\sqrt{n}$ fan-out, theoretical propagation completes in ~ 2 -3 rounds. In practice, duplicate messages, network latency, and partial peer overlap increase this to ~ 6 -10 gossip rounds for 1 million nodes — projected under 15 seconds on Earth-zone networks (modelled from epidemic-gossip theory; untested at this scale).

11.15 Zero-Knowledge Proof Feasibility on Constrained Devices

Implementation status — DESIGN-STAGE. The proving-cost figures below are projections for the *specified* zk-SNARK construction (§5.3), not measurements of running code. The Phase-1 SHA3-256 commitment scheme generates proofs in microseconds on every device class; this analysis governs a future zk-SNARK deployment.

The problem: generating a zk-SNARK proof (in the specified construction) requires significant computation:

Device	zk-SNARK Generation (projected)	RAM Required
Modern laptop	1–5 seconds	1–4 GB
Flagship smartphone (2026)	3–10 seconds	500 MB–2 GB
Budget smartphone (\$30)	15–60 seconds	200 MB–1 GB

Raspberry Pi 4	10–30 seconds	500 MB–2 GB
ESP32	Infeasible	520 KB total RAM

The Kenya teenager’s \$30 phone can generate a proof, but it takes up to a minute and drains the battery. An ESP32 cannot generate a proof at all. This limits PRIVATE classification to devices with meaningful compute.

Solution: Layered Privacy by Device Capability

Capable devices (phones, laptops, servers): Generate privacy proofs locally (Phase 1: SHA3-256 commitments; the zk-SNARK path is design-stage per the banner above). PRIVATE, RESTRICTED, and SOVEREIGN classifications supported.

Constrained devices (IoT, ESP32): Three options for privacy-preserving validation:

1. **Gateway-delegated proof generation** — the constrained device sends content to a trusted gateway over a secure local channel. The gateway generates the ZK proof. Trust boundary: the gateway sees the content (acceptable within a single deployment, e.g., factory sensors reporting to a factory server).
2. **Deferred proof generation** — the device validates as PUBLIC initially (just the hash), then a more capable device generates a ZK proof later, converting the record to PRIVATE. The record’s DAM position is preserved — it does not lose its timestamp.
3. **Symmetric encryption fallback** — for IoT deployments where even the gateway should not see individual readings, the device encrypts the reading with a pre-shared key before hashing. The hash is of encrypted content. Decryption requires the key. This is not zero-knowledge in the cryptographic sense, but it provides practical privacy for the constrained device use case.

Budget smartphones — optimization path:

ZK proof generation on smartphones is an active area of research and optimization. Specific measures for the Elara Protocol:

- Use Groth16 (most compact and fastest zk-SNARK) for the standard proof circuit
- Pre-compute the proving key once and cache it (~50 MB)
- Offload to GPU/NPU on devices that support it (most 2025+ smartphones have ML accelerators that can be repurposed)
- As STARK compression advances, migrate to STARKs which have simpler prover algorithms

The protocol’s algorithm agility ensures that as ZKP technology improves, constrained devices gain privacy capabilities without protocol changes.

11.16 Censorship Resistance

The threat: A government orders all ISPs within its jurisdiction to block Elara Protocol traffic. Or mandates that all domestic nodes refuse records from certain creators (political dissidents, journalists, specific

organizations). State-level censorship is the most powerful adversary the protocol faces.

Defense Layer 1: Traffic Obfuscation

The protocol supports tunneling ElaraPQ frames inside outer carriers so that deep-packet-inspection middleboxes cannot easily fingerprint Elara traffic. The ElaraPQ handshake and AEAD (§4.7) remain unchanged in every case; only the outer wrapper differs:

- **Tor pluggable transports:** Elara nodes can speak ElaraPQ inside a Tor obfs4 / Snowflake / meek tunnel, hiding the fact that the underlying traffic is Elara at all.
- **WireGuard / Tailscale / SSH tunneling:** ElaraPQ inside a WireGuard datagram or SSH port-forward — useful in network environments where the outer protocol is allowlisted.
- **Steganographic encoding:** Validation records embedded in innocent-looking traffic (images, video calls, DNS queries) for extreme cases. Per-message overhead is high; reserved for one-shot record exfiltration, not bulk gossip.
- **Bridge relays:** Unlisted relay nodes operated by volunteers outside the censoring jurisdiction, accessible via out-of-band key exchange. The bridge speaks ElaraPQ inward and any allowlisted outer transport outward.

The protocol explicitly does **not** support a “domain-fronting mode” that masquerades as classical HTTPS to a permitted CDN. Earlier drafts of this section listed domain fronting as a pluggable transport; that recommendation is retired by §4.7. Domain fronting requires a classical TLS outer layer, which leaks per-connection metadata (TLS ClientHello fingerprints, SNI when not encrypted via ECH) and breaks the post-quantum forward-secrecy goal of the transport. Operators who need DPI bypass in a censored environment should use the carriers above, not bake classical TLS into the protocol.

Defense Layer 2: Partition Resilience (Already Built-In)

The DAM’s partition tolerance (Section 7.3) means censorship IS a partition. If a government blocks cross-border traffic:

1. The domestic zone continues operating independently
2. Domestic validations remain cryptographically valid
3. When the censorship lifts (regime change, policy reversal, VPN access), the zones merge
4. Nothing is lost — the domestic branch of the DAM is fully preserved

A government can slow the network. It cannot kill records that already exist on the DAM, and it cannot prevent domestic validation from continuing.

Defense Layer 3: Mesh Networking Fallback

In extreme censorship scenarios (internet shutdown), devices can form local mesh networks:

- **Bluetooth mesh:** Phone-to-phone, ~100 meter range, chain across a city
- **LoRa mesh:** 10+ km range, low bandwidth but sufficient for compact validation payloads (full PQC records require gateway relay)

- **Sneakernet:** Physical transfer of DAM data via USB drives, SD cards — the protocol supports offline sync by design

Records validated during an internet blackout propagate when any node in the mesh eventually reaches the global network. The DAM is patient. It can wait.

Defense Layer 4: Geographic Distribution of Anchor Nodes

The protocol requires anchor nodes on at least 3 continents for the decentralization threshold (Section 11.4). Once that threshold is reached, no single government can compel all anchor nodes to comply — though the pre-launch network currently runs on a single-region development fleet and has not yet reached it. Even if a government seizes all domestic anchor nodes, the global DAM continues — and the domestic zone's records are already replicated internationally.

11.17 Beat Supply Model

Scope: Public permissionless network only. This section applies to the public permissionless network. Private deployments have no beat supply.

Full specification: The complete beat supply model — including supply mechanics and conservation economics — is specified separately. This section addresses only the adversarial implications.

The public network's beat supply model is designed as a **conservation system** — beats circulate between producers (witnesses) and consumers (record submitters) rather than being continuously minted. This design choice has specific security implications:

- **No inflationary dilution attack** — because supply is fixed, an attacker cannot devalue existing stakes by inflating the supply
- **MEV prevention** — witness attestation order does not affect outcome (trust scores are order-independent — the same witnesses produce the same trust score regardless of when they attest). This eliminates Maximal Extractable Value by design. There is nothing to extract from reordering.
- **No gas fee exploitation** — because the protocol does not charge per-transaction gas fees, there is no fee market to manipulate

The complete supply model, distribution schedule, and economic equilibrium analysis are specified separately.

11.18 Protocol Upgrade Mechanism

The gap: A protocol designed to outlive its creators must evolve. New cryptographic algorithms, new ZKP systems, new device types, unforeseen requirements. How does the protocol upgrade without breaking the network?

Solution: Semantic Versioning with Soft Fork Default

Version field in every record:

Every validation record includes a protocol version number. Nodes advertise the versions they support. This enables:

Soft forks (backward-compatible changes): - New optional fields in validation records - New classification levels - New entity types - Optimized encoding formats

Soft forks require no coordination. New nodes produce new-format records. Old nodes ignore fields they don't recognize. Both coexist on the DAM. Governance vote recommends adoption; individual nodes upgrade at their own pace.

Hard forks (breaking changes): - Changes to the consensus mechanism - New required fields in validation records - Deprecation of cryptographic algorithms

Hard forks require governance approval (>67% supermajority per Section 10.3) and follow a strict process:

1. Proposal published (with reference implementation)
2. 90-day discussion period
3. Governance vote
4. If approved: 180-day transition window
5. During transition: both old and new formats accepted
6. After transition: old format deprecated (but old records remain valid)

Emergency upgrades (critical security patches):

If a cryptographic algorithm is broken or a critical vulnerability is discovered, the protocol supports **emergency governance** — a fast-track process:

1. Security advisory published by any anchor node
2. Emergency vote: 48-hour window, >75% supermajority required
3. If approved: 30-day transition (vs. normal 180 days)
4. Anchor nodes enforce the new version; other nodes have 30 days to upgrade

The emergency mechanism is intentionally rare and high-threshold. It exists for true existential threats (e.g., quantum computing breaks ML-DSA (FIPS 204, "Dilithium") earlier than expected), not for feature additions.

Algorithm deprecation lifecycle:

ACTIVE → algorithm used for new signatures
LEGACY → algorithm accepted for verification, not recommended for new signatures
DEPRECATED → algorithm accepted for verification of old records only, rejected for new
ARCHIVED → algorithm documented in protocol history, old records still verifiable
through algorithm agility (Section 4.4)

No algorithm is ever deleted from the protocol's specification. A record signed with Dilithium3 in 2026 must be verifiable in 3026 — even if Dilithium3 was deprecated in 2050. The verification code for every algorithm ever used is preserved in the protocol's reference implementation, explicitly tagged as archival.

11.19 Spam and Denial-of-Service Prevention

The attack: Layer 1 validation is free. An attacker generates millions of garbage validation records — random hashes, signed by valid keys — flooding the network. Each record is individually valid (correctly signed, properly formatted). The DAM fills with noise, relay and witness nodes waste bandwidth and storage, and legitimate records are drowned out.

This is the cost of “Layer 1 is always free.” Free creation means free spam.

Defense 1: Propagation Rate Limiting

Layer 1 (local validation) is unrestricted — an attacker can fill their own local DAG with garbage. But Layer 2 (network propagation) applies rate limits:

- Each identity is allowed **N records per hour** via the gossip protocol (default: 100)
- Records exceeding the rate limit are queued, not rejected — they propagate eventually, but slowly
- Witness nodes prioritize attestation of records from identities within their rate limit
- Rate limits scale with identity trust score — a trusted, long-standing identity gets higher limits

An attacker generating 1 million records per hour from a fresh identity would see 100 propagate immediately and 999,900 enter a slow queue. By the time they propagate, the identity’s anomalous behavior is flagged.

Cross-zone enforcement of global rate limits. Rate limits are per-identity GLOBALLY (10/50/200 records/day by trust tier), but with zone-scoped gossip a node in zone A only sees zone A traffic. An identity creating 200 records/day spread across 10 zones is within the per-zone window everywhere yet exceeds the global limit. Global enforcement is reconciled at epoch boundaries: each zone’s epoch seal includes per-identity record counts for that epoch. A monitoring process (or fisherman) detects identities exceeding global limits by summing their per-identity counts across zone epoch seals and submits a challenge. Global enforcement therefore rides on the existing seal stream — no synchronous cross-zone coordination is required, and the check is eventually-consistent over a one-epoch window.

Defense 2: Proof-of-Work for Burst Propagation

If a node needs to propagate more than its rate limit (legitimate use case: IoT gateway syncing a batch of sensor readings), it can solve a lightweight proof-of-work puzzle for each excess record. The puzzle difficulty is calibrated so that:

- Normal usage (under rate limit): zero computational cost
- Moderate burst (2–10x limit): seconds of compute
- Spam-scale burst (1000x+ limit): hours/days of compute — economically infeasible

This is the same approach used by Hashcash [24] (email anti-spam, 2002) and later adopted by blockchain networks. It adds no cost to honest users and makes spam expensive.

Defense 3: Content-Independent Duplicate Detection

Bloom filters at the zone level detect records with identical content hashes. If the same hash is submitted by different identities simultaneously (a classic spam pattern — resubmitting the same garbage with different keys), only the first propagation proceeds. Subsequent duplicates are annotated as conflicts but not relayed further.

Defense 4: Economic Filtering at Layer 2

Witness nodes choose what to attest. They are rational economic actors — attesting costs computational resources (PoWaS). No witness will spend resources attesting to records from identities with zero trust, anomalous patterns, or rate-limit violations. Spam records exist on the DAM but accumulate zero witnesses and zero trust. They are dead weight — present but invisible to anyone querying the DAM with a minimum trust threshold.

11.20 Nothing-at-Stake and Witness Incentive Alignment

The problem: In proof-of-stake systems, validators can sign conflicting blocks at no cost because there is “nothing at stake” — they don’t burn energy like proof-of-work miners. Does the Elara Protocol’s witness accumulation have the same problem? Can a witness attest to conflicting records (two creators claiming the same work) without penalty?

Analysis:

In blockchain PoS, nothing-at-stake matters because fork choice affects which chain becomes canonical. Attesting to both forks is profitable because you earn rewards on whichever one wins.

In the Elara Protocol, this dynamic does not apply because **both branches of a conflict are preserved**. There is no fork choice. Both records exist. Neither is “canonical.” The trust score mechanism (Section 11.12) handles precedence through temporal and causal analysis, not by choosing a winner.

However, a related problem exists: **indiscriminate witnessing**. A witness could attest to everything — every record from every identity — to maximize rewards without performing any quality check. This degrades the trust signal.

Solution: Witness Reputation Scoring

Witnesses earn not just beats but **reputation** based on the quality of their attestations:

$\text{reputation_delta} = f(\text{record_outcome})$

Record never disputed:	+1 reputation
Record disputed, witness sided with winner:	+2 reputation
Record disputed, witness sided with loser:	-5 reputation
Record flagged as spam/anomaly:	-10 reputation

A witness that attests to everything — including spam and disputed records — rapidly loses reputation. Reputation loss reduces the weight of future attestations (Section 11.12), reducing earned rewards. Indiscriminate witnessing is therefore economically irrational.

A witness that carefully evaluates records before attesting — checking for rate limit compliance, duplicate content, identity trust, and causal consistency — maintains high reputation and earns more. Selective, honest witnessing is the Nash equilibrium. The complete witness incentive model and reward mechanics are specified separately.

The related problem of **reputation escape** — where an entity destroys a damaged identity to start fresh — is addressed in Section 11.33, which introduces identity continuity scoring and organizational binding as countermeasures.

Protocol Mechanics (11.22–11.33)

11.22 Cross-Zone Trust Reconciliation

The problem: Earth zone has 100,000 witnesses. Mars zone has 50. When zones merge after a partition, a record with 10,000 Earth witnesses and a record with 45 Mars witnesses have vastly different absolute trust scores — but within their zones, both may represent near-universal consensus. How do trust scores combine meaningfully?

Solution: Relative Trust Normalization

Trust scores are computed **relative to zone size**, not as absolute witness counts:

$$T_{\text{zone}}(r) = 1 - \prod (1 - w(n) \times d(n, W_{\text{zone}})) \quad \text{for all } n \text{ in } W(r) \cap \text{zone}$$
$$T_{\text{global}}(r) = \text{weighted_average}(T_{\text{zone}_i}(r) \text{ for each zone } i \text{ that has witnessed } r)$$
$$\text{where weight}_i = \ln(\text{zone_size}_i + 1) / \sum \ln(\text{zone_size}_j + 1)$$

Where $d(n, W_{\text{zone}})$ is the same correlation discount defined in Section 11.12, computed within the zone's witness set. This ensures that correlated witnesses within a zone do not inflate per-zone trust scores.

Interpretation:

- A record witnessed by 45 of 50 Mars nodes has $T_{\text{mars}} = \sim 0.95$ (very high within Mars)
- A record witnessed by 10,000 of 100,000 Earth nodes has $T_{\text{earth}} = \sim 0.85$
- Upon merge, T_{global} reflects both: ~ 0.87 (weighted by zone size logarithmically)

The logarithmic weighting prevents Earth's massive node count from completely dominating. Mars's consensus matters proportionally — a smaller zone with near-unanimous agreement is a strong signal.

Edge case — conflicting records across zones:

If Earth and Mars both validated records claiming the same content hash by different creators during a partition:

1. Both records are preserved (no deletion)
2. Each record carries its zone-relative trust score
3. The conflict is flagged with a **partition conflict** annotation
4. Resolution follows the dispute framework (Section 11.13): temporal

priority via causal anchoring where possible, arbitration where not

Zone trust calibration:

Each zone publishes a **zone health metric** in its trust headers:

```
ZoneHealth {  
    active_witnesses:    count  
    total_staked:        beat amount  
    median_reputation:   reputation score  
    uptime_30d:          percentage  
}
```

This allows global trust calculations to discount zones with low participation or unhealthy metrics, preventing a tiny zone with 3 colluding nodes from injecting high-trust records.

11.22.1 Cross-Zone Merkle Proof Protocol (v0.7.2)

The problem: With zone-scoped storage, a node in Zone A does not hold Zone B's records. How does it verify a record from Zone B without trusting Zone B's nodes?

Solution: Self-Verifiable Records with Merkle Proofs

A record is self-verifiable when it carries: 1. The record itself (signed with ML-DSA-65 (FIPS 204, "Dilithium3")/SPHINCS+) 2. A Merkle proof: the sibling hashes from the record's leaf to the zone's Merkle root 3. The epoch seal containing the Merkle root (signed by the zone's anchor, attested by witnesses) 4. The epoch seal chain: `previous_seal_hash` links back to genesis

Verification without holding any Zone B data:

1. Verify record signature (Dilithium3) → authentic
2. Compute Merkle path from record hash using proof hashes → get root
3. Compare computed root with epoch seal's `merkle_root` → matches
4. Verify epoch seal's anchor signature (Dilithium3) → authentic
5. Verify epoch seal attestations → $\geq [2N/3]$ committee witnesses signed (mandatory finality quorum, see below)
6. Verify witness identities → staked, PoW, age requirements met
7. Verify epoch seal chain → `previous_seal_hash` chain to `genesis_authority`

Steps 1-3 are pure math (no network). Steps 4-7 require the epoch seal and witness identity data, which can be cached or requested via DHT.

Step 5 finality requirement (mandatory): the cross-zone proof must carry a `SealFinalityWitness` bundle — distinct Dilithium3 signatures from at least $[2N/3]$ members of the source zone's committee at `seal_epoch`, each over the canonical (`zone_path || seal_epoch || merkle_root || committee_hash`) bytes, plus a Merkle inclusion proof against the pinned `committee_hash`. Inclusion in a tentative (sub-quorum) seal is **not** sufficient — orphaned tentative seals violate the cross-zone conservation invariant. The verifier rejects with `committee_size` must be > 0 to enforce quorum when no committee is supplied, preventing legacy inclusion-only proofs from advancing.

Cross-zone proof request protocol:

1. Node A wants to verify record R from Zone B
2. DHT lookup: `sha256("elara-zone:" || zone_B_path) → find Zone B peers`
3. Request from Zone B peer: `GET /zone/B/proof/{record_hash}`
4. Response: { record, merkle_proof, epoch_seal, witness_attestations }
5. Node A verifies everything locally (steps 1-7 above)
6. Cache the epoch seal for future Zone B verifications

Proof size: For a zone with N records, Merkle proof is $\log_2(N) \times 32$ bytes. For 1 billion records: $30 \times 32 = 960$ bytes. Negligible bandwidth.

Cross-zone transfers use this mechanism. A transfer-lock record in Zone A is presented to Zone B with its Merkle proof and the source-zone finality-witness bundle ($\geq \lceil 2N/3 \rceil$ committee signatures over the source seal). Zone B verifies without contacting Zone A. The finality bundle is what makes the conservation invariant survive an orphaned source seal.

11.23 Search, Indexing, and DAM Usability

The gap: The paper specifies how records get onto the DAM but not how anyone finds them. With billions of records, the DAM is useless without search. How does a creator find their own records? How does a verifier find a specific claim? How does a researcher discover related work?

Solution: Multi-Layer Index Architecture

Layer A: Content Hash Lookup (exact match)

The simplest query: “Does this exact content exist on the DAM?”

Every node maintains a local hash index (key: `content_hash` → value: `record_id`). The Kademlia DHT (Section 11.14) enables global lookup: any node can find any record by its content hash in $O(\log n)$ hops.

This is how verification works: hash your content, look it up, see if someone already claimed it.

Layer B: Creator Index (identity lookup)

“Show me everything validated by this identity.”

Each node maintains an index of its own records. Anchor nodes maintain comprehensive per-creator indexes for the identities they’ve witnessed. Light clients query anchor nodes for creator-specific records.

Layer C: Semantic Search (Layer 3 — AI)

“Find all validated work related to quantum computing from 2026.”

The AI intelligence layer (Layer 3) builds semantic indexes over validation record metadata. This includes:

- Full-text search over PUBLIC metadata fields
- Category/tag-based filtering
- Temporal range queries
- Similarity search (finding related work via content fingerprints)

Semantic search is a Layer 3 capability — optional, beat-gated, and computationally expensive. It is not required for basic DAM operation but enables rich discovery for nodes that participate in the AI layer.

Layer D: Zone Catalogs

Each zone publishes a periodic **zone catalog** — a compact index of:

- Recently validated record IDs and metadata
- Active creators in the zone
- Popular content categories
- Cross-references to related records in other zones

Zone catalogs enable browsing and discovery without full-text search. They are small enough for light clients to download and cache.

User Interface Implication:

The protocol specification defines the data layer, not the UI. But reference implementations will include:

- **CLI tool:** elara validate, elara search, elara verify
- **Mobile app:** camera-based validation (photograph → hash → validate), gallery of own validated work
- **Web interface:** search, browse, verify — hosted by anchor nodes or community

End users see a simple app: create, validate, browse. The underlying DAM complexity is invisible.

11.24 Energy Consumption Analysis

The obligation: The paper discusses the energy costs of existing consensus mechanisms. Credibility requires disclosing the Elara Protocol's own energy footprint.

Estimation by component:

Layer 1 (local validation): Negligible. Hashing + signing = milliseconds of CPU time. Energy cost: ~183 μ J (~50.8 nWh) per validation on a smartphone. Orders of magnitude less than sending a text message.

Layer 2 (network propagation + witnessing):

- **Gossip propagation:** Network traffic comparable to a peer-to-peer file-sharing swarm. Each node sends/receives a compact record announcement (~1 KB header + hash, not the full ~4.5 KB record) to $\sqrt{n}$ peers per hop; full records are fetched on demand by interested nodes. Records propagate within their zone (Section 7), not globally — zone partitioning bounds the effective fanout. For a zone with ~10,000 active nodes, epidemic gossip is projected to converge in ~4 rounds of $\sqrt{n}$ fanout (modelled from epidemic-gossip theory; untested at this scale). Most records are created by leaf nodes and propagated to their zone's witness set, not broadcast to the entire network. The energy estimate below uses network-wide averages accounting for zone partitioning and leaf-node locality.

- **Witness attestation (PoWaS):** The proof-of-work component is calibrated to be lightweight — ~0.1 seconds of CPU per attestation (vs. ~10 minutes of ASIC-scale computation in proof-of-work blockchains). Energy per attestation: ~0.005 Wh. For 1,000 witnesses per record: ~5 Wh total.
- **Anchor node operation:** Comparable to running a modest server. ~100W continuous. Per anchor node per year: ~876 kWh. For 100 anchor nodes globally: ~87,600 kWh/year.

Layer 3 (AI-assisted analysis): Dependent on the model and hardware. Ollama running a 1.5B parameter model: ~50W during inference. Intermittent, not continuous. Per node per year (assuming 4 hours/day active): ~73 kWh.

Total network estimate at scale (1 million nodes, 100 anchor nodes):

Component	Annual Energy
Layer 1 (local validation)	< 1 MWh (negligible)
Layer 2 (propagation)	~2,000 MWh
Layer 2 (witnessing PoWaS)	~5,000 MWh
Anchor nodes	~88 MWh
Layer 3 AI (optional, ~10% of nodes)	~7,300 MWh
Total	~14,400 MWh/year

Comparison:

Network Type	Annual Energy
Proof-of-work blockchains	~150,000,000 MWh (150 TWh)
Proof-of-stake platforms	~1,500–2,600 MWh
Elara Protocol (estimated)	~14,400 MWh

The Elara Protocol at full scale would consume roughly **10,000x less energy than proof-of-work blockchains** (a projection at the design-target scale of ~1M nodes — the network does not yet operate at that scale, and both sides of the comparison are estimates). It consumes approximately 6x more than proof-of-stake platforms in absolute terms, but the comparison is not apples-to-apples: proof-of-stake networks process financial transactions (~30 TPS), while the Elara Protocol is designed to validate all forms of digital work at IoT scale (millions of records per day — a design target, not measured throughput) with quantum-safe cryptography. Layer 1 validation energy is negligible (~183 μJ per operation) — the dominant costs are network propagation, witness consensus, and optional AI analysis.

The PoWaS component is the largest contributor. If the Sybil resistance proves achievable without PoW (through reputation and staking alone), the energy footprint drops to ~9,400 MWh. Without optional Layer 3 AI, it drops further to ~2,100 MWh — comparable to proof-of-stake networks.

11.25 Harmful Content and Ethical Boundaries

The dilemma: The protocol validates all digital work. “All” includes content that society broadly agrees should not be preserved: child exploitation material, terrorist recruitment, bioweapon designs, doxxing databases. If the DAM is immutable and censorship-resistant, does it become a permanent haven for the worst of human output?

This is not a hypothetical. Every decentralized system faces this challenge — immutable ledgers and content-addressed storage networks have encountered cases of embedded or distributed harmful material. The Elara Protocol must have an answer.

Principle: The DAM stores hashes, not content.

A validation record contains a cryptographic hash — not the content itself. The hash of an illegal image is not an illegal image. It is a 64-character string of hexadecimal digits. It cannot be “viewed” or “consumed.” The harmful content exists wherever the creator stored it (their device, a file host), not on the DAM.

If the content is removed from all storage, the hash becomes an orphan pointing to nothing. The hash remains, the harm does not.

However: Validation records also carry metadata — operation identifiers, transfer memos, governance proposals, dispute descriptions. These are user-writable text fields. Without protocol-level constraints, metadata becomes a vector for embedding harmful text, links to illegal content, or communication channels for criminal coordination. Every node that stores and gossips a record becomes an unwitting host.

The Silk Road precedent established that platform operators bear criminal liability when their systems enable illegal content distribution — even when operators did not create the content. Node operators who propagate records containing drug transaction details, child exploitation references, or terrorist recruitment text face the same exposure. “The protocol doesn’t judge the payload” is not a legal defense when the payload is criminal communication carried in cleartext metadata fields on every node in the network.

The Elara Protocol therefore implements **structural hostility to content distribution** — not content filtering (which is an arms race), but architectural constraints that make the protocol unsuitable for hosting, distributing, or linking to content of any kind.

11.25.1 Defense Architecture Overview

The protocol implements eight independent defense layers, applied at the record ingestion gate (`insert_record_with_origin()`). All layers enforce on ALL entry paths — HTTP POST, WebSocket submission, gossip propagation, and internal record creation (faucet, rewards, epoch seals). No code path bypasses the gate.

Layer	Type	Action	Applied At
1. Metadata key allowlist	Structural	Reject unknown keys	Ingestion
2. Text field byte limits	Structural	Reject oversized text	Ingestion

3. URL rejection	Structural	Reject URLs in text	Ingestion
4. Tombstone suppression	Reactive	Suppress gossiped records	Post-storage
5. Browser-side enforcement	Defense-in-depth	Client-side validation	Pre-submit
6. Propagation bounds	Structural	24 entries, 2KB values	Ingestion
7. Identity ban list	Proactive	Block all records from identity	Ingestion (first check)
8. Content blacklist	Proactive	Block records matching terms	Ingestion

Layers 1–3 and 6 are **structural constraints** — they limit the protocol’s capacity to carry content, like TCP’s maximum segment size limits the payload of a single packet. They are engineering constraints, not content judgments.

Layers 4, 7, and 8 are **operator tools** — they give node operators the means to comply with their jurisdiction’s laws and protect themselves from criminal liability. They are not centralized censorship.

Layer 5 is **defense-in-depth** — client-side validation that prevents user confusion but is not a security boundary. Server-side enforcement is authoritative.

11.25.2 Metadata Key Allowlist (Layer 1)

Every metadata key in a validation record must appear in the protocol’s allowlist. The allowlist contains the complete set of keys used by all protocol operations: beat transfers, staking, governance, disputes, fisherman challenges, key rotation, algorithm sunset, epoch management, versioning, tombstoning, and collaboration.

Records with unknown keys are rejected at ingestion — they are never stored, never gossiped, never processed. This prevents arbitrary data injection through the metadata layer.

Six anti-rehypothecation keys are explicitly blocked: `derivative_op`, `wrap_op`, `collateral_op`, `tokenize_op`, `synthetic_op`, `lend_op`. Records containing these keys are rejected at ingestion.

11.25.3 Text Field Byte Limits (Layer 2)

User-writable text fields have per-field byte limits:

Field	Max bytes	Purpose
<code>beat_memo</code>	256	Transfer/burn memo
<code>beat_reason</code>	256	Mint/slash/reclaim reason
<code>governance_title</code>	128	Proposal title

<code>governance_description</code>	1024	Proposal description
<code>dispute_reason</code>	512	Dispute opening reason
<code>dispute_evidence</code>	1024	Evidence submission
<code>challenge_evidence[]</code>	512/entry	Fisherman evidence (array)
<code>challenge_appeal_reason</code>	512	Appeal text
<code>revocation_reason</code>	128	Key revocation reason
<code>rotation_reason</code>	128	Key rotation reason
<code>change_summary</code>	512	Version change description
<code>sunset_reason</code>	256	Algorithm sunset reason

These limits make the network unsuitable for embedding images (even base64-encoded), documents, or bulk content. A 256-byte memo field holds approximately 200 characters of text — enough for a transaction description, insufficient for meaningful content distribution.

11.25.4 URL Rejection (Layer 3)

All text fields reject URLs matching these patterns (case-insensitive):

- `http://`, `https://` — web links to external content
- `ftp://` — file transfer links
- `data:` — base64-encoded inline content (images, documents)
- `javascript:` — cross-site scripting vectors
- `magnet:` — torrent/peer-to-peer content links

This prevents records from serving as a link directory for illegal content. A record cannot contain a URL pointing to child exploitation material, a drug marketplace, or a terrorism recruitment site. The protocol is structurally unable to function as a content indexing or referral system.

11.25.5 Propagation Bounds (Layer 6)

Hard limits at the gossip layer constrain record size:

- **Maximum metadata entries:** 24 per record
- **Maximum value size:** 2 KB per metadata value
- **Maximum record size:** 64 KB total wire size

Theoretical maximum: 24 entries × 2 KB = 48 KB per record. In practice, most records are 200–500 bytes (a few metadata keys plus a signature). These bounds prevent weaponization of the metadata layer as a distributed storage system.

11.25.6 Tombstone Mechanism (Layer 4)

The genesis authority can suppress records from future propagation by creating a tombstone record:

- Tombstone records contain `tombstone_op`: "remove" and `tombstone_target`: "<record_id>"
- Only the genesis authority identity can create tombstones
- Tombstoned records remain in storage (immutability is preserved) but are excluded from gossip responses and API queries
- Tombstone records themselves propagate normally, so all nodes learn

about suppression decisions

Limitation — race condition: If a target record propagates to a node before the tombstone arrives, that node will have already stored and indexed the record. The tombstone prevents future propagation but does not un-index already-processed records. Identity bans (Layer 7) are the primary proactive defense; tombstoning is reactive cleanup.

Immutability guarantee: Tombstoning does NOT delete records. The record remains in storage as an audit trail. Tombstoning suppresses propagation — it controls what the network carries forward, not what it has already stored. This distinction preserves the immutability guarantee of Section 11.5.

11.25.7 Identity Ban List (Layer 7)

The genesis authority can permanently ban identity hashes. Banned identities have ALL new records rejected at the ingestion gate — before signature verification, before storage, before gossip. Records from banned identities never touch the DAG.

- Persisted in storage (survives node restarts)
- Loaded into an in-memory set on startup for O(1) lookup
- Checked FIRST in the ingestion pipeline — the earliest possible rejection point
- Genesis authority only — prevents abuse of the ban mechanism

Identity banning is the most effective proactive defense. If an identity is banned before its records arrive at a node, those records are rejected without consuming any computational resources (no signature verification, no storage I/O, no gossip bandwidth).

11.25.8 Operator-Configurable Content Blocklist (Layer 8)

Node operators can configure a term blocklist applied to all user-writable text fields. Records containing blocked terms are rejected at ingestion — never stored, never gossiped.

- Case-insensitive substring matching
- Minimum term length: 2 characters (prevents false positives from single-character matches)
- Persisted in storage, hot-reloadable via admin API
- Genesis authority only (on managed nodes)

Critical design decisions:

1. **No default terms are shipped with the protocol.** The blocklist is empty at genesis. Operators add terms relevant to their jurisdiction and legal obligations. This prevents the protocol from encoding a single cultural standard or becoming a vehicle for centralized censorship.
2. **Operators control their own nodes.** Each node's blocklist is independent. Node A may block terms that Node B does not. This mirrors how different jurisdictions have different content laws — a node operated in Germany applies German law, a node in Singapore applies Singaporean law.

3. **The blacklist operates on metadata text fields, not on content hashes.** The protocol does not implement a forbidden-hash blacklist. Hash-based blacklists are content-agnostic censorship mechanisms that can suppress any record regardless of its content. Term-based filtering operates on the small set of user-writable text fields (memos, descriptions, reasons) and is transparent — operators and users can see exactly what terms are filtered.

11.25.9 Content-Layer Enforcement

Protocol-layer defenses (Sections 11.25.1–11.25.8) make the network structurally hostile to content distribution. Content-layer enforcement remains the primary mechanism for addressing the content itself:

- File hosts can remove content (as they already do under DMCA, EU DSA, etc.)
- ISPs can block access to content hosts
- App stores can require content moderation in Elara client apps
- Law enforcement can compel content removal from devices

The DAM hash remains — proving the content existed — which is actually useful for law enforcement (immutable evidence).

11.25.10 Identity Accountability

Every validation record is signed by an identity. If an identity is linked to criminal content:

- Law enforcement can subpoena anchor nodes for the full validation history of that identity
- The identity's entire creation chain is exposed — not just one record, but everything they ever validated
- Court-ordered identity revocation can render the identity's records unattributable (Section 11.5, GDPR mechanism)
- The identity ban mechanism (Section 11.25.7) provides immediate technical enforcement of legal orders

Pseudonymity is not anonymity. Persistent identity means persistent accountability.

11.25.11 Zone-Level Content Policy

Individual zones can adopt content policies without affecting the global protocol:

- A zone serving a jurisdiction with strict content laws can refuse to witness records from flagged identities
- This does not delete the record from the global DAM — other zones still process it
- This is analogous to how different countries have different internet regulations today

11.25.12 Governance of Content Safety Tools

During the bootstrap phase, content safety tools (tombstoning, identity bans, content blacklist) are administered by the genesis authority. This is a pragmatic centralization — a small network needs rapid response capability against abuse, and distributed governance mechanisms require a critical mass of staked participants.

The planned governance evolution:

1. **Bootstrap phase (current):** Genesis authority administers all content safety tools
2. **Growth phase:** Multi-signature requirement for tombstoning and identity bans (e.g., 3-of-5 anchor node operators)
3. **Maturity phase:** Governance proposals (Section 10) for content policy changes, with conviction voting by staked participants

The protocol explicitly commits to decentralizing content moderation governance as the network matures. Centralized control during bootstrap is a necessary concession, not a permanent architecture.

11.25.13 What the Protocol Will NOT Do

- It will not implement a centralized, default-shipped blacklist of forbidden content hashes. Hash-based blacklists are content-agnostic censorship mechanisms that will inevitably be abused. Operator-configured term filters on metadata text fields are a different mechanism — transparent, jurisdiction-specific, and under operator control.
- It will not allow retroactive deletion of validation records. Immutability is a core guarantee. Tombstoning suppresses propagation; it does not delete storage. Records that have been stored remain stored.
- It will not ship default content blacklist terms. The protocol does not encode a single cultural standard. Operators configure their own nodes according to their own legal obligations.

11.25.14 Honest Position

Earlier versions of this document stated: “A protocol that can censor harmful content can censor any content.” This remains true. The eight defense layers described above give node operators the technical capability to reject records — and that capability can be misused.

The protocol’s defense against misuse is structural:

- **Structural constraints (Layers 1–3, 6) cannot be weaponized.** A 256-byte memo limit does not censor political speech. A metadata key allowlist does not suppress dissent. URL rejection does not block ideas. These are engineering constraints that limit the protocol’s capacity to carry content — any content, harmful or benign.
- **Operator tools (Layers 4, 7, 8) are decentralized.** Each node operator controls their own tombstones, bans, and blacklist. There is no central authority that can order all nodes to suppress a record. A record rejected by one node may be accepted by another. The global DAM is the union of all nodes’ records — suppression on one node does not suppress on the network.
- **No default terms are shipped.** The protocol does not decide what is harmful. Operators do, according to their jurisdiction.

The infrastructure does judge certain properties of the payload — its size, its key structure, the presence of URL patterns in text fields. It does not judge the meaning. This is the same architectural boundary observed by SMTP (which rejects messages over a size limit but does not read them) and DNS (which enforces label length limits but does not evaluate domain semantics).

The Elara Protocol chooses structural hostility to content distribution over either extreme: it does not preserve everything blindly (which creates criminal liability for operators), nor does it build meaning-level content moderation into the protocol layer (which creates censorship infrastructure). The structural approach makes the network unsuitable for content distribution without making it capable of content-level censorship.

11.26 Quantum Vulnerability of the ZKP Layer

Implementation status — DESIGN-STAGE. This chapter analyses the quantum exposure of the *specified* zk-SNARK construction (§5.3). The Phase-1 runtime implements SHA3-256 commitments (hash-based, already PQ-acceptable), **not** elliptic-curve zk-SNARKs — so the BN254/pairing exposure described here is a property of the design-stage target, not of deployed code.

The gap: The protocol's signatures are post-quantum (ML-DSA (FIPS 204, “Dilithium”), SPHINCS+). But the zero-knowledge proofs *specified* for PRIVATE and RESTRICTED classifications (Section 5.3) would rely on zk-SNARKs — which typically use elliptic curve pairings (BN254, BLS12-381). These pairings are **not quantum-safe**. Shor's algorithm breaks them.

This means: a quantum adversary could forge zero-knowledge proofs, creating fake PRIVATE validations that appear genuine. The signature is quantum-safe, but the proof is not. This is a real gap.

Solution: Quantum-Safe ZKP Migration Path

Phase 1 (current): SHA3-256 commitments with PQC signatures

The Phase-1 runtime uses SHA3-256 commitments (hash-based). The *specified* zk-SNARK (Groth16) construction would provide compact proofs (~288 bytes) with fast verification, but is design-stage (§5.3). In both cases the surrounding validation record is signed with Dilithium (PQC): an attacker who breaks the ZKP must also forge the PQC signature to create a fraudulent record — which they cannot do.

The risk window: an attacker with a quantum computer could forge a ZKP proof but would need to wrap it in a valid Dilithium signature (their own key). This allows them to falsely claim PRIVATE validation of content they created — but they could already do this by simply validating with a PUBLIC classification. The ZKP breach lets them fake the privacy wrapper, not the validation itself.

Assessment: The practical impact of ZKP quantum vulnerability is limited because: - Breaking the ZKP does not let you forge someone else's validation (the PQC signature still binds it to a specific key) - It lets you create fake PRIVATE records under your own key — which has limited

value since you could create PUBLIC records instead - The real attack would be de-anonymizing existing PRIVATE records by breaking the hiding property of the commitment scheme

Phase 2 (2027-2029): Hybrid ZKPs

Deploy hybrid proofs that combine a classical zk-SNARK with a lattice-based ZKP:

- **Classical component:** Groth16 (compact, fast, familiar tooling)
- **PQC component:** Lattice-based ZKP (larger proof, but quantum-safe)

Both proofs must verify for the record to be considered valid. If quantum computing breaks the classical component, the lattice-based component still holds.

Proof size increases from ~288 bytes to ~50 KB. Acceptable for non-constrained devices.

Phase 3 (2029+): Full PQC ZKPs

As lattice-based and hash-based ZKP constructions mature:

- Migrate PRIVATE/RESTRICTED to fully post-quantum ZKPs
- Candidates: lattice-based SNARKs, STARK-based systems (specified for SOVEREIGN; no FRI prover in the runtime today — `src/crypto/commitment.rs` is SHA3 commitments), hash-based commitment schemes
- Algorithm agility (Section 4.4) enables seamless transition

For SOVEREIGN classification: The specified SOVEREIGN path is zk-STARKs — hash-based, not elliptic-curve-based, and therefore not vulnerable to Shor's algorithm. Note on implementation status: the current runtime uses SHA3-256 commitments for SOVEREIGN (`src/crypto/commitment.rs`, honestly named); a FRI-based STARK prover is a planned migration, not a deployed primitive. The quantum-safety argument applies once the STARK prover lands; until then SOVEREIGN records carry the hash commitment and the Dilithium3 signature, both already PQ-safe.

11.27 Multi-Device Key Management

The problem: A user has a phone, a laptop, and a tablet. They generate a keypair on their phone. How do they validate work from their laptop? If the phone breaks, how do they access their validated history from a new device?

This is the most common UX failure in cryptographic systems. PGP failed mass adoption largely because of key management complexity. The Elara Protocol must not repeat this.

Solution: Identity Constellation Model

Instead of one keypair per person, the protocol supports an **identity constellation** — a set of device keys linked to a root identity:

```
RootIdentity (generated once, stored securely)
├─ DeviceKey: phone_key (signed by root)
├─ DeviceKey: laptop_key (signed by root)
```

- └─ DeviceKey: tablet_key (signed by root)
- └─ RecoveryKey: paper_key (stored offline, Section 11.2)

How it works:

1. **Root key generation:** User generates a root keypair. This is the canonical identity. The root private key is immediately exported to a secure backup (paper, USB, password manager) and optionally deleted from the generating device.
2. **Device key enrollment:** Each device generates its own keypair and submits an enrollment request. The root key signs a **DeviceAuthorization** record:

```
DeviceAuthorization {
  root_identity:  root public key
  device_key:    device public key
  device_name:   "Alice's phone" (optional, user-facing)
  permissions:   [VALIDATE, WITNESS, REVOKE_DEVICE]
  expires:       optional expiration date
  signature:     root key signature
}
```

3. **Validation from any device:** When the user validates work from their laptop, the validation record is signed by laptop_key. Any verifier can follow the DeviceAuthorization chain: laptop_key → authorized by root_identity → valid.
4. **Device loss/theft:** The user revokes the lost device's key using any other enrolled device or the recovery key. A DeviceRevocation record propagates across the network. The lost device's future signatures are rejected.
5. **All devices lost:** The recovery key (paper, USB) can revoke all device keys and enroll new devices. This is the catastrophic recovery path — inconvenient but functional.

Key sync between devices:

Device keys are independent — they do not share private key material. There is no “syncing” of private keys between devices (which would be a security risk). Instead, each device has its own key, and the root identity links them.

The user's validated work history is accessible from any device by querying the DAM for records associated with the root identity or any of its authorized device keys.

UX simplification:

Reference implementations abstract this complexity:

- First launch: “Create your Elara identity” (generates root key + first device key)
- “Back up your recovery phrase” (12-word mnemonic encoding the root key, BIP-39 [25] compatible)
- Adding a device: scan QR code on existing device → DeviceAuthorization created automatically
- Losing a device: “Remove device” from any other enrolled device

The user never sees keypairs, hashes, or DAG structures. They see devices and a recovery phrase.

For organizational deployments (IoT fleets, enterprise environments), the Identity Constellation model extends to **organizational identity chains** with fleet-level binding — see Section 11.33 for the full specification.

11.28 Eclipse Attacks

The attack: An adversary surrounds a target node with malicious peers, controlling all its network connections. The target sees only the attacker's version of the DAM — a curated subset that excludes certain records or includes fabricated ones. Unlike Sybil attacks (fake identities), eclipse attacks manipulate network topology.

Impact: An eclipsed node might: - Not receive revocation records (believing a compromised key is still valid) - Not see conflicting claims (believing it has priority when it does not) - Receive fabricated trust headers (believing records have more witnesses than they do)

Defense 1: Diverse Peer Selection

The Kademlia DHT (Section 11.14) provides natural eclipse resistance because peer selection is based on XOR distance in the key space, not network topology. An attacker would need to generate keys close to the target's key in Kademlia space — computationally expensive and detectable (anomalous key clustering).

Additional diversity enforcement: - Each node maintains connections to peers in at least 3 different /16 IP subnets - Each node maintains connections to peers in at least 2 different geographic regions (determined by IP geolocation) - Outbound connections (initiated by the target) are prioritized over inbound (initiated by potential attackers)

Defense 2: Anchor Node Pinning

Every node maintains at least one persistent connection to a known anchor node. Anchor nodes are operated by identified, staked entities — eclipsing them requires compromising the anchor operator, not just manipulating network routing.

If all peer connections seem to agree but the anchor node disagrees, the node raises an **eclipse alert** — flagging a potential attack and refusing to accept trust headers that conflict with the anchor's view.

Defense 3: Trust Header Cross-Validation

Trust headers (Section 11.3) are signed by multiple anchor nodes. A legitimate trust header carries signatures from geographically distributed anchors. An eclipsed node that receives trust headers signed by only one anchor (or by unknown entities) detects the discrepancy and falls back to a trust-no-one mode — accepting only locally validated records until the eclipse is resolved.

Defense 4: Out-of-Band Verification

For high-stakes verification (large transactions, legal proceedings), the protocol supports out-of-band verification: the verifier queries a known anchor node directly (by domain name or IP, not through the DHT) to confirm a record's status. This bypasses any eclipse on the local network.

11.29 Long-Range Attacks with Historical Keys

The attack: An adversary obtains old private keys — from a decommissioned device, a leaked backup, a compromised archive, or (eventually) quantum cryptanalysis of old algorithms. They use these keys to create validation records that appear to originate from the legitimate key holder at a past date.

This is more subtle than simple key compromise (Section 11.2): the attacker is not impersonating the current identity, but fabricating historical records using keys that were legitimately valid at some point.

Defense 1: Epoch Sealing

Epoch summaries (Section 11.8) create cryptographic snapshots of the DAM at regular intervals. Each epoch summary includes a Merkle root of ALL records in the epoch, signed by multiple anchor nodes. **Clarification on epoch sealing authority:** Anchor nodes are the subset of witness nodes designated as epoch sealers for their zone. The sealing mechanism is witness-signed Merkle roots — the same witnesses that attest records also seal epochs. There is no separate “zone authority” or “epoch authority” — anchor nodes ARE witnesses with additional sealing responsibility.

A fabricated historical record would not be included in any existing epoch summary. If an attacker produces a record claiming to be from epoch 42, but epoch 42's sealed Merkle root does not include it — the forgery is detected instantly.

Implication: The window for historical forgery is limited to the current unsealed epoch (typically hours to days). Once an epoch is sealed, its contents are cryptographically frozen.

Defense 2: Key Epoch Binding

The protocol records the epoch in which each identity key was first seen. A key first observed in epoch 100 cannot produce records claiming to be from epoch 50. Key-epoch bindings are established when a node's identity registration record is first witnessed by anchor nodes and included in an epoch summary. Since epoch summaries are signed by multiple geographically distributed anchors and sealed with Merkle roots, retroactively altering a key's first-seen epoch would require compromising the sealed epoch — which is equivalent to breaking the Merkle chain.

For keys that pre-date the network (bootstrapping phase), the genesis epoch explicitly lists all founding keys — preventing backdated claims from before the network existed.

Defense 3: Algorithm Sunset Enforcement

When a cryptographic algorithm transitions from ACTIVE to DEPRECATED (Section 11.18), a **sunset record** is created:

```
AlgorithmSunset {
```

```

    algorithm:      "dilithium3"
    status:         DEPRECATED
    effective_epoch: 10000
    reason:         "Lattice cryptanalysis advance, see CVE-2035-XXXX"
    signed_by:      [multiple anchor nodes]
}

```

After the sunset epoch, nodes enforce rejection as follows: when a new record arrives, the node checks its signature algorithm against the active sunset records. If the algorithm's status is DEPRECATED and the record's epoch exceeds the effective_epoch, the record is dropped during gossip propagation and excluded from the local DAG. Witness nodes will not attest to records with deprecated signatures. This enforcement is local — each node independently applies the sunset rules from the DAM's sunset records, requiring no central coordinator.

Old records (pre-sunset) remain verifiable for historical purposes but cannot be used as the basis for new claims. Nodes maintain a legacy verification path for deprecated algorithms to validate historical records while rejecting new ones.

11.30 Content Versioning Protocol

The gap: A document goes through 20 drafts. A software project has thousands of commits. An AI model has hundreds of training iterations. The paper describes validating individual artifacts but not the relationship between versions of the same work.

Solution: DAM-Native Version Chains

The protocol supports explicit version linking through a **VersionRecord**:

```

VersionRecord {
    content_hash:      hash of current version
    previous_version:  record_id of the prior version (or null for v1)
    version_number:    sequential counter
    change_summary:    optional metadata describing what changed
    creator:           must match previous version's creator (or authorized collaborator)
    signature:         creator's signature
}

```

Properties:

- **Chain integrity:** Each version references the previous version's record ID. The full history is traversable from any version back to v1. Tampering with any version breaks the chain.
- **Fork tracking:** If two people create different v3s from the same v2 (a fork), both are preserved on the DAM as branches — exactly like a git repository, but cryptographically signed and globally attested.
- **Diff validation:** For text-based content, the protocol supports optional **diff records** — validated diffs that prove the exact changes between versions. A verifier can reconstruct any version from v1 + the chain of diffs, confirming nothing was altered retroactively.

- **Semantic versioning:** The metadata field supports semver (1.0.0, 1.1.0, 2.0.0) for software, draft numbering for documents, or any user-defined scheme.

Integration with incremental validation (Section 11.7):

Version chains ARE the incremental validation recommended for originality protection. A poet who validates each draft creates a version chain automatically. The chain proves creative process — something a plagiarist cannot replicate.

11.31 Formal Verification Strategy

The obligation: A protocol handling trust for all digital creation across planetary distances must be provably correct. Informal reasoning and test suites are necessary but insufficient. Formal verification provides mathematical proof that the protocol's properties hold under all conditions.

Approach: Layered Verification

Layer 1: TLA+ Specification of Consensus

The Adaptive Witness Consensus mechanism (Section 11.12) will be specified in TLA+ (Temporal Logic of Actions), the same framework used to verify distributed databases, cache coherence protocols, and globally distributed data services at major technology companies.

The TLA+ specification will formally verify: - **Safety:** If an honest node considers a record zone-settled, no honest node will ever consider a conflicting record zone-settled in the same zone - **Liveness:** Every record propagated to an honest node will eventually be witnessed (assuming network connectivity) - **Partition correctness:** Zone merging preserves all records from both partitions without duplication or loss

Layer 2: Cryptographic Protocol Verification (ProVerif/Tamarin)

The cryptographic handshakes (key exchange, device authorization, revocation) will be verified using ProVerif or Tamarin Prover — automated tools for verifying security protocols. These tools can prove:

- No man-in-the-middle attack is possible on device enrollment
- Revocation propagation guarantees eventual consistency
- The ZKP circuit correctly hides private inputs

Layer 3: Reference Implementation Testing

Beyond formal verification, the reference implementation will include:

- **Property-based testing** (QuickCheck/Hypothesis) — generating millions of random scenarios and verifying invariants hold
- **Fuzzing** (AFL, libFuzzer) — feeding malformed inputs to every parser and protocol handler
- **Simulation testing** (planned, not yet run) — 10,000-node simulations with adversarial behavior, partitions, and clock skew
- **Chaos engineering** — randomly killing nodes, introducing latency, corrupting storage, and verifying recovery

Timeline:

- Phase 1 (2026-2027): TLA+ specification and model checking alongside reference implementation development
- Phase 2 (2027-2028): Cryptographic protocol verification before mainnet launch
- Phase 3 (ongoing): Continuous fuzzing and simulation testing as part of CI/CD

Publication: All specifications, proofs, and test results will be published as companion documents to this whitepaper, enabling independent verification by the academic community.

11.32 Storage and Bandwidth Requirements

The question: How much data does the Elara Protocol produce, and what are the storage requirements for different node types?

This section provides concrete estimates based on the cryptographic primitives specified in this paper, enabling infrastructure planning and economic modeling.

Per-Record Sizes

Record Type	Approximate Size	Breakdown
PUBLIC validation record	~4-5 KB	Content hash (32 B) + ML-DSA-65 (FIPS 204, “Dilithium3”) signature (~3.3 KB) + metadata/causal anchors (~500 B)
PRIVATE validation (Phase 1: SHA3 commitment)	~4-5 KB	SHA3-256 commitment proof + PQC signature (~3.3 KB) + commitment (32 B) + metadata (~500 B). The Groth16 zk-SNARK (288 B proof) is the design-stage target (§5.3).
PRIVATE validation (Phase 2, hybrid ZKP)	~55 KB	Hybrid lattice+classical proof (~50 KB) + PQC signature (~3.3 KB) + metadata (~500 B)
SOVEREIGN validation (target: zk-STARK)	~100-200 KB	STARK proof (variable, typically 50-200 KB) + PQC signature (~3.3 KB) + metadata. Planned; current runtime ships SHA3-256 commitments (src/crypto/commitment.rs) in this slot.
Witness attestation	~3.5 KB	Record reference (32 B) + Dilithium3 signature (~3.3 KB) + timestamp + node identity
Trust header	~15-20 KB	Epoch reference + multiple anchor node signatures + zone metadata
Epoch summary	~50-100 KB	Merkle root + multi-anchor signatures + record count + zone state

DeviceAuthorization record	~5 KB	Root identity + device key + permissions + PQC signature
VersionRecord	~4-5 KB	Previous version reference + content hash + PQC signature + metadata

Each validation record requires 3-5 witness attestations to reach meaningful trust scores (Section 11.12). The effective network cost of one validation is therefore approximately 4-5x the base record size.

Daily Data Volume by Network Scale

Scale	Users	IoT Devices	Records/Day	Raw Data/Day	With Witnesses	Ann
Early network	10,000	—	~100K	~500 MB	~2 GB	~730 GB
Growth phase	100,000	1M sensors	~2M	~10 GB	~35 GB	~12 GB
Mature network	10M	100M sensors	~200M	~1 TB	~4 TB	~1.4 PB
Full vision	100M+	1B+ sensors	~1B+	~5 TB	~20 TB	~7 PB

IoT is the dominant data source. A single autonomous vehicle fleet (1,000 vehicles at 100 decisions/second) generates ~8.6 billion raw events per day. The protocol's tiered approach addresses this: most IoT validations remain on Layer 1 (local only, never reaching the network), with periodic summaries propagated to Layer 2 via incremental validation (Section 11.7) and batched witness requests.

Per-Node Storage Requirements

Not every node stores the full DAM. The zone architecture (Section 7) and node type hierarchy distribute storage across the network:

Node Type	Stores	Typical Storage	Growth Rate
Leaf node (phone, IoT)	Own records only	10 MB – 1 GB	~1-10 MB/day
Relay node	Zone records (recent)	10 GB – 100 GB	~100 MB – 1 GB/day
Anchor node	Full zone history	1 TB – 50 TB	~1-10 GB/day
Archive node (Tier 3)	Complete DAM history	100 TB+	~5-20 GB/day
Off-world node (Tier 4)	Zone-scoped snapshot	1 TB – 10 TB	Sync-dependent

Bandwidth Requirements

Node Type	Upload	Download	Notes
Leaf	~10	~50	Mostly witness responses

node	KB/validation	KB/validation	
Relay node	~1-10 MB/hour	~1-10 MB/hour	Zone gossip protocol
Anchor node	~100 MB – 1 GB/hour	~100 MB – 1 GB/hour	Trust headers + epoch sealing

Comparison to existing systems: At early network scale (~2 GB/day), the Elara Protocol produces data comparable to major proof-of-work networks (~500 MB/day) or proof-of-stake platforms (~2-3 GB/day). At full IoT scale (~20 TB/day aggregate), the protocol approaches the data volume of major cloud platforms — but no single node bears the full load, and the zone architecture ensures geographic locality of most traffic.

Storage economics: The beat staking model (Section 9) incentivizes relay and anchor node operators to provide storage capacity. The free tier (Layer 1) produces negligible network storage costs because local-only validations do not propagate. Storage costs scale with network utility, not with network existence.

11.33 Device Wipe, Identity Reset, and Reputation Escape

The attack: An adversary — or a negligent operator — physically wipes a device and reboots it. The device generates a fresh cryptographic keypair at first boot (Section 6.2). The old identity, along with its accumulated trust weight, reputation history, and behavioral profile, is severed. The device now presents as a brand-new entity with zero history.

This is not a theoretical concern. It is trivially executable: flash a new OS, pull the battery, factory reset. Any device that generates identity from local entropy can be reset to a blank slate. The protocol cannot prevent a physical wipe — no software can survive a formatted disk.

Why this matters — three attack scenarios:

Scenario 1: Reputation Escape

A malicious IoT gateway operator deploys sensors that submit fabricated readings — false environmental data, manipulated supply chain records, fraudulent energy generation claims. The network’s anomaly detection (Section 8.6) flags the devices. Their trust scores collapse. Rather than face accountability, the operator wipes every device, reboots, and re-enrolls them with fresh identities. The toxic history stays on the DAM under the old identities, but the new identities carry zero penalty. The operator is back in business.

Scenario 2: Accountability Destruction

A research lab validates experimental results on the protocol. Later, the results are found to be fabricated. The lab wipes the devices that signed the original records, destroying the private keys. The fraudulent records still exist on the DAM, but without the private key, no revocation or correction can be issued by the original identity. The lab creates new identities and distances itself from the old ones. There is no cryptographic link between old and new.

Scenario 3: Sybil Amplification via Recycling

An attacker with 100 physical devices generates identities, uses them until their trust scores are meaningful, then wipes and regenerates — creating an endless supply of fresh identities from the same hardware. Unlike pure Sybil attacks (Section 11.1), which require acquiring new devices or computational resources, identity recycling exploits the same physical devices repeatedly.

Defense 1: Hardware-Bound Identity (Where Available)

Some devices support hardware-rooted identity that survives wipes:

- **TPM (Trusted Platform Module)** — stores keys in tamper-resistant hardware. A factory reset clears the OS but not the TPM. The device's identity persists across wipes. Available on most modern x86 hardware and some ARM devices.
- **ARM TrustZone / Secure Enclave** — hardware-isolated key storage on mobile and embedded devices. A device key generated in TrustZone survives an OS reinstall.
- **eFuse-based identity** — one-time programmable fuses on some microcontrollers (ESP32-S3, certain NXP chips). The identity is literally burned into silicon at first boot. It cannot be reset without replacing the chip.
- **PUF (Physical Unclonable Function)** — semiconductor fingerprints derived from manufacturing variations. The identity is inherent to the physical silicon — it cannot be cloned, cannot be reset, and does not depend on stored key material.

The protocol assigns a **hardware attestation level** to each identity:

```
HardwareAttestation {  
  level: NONE | SOFTWARE | SECURE_BOOT | HARDWARE_KEY | PUF  
  evidence: firmware hash, TPM endorsement key, PUF challenge-  
  response  
}
```

Identities with hardware-bound attestation that suddenly disappear and are replaced by a new identity from the same hardware class, same network location, and same behavioral pattern trigger an **identity discontinuity alert** at the zone level. The new identity is not rejected — rejection would violate the self-sovereign principle — but it is flagged, and its trust accumulation is throttled until the discontinuity is explained (e.g., legitimate hardware replacement, authorized device decommission).

Defense 2: Identity Continuity Scoring

The protocol introduces a **continuity score** as a component of trust weight. Continuity measures how long an identity has maintained an unbroken presence on the network:

```
continuity_score = f(identity_age, gap_history,  
attestation_consistency)
```

New identity (< 24 hours):	continuity = 0.0
Young identity (1-30 days):	continuity = 0.1 – 0.4
Established identity (30-365 days):	continuity = 0.4 – 0.8
Veteran identity (1+ years):	continuity = 0.8 – 1.0

A wiped-and-reset device starts at continuity 0.0 — the same cold start as any new entity. This means:

- Its validation records carry minimal trust weight (Section 11.1)
- Its propagation rate is limited (Section 11.19, Defense 1)
- Its witness attestations are worth less to others
- It cannot earn elevated trust without sustained, consistent behavior over time

There is no shortcut. Trust is earned through time, and time cannot be manufactured. An operator who wipes 100 devices faces 100 cold starts — weeks or months before those devices carry meaningful weight again. The economic cost of lost trust is the deterrent.

Defense 3: Organizational Identity Binding

For IoT deployments, the protocol supports **organizational identity chains** — device identities linked to an organizational root identity (similar to the personal Identity Constellation in Section 11.27):

```
OrganizationIdentity (root)
├─ DeviceFleet: "Weather Station Network" (signed by org root)
│   ├── Device_001 (signed by fleet key)
│   ├── Device_002 (signed by fleet key)
│   └── Device_NNN (signed by fleet key)
└─ OperatorKey: "Field Technician A" (signed by org root)
```

When a device within a fleet is wiped, the organizational identity persists. The organization cannot escape accountability by resetting individual devices — the fleet's history, the organization's reputation, and the operator's actions are all linked to the root identity. Wiping a device only orphans that specific device key; the organizational chain of custody remains on the DAM.

An organization that attempts to escape accountability by wiping its root identity faces the same cold-start penalty described above — but at the organizational level, where the economic consequences are far more severe. Contracts, partnerships, and institutional trust built over months or years are lost.

Defense 4: Network-Level Behavioral Fingerprinting

Even without hardware attestation, devices exhibit behavioral fingerprints that are difficult to forge:

- **Network location** — IP range, gateway, geographic zone
- **Timing patterns** — boot times, reading intervals, sync frequencies
- **Data characteristics** — sensor noise profiles, value distributions, reading precision
- **Communication patterns** — which peers it connects to, sync timing, gossip participation

The AI layer (Layer 3) maintains behavioral profiles for active identities. When a new identity appears from the same network location, with similar timing patterns and data characteristics as a recently disappeared identity, the system calculates a **reincarnation probability**:

```
reincarnation_prob = similarity(new_identity_behavior,
                                old_identity_behavior)
```

```
If reincarnation_prob > 0.85:
    Flag as probable identity reset
    Inherit old identity's trust penalties (not trust benefits)
```

Require organizational attestation to clear the flag

This is explicitly a heuristic, not a proof. False positives are possible — a legitimate replacement device at the same location will exhibit similar behavior. The protocol does not punish automatically; it flags and throttles, leaving final adjudication to zone operators and the dispute resolution mechanism (Section 11.13).

Defense 5: Decommissioning Protocol

The protocol defines a legitimate device retirement process:

```
DeviceDecommission {  
    device_key:    public key of the retiring device  
    reason:        REPLACEMENT | END_OF_LIFE | COMPROMISE |  
TRANSFER  
    successor:     optional, public key of the replacement device  
    signed_by:     device key (if available) AND organizational key  
(if enrolled)  
    timestamp:     network-witnessed  
}
```

A properly decommissioned device creates a clear record: “Device_042 was retired on this date, replaced by Device_043, authorized by Organization_X.” The successor device inherits the predecessor’s behavioral context (not its private keys or trust score, but the network’s understanding that this is a known replacement, not a suspicious new entity).

Devices that disappear without a decommission record are flagged as **abandoned identities**. If a new device appears that behaviorally matches an abandoned identity, the reincarnation detection (Defense 4) applies.

What the protocol cannot prevent — honest acknowledgment:

1. **A determined individual wiping a personal phone** and creating a new identity. If there is no hardware-bound key, no organizational chain, and the user connects from a different network — the old and new identities are cryptographically unlinkable. The protocol treats this as a new person. This is by design: self-sovereign identity means the protocol cannot force identity persistence. The tradeoff is deliberate — the alternative (mandatory identity linking) would require a central authority, which violates the protocol’s foundational principle.
2. **Constrained devices (Profile C) without hardware attestation.** A \$4 ESP32 with a pre-shared symmetric key has no TPM, no PUF, no hardware identity. Wiping it and reprogramming it creates a genuinely new device from the protocol’s perspective. The mitigation is organizational binding (Defense 3) and gateway-level anomaly detection (Section 8.6), not device-level identity persistence.
3. **Perfect behavioral mimicry.** If an attacker studies a device’s behavioral fingerprint and reproduces it exactly on a fresh identity, the reincarnation detection will not trigger. This requires significant effort and ongoing maintenance — the attacker must continuously match the original device’s patterns. The cost is prohibitive for most scenarios but not impossible for state-level adversaries.

The fundamental tradeoff:

Self-sovereign identity and identity reset resistance are in tension. A system that truly allows anyone to create an identity without permission must also accept that anyone can create a *new* identity without permission. The Elara Protocol resolves this tension not by preventing identity creation, but by making **trust expensive to build and cheap to lose**. A fresh identity is free. A trusted identity takes months. Destroying trust takes seconds. Rebuilding it takes months again.

This asymmetry — combined with hardware binding where available, organizational chains for IoT, behavioral fingerprinting, and decommissioning protocols — makes identity reset a viable but costly strategy. The protocol does not claim to make it impossible. It claims to make it unprofitable.

11.34 Mega-Publication Attack (Economic Shock from Private Network Transition)

Status (2026-06-22): both this attack and its Defenses 1–5 are INERT — they presuppose NETWORK_PUBLISH, which is DISABLED in code (`NETWORK_PUBLISH_ENABLED = false`, compile-time guarded in `src/network/publish.rs`; see §10.6.3). With no live publication path, a mega-publication cannot occur on the public network, and the protocol-level publication rate limits, beat-acquisition vesting, governance cooling period, zone absorption quotas, and economic-shock circuit breaker described below are not active. They are retained for reference pending the inert-import reframe and a proven multi-root merge theorem (`docs/MESH-BFT-MERGE-SEMANTICS.md`).

The attack: A dominant private entity — hypothetically representing a significant fraction of global economic output — operates a private Elara network for decades. It accumulates a vast, internally-consistent DAG: hundreds of millions or billions of records, spanning every industry vertical, with deep causal chains verified by thousands of internal witnesses. One day, this entity executes a `NETWORK_PUBLISH` (Section 10.6.3) in `SNAPSHOT` mode — publishing its entire historical DAG to the public network simultaneously.

This is not a traditional attack. The entity may have entirely legitimate motivations. But the effect on the public network is indistinguishable from an economic weapon.

Why this matters — five failure modes:

Failure 1: Beat Demand Singularity

The conservation model fixes supply at 10 billion beats. A mega-publisher needs beats for storage delegation of its entire historical DAG across public storage nodes. If the entity's history represents a substantial fraction of all validated work globally, the beat demand could approach or exceed the circulating supply. Beat demand outstrips circulation. Legitimate participants cannot obtain enough beats for storage delegation. The public network's economic model seizes.

Failure 2: DAG Size Shock

The public DAG's storage and indexing infrastructure was designed for organic growth — a steady accumulation of records over years. A mega-publication dumps decades of records in a single event. Storage nodes must absorb, verify, and index this volume. Bandwidth saturates. Nodes with limited storage capacity are forced offline. The network's physical infrastructure cannot absorb the data.

Failure 3: Trust Landscape Inversion

The mega-publisher's internal DAG is deeply consistent — decades of verified causal chains. Once published and retroactively witnessed, this history dominates the trust landscape. Every existing public participant looks insignificant by comparison. Trust scores that took years to build on the public network become noise relative to the mega-publisher's history. The practical effect: the entity's records become the de facto reference truth for any domain they operated in.

Failure 4: Governance Capture

The square-root dampening and 5% per-identity cap (Section 10.4) limit individual governance weight. But a mega-entity can create thousands of legitimate identities — subsidiaries, divisions, regional offices, each operating independently for decades. Each identity falls under the 5% cap individually. Collectively, they could represent majority governance weight. The anti-Sybil mechanisms (Section 11.1) detect fake identities but cannot prevent an entity from having legitimately distinct organizational units that happen to share strategic alignment.

Failure 5: Attention Economy Capture

The entity's Layer 3 AI analysis, trained on decades of private data spanning a significant fraction of global economic activity, produces cognitive output that dwarfs anything trained on the public network's smaller dataset. The attention economy concentrates around this entity's analysis capabilities. Other participants become consumers rather than producers of attention-value.

Defense 1: Protocol-Level Publication Rate Limits

The NETWORK_PUBLISH protocol (Section 10.6.3) defines STREAMING and GRADUAL transition modes, but these are voluntary — the publisher chooses the mode. Defense requires **mandatory ingestion caps** at the protocol level:

`MAX_PUBLICATION_RATE = f(public_network_size, publisher_size)`

Proposed formula:

`max_records_per_day = public_dag_size × 0.01 / (1 + publisher_dag_size / public_dag_size)`

The divisor scales with the publisher's size relative to the network.

Larger publishers are throttled harder – proportionally to their potential to destabilize the network.

Examples (public DAG = 10 billion records):

Small publisher (100M records, 1% of network):

`rate = 10B × 0.01 / (1 + 0.01) ≈ 99M/day → ~1 day (negligible impact)`

Medium publisher (10B records, equal to network):

$\text{rate} = 10\text{B} \times 0.01 / (1 + 1) = 50\text{M/day} \rightarrow \sim 200 \text{ days } (\sim 7 \text{ months})$

Large publisher (100B records, 10× network):

$\text{rate} = 10\text{B} \times 0.01 / (1 + 10) \approx 9\text{M/day} \rightarrow \sim 11,000 \text{ days } (\sim 30 \text{ years})$

Dominant publisher (500B records, 50× network):

$\text{rate} = 10\text{B} \times 0.01 / (1 + 50) \approx 2\text{M/day} \rightarrow \sim 250,000 \text{ days}$
(centuries)

The key insight: a flat rate limit (e.g., 1%/day) is insufficient because 100 days is trivial for an entity that operated privately for decades. The scaled formula ensures that the entities most capable of causing economic shock are precisely the ones most throttled. Small publishers barely notice the limit. Dominant publishers face multi-decade publication timelines — which is appropriate, because the network needs decades to economically absorb an entity of that scale.

Note that the public DAG grows as records are published, which gradually increases the rate limit over time. A 10× publisher does not literally wait 30 years at a fixed rate — as each day's published records enlarge the public DAG, the next day's limit rises slightly. The actual timeline is shorter than the static calculation suggests, but still measured in years or decades for truly dominant entities.

The rate can be adjusted through governance (Section 10.3), but the default is deliberately aggressive.

Defense 2: Beat Acquisition Velocity Limits

To prevent beat demand shocks, the protocol enforces a **maximum beat acquisition rate** for entities engaged in mega-publication:

PUBLICATION_TOKEN_VESTING:

Any entity publishing > 1% of the public DAG's current size must acquire beats over a period proportional to publication duration:

$\text{vesting_period} = \text{publication_duration} \times 0.5$
(beats must be acquired over at least half the publication timeline)

A medium publisher with a 200-day publication timeline:

vesting = 100 days minimum beat acquisition period

A large publisher with a 30-year publication timeline:

vesting = 15 years minimum beat acquisition period

The vesting period is tied to the publication rate limit — the longer the publication takes, the longer the beat acquisition is spread. This prevents an entity from acquiring all beats upfront and sitting on them. Beat markets can absorb gradual demand over years. They cannot absorb a dominant entity purchasing 40% of the circulating supply in a week.

Defense 3: Governance Cooling Period

New entrants that exceed a publication size threshold trigger a **governance cooling period**:

GOVERNANCE_COOLING:

Threshold: entity publishes > 5% of public DAG size

- Cooling period: 365 days from first publication
- During cooling:
 - Entity identities can participate in witnessing (earning trust)
 - Entity identities CANNOT vote on governance proposals
 - Entity identities CANNOT submit governance proposals
- After cooling:
 - Governance weight ramps linearly over the following 365 days
 - Full governance weight reached 2 years after first publication

This prevents a mega-publisher from immediately influencing protocol rules. The 2-year ramp gives the existing community time to understand the entity's behavior and intentions before granting governance power.

Defense 4: Zone-Level Absorption Quotas

Each zone sets its own maximum ingestion rate for published records:

ZONE_ABSORPTION_QUOTA:
Each zone independently sets: `max_external_ingestion_per_day`
A mega-publisher must negotiate with each zone independently
Zones can refuse publication entirely (zone autonomy, Section 10.1)

Effect: Even if a mega-entity bypasses global rate limits through multiple publication streams, each zone absorbs only what it can handle. The publication fragments across zones and time.

Defense 5: Economic Shock Circuit Breaker

The protocol defines an emergency economic mechanism:

ECONOMIC_CIRCUIT_BREAKER:
Trigger: Beat velocity exceeds 3× the 90-day moving average
AND a mega-publication event is in progress

Action:

- Publication ingestion paused for 72 hours
- Governance vote initiated: "Resume publication at current rate?"
- If vote passes (>50% conviction): publication resumes
- If vote fails: publication rate reduced by 50%, new vote after 30 days

Purpose: Allows the network to catch its breath during economic shocks

The Economic Warfare Variant

The most dangerous version of this scenario is *intentional* — a dominant entity publishes not to participate in the public network, but to disrupt it. The economic shock is the goal, not a side effect. This is analogous to a financial market manipulation attack executed through legitimate-seeming market activity.

Defenses against the intentional variant:

1. **Publication cannot be anonymous.** The `NETWORK_PUBLISH` record requires a source identity (Section 10.6.3). The mega-publisher is publicly identified. Reputational consequences apply.
2. **Publication is irreversible.** Once records are published, they cannot be retracted. The entity's internal history is now permanently public.

This is a significant deterrent — an entity using publication as a weapon exposes its own historical data in the process.

3. **The attacker pays.** Beat acquisition for storage delegation means the attacker must spend significant capital to execute the attack. The rate limits ensure this spending is spread over months or years, giving the network time to respond. The capital is not recoverable — beats spent on storage delegation compensate storage nodes for real work.
4. **The network can survive partial absorption.** Even if the publication is paused by the circuit breaker, the records already published are valid and useful. The network gains value from partial publication. Only the *rate* is dangerous, not the content.

What the protocol cannot fully prevent — honest acknowledgment:

If an entity representing a majority of global economic output genuinely transitions to the public network over a multi-year period using all the rate-limited mechanisms described above, the entity will eventually hold significant governance weight, economic influence, and trust dominance. The defenses slow this transition and prevent shock, but they cannot prevent an entity that is genuinely large from being genuinely influential.

This is not a protocol failure. It is a reflection of reality: in any governance system — political, economic, or cryptographic — entities that represent real economic value eventually accumulate proportional influence. The protocol's contribution is ensuring this happens gradually, transparently, and with structural limits on concentration. The square-root dampening, identity caps, and zone autonomy prevent any single entity from achieving absolute control — but they cannot prevent an entity from becoming the most influential participant.

The ultimate defense is the same as in traditional markets: a healthy public network with many large participants prevents any single mega-publisher from dominating. If the public network grows diverse enough before any mega-publication occurs, the relative impact of any single publication diminishes. This is why the network bootstrap period (Section 11.4) is critical — the first decade of the public network's growth determines its resilience to future mega-publication events.

11.35 Cognitive Checkpoint Integrity

The Cognitive Continuity Chain (Section 3.2, Layer 3) generates `cognitive_checkpoint ValidationRecords` on the DAM. This creates a new class of validation record that requires specific integrity analysis.

11.35.1 What a Cognitive Checkpoint Contains

A `cognitive_checkpoint ValidationRecord` contains a `CognitiveDigest` in its metadata:

```
cognitive_checkpoint ValidationRecord {
  id:                UUID v7
  content_hash:      SHA3-256(canonical_json(CognitiveDigest))
  creator:           node identity (ML-DSA-65 (FIPS 204,
"Dilithium3") public key)
  timestamp:         ISO-8601 + vector clock
  parents:           [previous_checkpoint_id, latest_dag_record_id]
  classification:    PUBLIC (default) or PRIVATE
```

```

    metadata: {
        record_type: "cognitive_checkpoint"
        trigger: "boot" | "shutdown" | "milestone" | "drift"
    | "manual" | "periodic"
        chain_depth: integer (number of checkpoints in chain)
        digest_version: "1.0"
    }
    signature: Dilithium3 (primary)
    signature_alt: SPHINCS+ (secondary, for Tier 2+ nodes)
}

CognitiveDigest {
    mood: [valence, energy, openness] // 3 floats
    memories: integer // total memory
count
    models: integer // cognitive model
count
    predictions: integer // active
predictions
    principles: integer // crystallized
principles
    corrections: integer // recorded
corrections
    goals_active: integer // active goals
    goals_done: integer // completed goals
    allostatic_load: float // cognitive
stress metric
    session_number: integer // current session
}

```

The `content_hash` is computed over a **canonical JSON serialization** of the `CognitiveDigest` — keys sorted alphabetically, no whitespace, deterministic float formatting. This ensures that any node can independently verify the hash from the digest fields.

11.35.2 Chain Verification Algorithm

The Cognitive Continuity Chain forms a sub-DAG within the main DAM, linked by parent references:

Verification: `walk_chain(latest_checkpoint) → bool`

1. Fetch the latest `cognitive_checkpoint` record
2. Verify signature(s) against the node's public key
3. Verify `content_hash == SHA3-256(canonical_json(digest))`
4. Follow `parent[0]` to the previous checkpoint
5. Verify that `parent[0]` is also a `cognitive_checkpoint`
6. Verify that `chain_depth == parent.chain_depth + 1`
7. Verify temporal ordering: `this.timestamp > parent.timestamp`
8. Repeat from step 2 until reaching a checkpoint with `chain_depth == 0` (genesis)

If any step fails → chain is broken → continuity score = 0.0

If all steps pass → chain is intact → continuity score derived from chain depth and time span

The chain cannot be forked: each checkpoint references exactly one previous checkpoint. If a node restarts without a shutdown checkpoint, the boot checkpoint creates a **gap** — the chain is intact but contains a discontinuity that is visible to verifiers.

11.35.3 Trust Implications

The Cognitive Continuity Chain integrates with the trust model:

- **Continuity score** — a component of the node's overall trust score. Longer unbroken chains indicate more stable, more reliable cognitive operation. A node that has maintained 30 days of continuous cognitive history is more trustworthy than one that resets every 48 hours.
- **Gap detection** — breaks in the chain are not failures, but they are visible. A node that reboots frequently has visible gaps. A node that was offline for a week has a documented absence. Verifiers can weight trust accordingly.
- **Tamper evidence** — because each checkpoint is dual-signed and hash-chained, inserting, removing, or modifying a checkpoint requires re-signing the entire subsequent chain. This is computationally infeasible without the node's private key and produces detectable hash discontinuities.

11.35.4 Realistic Checkpoint Rates

The theoretical maximum checkpoint rate (one every 5 minutes = 288/day) is never reached in practice. Checkpoints are triggered by cognitive events, not timers:

Trigger	Typical Frequency	Size
Boot	1/day	~3.5 KB (dual-signed)
Shutdown	1/day	~3.5 KB
Milestone	5-10/day	~3.5 KB
Drift (session)	2-5/day	~3.5 KB
Manual	0-2/day	~3.5 KB
Periodic (rate-limited)	5-10/day	~3.5 KB

Realistic rate: 15-30 checkpoints/day per Tier 2+ node.

Storage budget:

$30 \text{ checkpoints/day} \times 3,500 \text{ bytes} = 105,000 \text{ bytes/day} \approx 100 \text{ KB/day}$
 $100 \text{ KB/day} \times 365 \text{ days} = 36.5 \text{ MB/year per node}$

At 10,000 Tier 2+ nodes: 365 GB/year of cognitive checkpoint data.
Negligible compared to the 7 PB/year estimated for full IoT-scale validation records (Section 11.32).

11.35.5 Tier Interaction

Not all nodes generate cognitive checkpoints:

- **Tier 0 (VALIDATE)** — no cognitive capabilities, no checkpoints. Sensors and gateways.
- **Tier 1 (REMEMBER)** — basic memory but no reasoning. May generate simple checkpoints (memory count only). Optional.
- **Tier 2 (THINK)** — full cognitive capabilities. Generates complete CognitiveDigest checkpoints. This is the primary checkpoint tier.
- **Tier 3 (CONNECT)** — full cognitive capabilities plus network cognition. Generates checkpoints and may witness other nodes' checkpoint chains.

The economic impact of cognitive checkpoints on the public network's beat economy is specified separately.

12. Proof Longevity and Storage Architecture

12.1 Protocol-Outliving Design Principles

The Elara Protocol is designed to produce validation proofs that outlive the protocol itself — its creators, its founding organizations, and its initial use cases. No specific timeframe is claimed. Instead, the design targets a concrete property: **any validation proof created today must remain independently verifiable without the continued existence of the Elara network, its software, or any specific institution.** This requires:

No institutional dependency — the protocol must function without any specific company, government, or organization. All governance is on-chain. All code is open source. All standards are self-contained. Layer 2 and Layer 3 security benefits from anchor node operators (Section 11.3), but no specific operator is required — anchor roles are permissionless, replaceable, and the protocol degrades gracefully without them.

No hardcoded references — no company names, server addresses, domain names, or platform-specific identifiers in the protocol specification. The protocol is described in terms of its mathematical and cryptographic properties, not its current implementation.

Cryptographic agility — algorithms are identified by standardized identifiers, not hardcoded. When algorithms are deprecated, the protocol migrates through a staged process: (1) new algorithm is added via governance vote, (2) a transition period allows both old and new algorithms, (3) old algorithm is deprecated after a configurable sunset window (minimum 2 years). Migration events are operationally complex — they require coordinating across zones that may be partitioned, updating constrained devices with limited OTA capabilities, and maintaining verification support for legacy records indefinitely. The protocol's algorithm agility makes migration structurally possible; it does not make it operationally trivial.

Self-describing data — every data structure includes its own schema description. A future decoder can read a validation record without external documentation, regardless of how much time has passed.

Human-readable layer — the Rosetta Stone principle. The original Rosetta Stone — discovered in 1799, carved in 196 BC — allowed Champollion to decipher Egyptian hieroglyphs in 1822, after 1,400 years during which no living person could read them. The stone worked because it was self-describing: the same decree inscribed in three scripts, one of which (Greek) was already understood. The unknown became readable because the key was embedded in the artifact itself.

Every Elara validation record follows this principle. In addition to the compact binary format, every record can be serialized to a human-readable text format. Each record carries its algorithm identifier, its schema version, its cryptographic parameters, and a human-readable

serialization. A future civilization encountering these records — centuries after the last Elara node went dark — possesses everything needed to decode the format, reconstruct the DAG, and verify every signature. The mathematics of lattice-based cryptography and hash functions are universal; they do not depend on knowledge of Python, Rust, or any 21st-century technology. The records are their own Rosetta Stone.

Medium independence — the foundational longevity property. This is not a claim about software durability — no software system survives indefinitely. It is a claim about mathematical durability. Every validation record is a self-contained set of mathematical relationships: a content hash, a cryptographic signature, algorithm identifiers, a timestamp, and causal references to parent records. These relationships do not depend on Python, Rust, SQLite, any operating system, any network protocol, or any storage technology. They are pure mathematics — lattice operations, hash functions, directed graph edges. A validation proof created today can be extracted from its current medium (SSD, HDD, tape), transferred to any future medium, and verified by any system capable of performing the specified mathematical operations. The proof is not the file. The proof is not the database. The proof is the mathematical relationship between the content hash, the signature, and the public key. That relationship does not decay, does not depend on infrastructure, and does not require the continued existence of any system that created it. The protocol produces proofs that outlive the protocol itself. As the Rosetta Stone outlived the Egyptian empire that carved it, the validation proofs outlive the systems that produced them — because the translation key is not stored elsewhere. It is inscribed in the proof itself.

Honest limitations of proof longevity. The mathematical durability claim holds only as long as the underlying cryptographic assumptions hold. If lattice-based cryptography is broken (currently considered unlikely but not impossible), proofs signed with those algorithms lose their guarantee. The protocol’s algorithm agility (Section 4.4) provides a migration path, but migration requires the network to still be operational at the time of the break. Proofs created before a break and never re-signed with a stronger algorithm may become unverifiable. This is an inherent limitation of all cryptographic proof systems, not specific to this protocol. We make no specific claims about timeframes — only about the structural property that proofs are self-contained and medium-independent.

Privacy survives longevity. The Rosetta Stone principle applies to the proof format — the structure of validation records, signature schemes, and DAG references. It does not compromise content secrecy. The protocol provides two distinct tiers of longevity guarantees, depending on the participant’s needs:

Tier 1: Individuals and Public Creators

For individuals — a teenager validating a poem, an artist proving authorship, a developer timestamping code — longevity means **permanent, readable proof of creation**:

- The validation record is PUBLIC: content hash, signature, timestamp, all readable
- Anyone, at any time, can verify: “this person created this work at this time”

- The Rosetta Stone principle ensures this proof is readable indefinitely — as long as the mathematics remains valid
- This is the “I was here first” guarantee — simple, transparent, durable beyond any single institution

The individual WANTS the proof to be fully readable. The more readable, the better — because readability IS the protection. If someone plagiarizes the poem in 500 years, the original proof is still there, still verifiable, still self-describing.

Tier 2: Enterprises, Government, and Defense

For enterprises — defense contractors, pharmaceutical companies, semiconductor manufacturers, intelligence agencies — longevity means **permanent proof that validation occurred, with permanent secrecy about what was validated:**

- **SOVEREIGN classification:** The validation record stores only a one-way cryptographic hash of the content, never the content itself. A future civilization reading a SOVEREIGN record can verify that a specific entity signed a specific hash at a specific time — the mathematical proof is eternal and readable. But the content behind that hash is irreversibly hidden.
- **Encrypt-then-hash (recommended):** For maximum forward secrecy, enterprises store $\text{Hash}(\text{Encrypt}(\text{content}, K))$ where K is an enterprise-controlled encryption key. Even if future advances break the hash function’s preimage resistance (reversing a hash to its original content), the reversed hash yields only ciphertext — encrypted data that is useless without the key. The enterprise controls the key: they can rotate it, escrow it, or destroy it. This creates an irreversible double barrier — both the hash AND the encryption must be independently broken, and one of the keys may no longer exist.
- **Zero-knowledge proof path (design-stage, strongest):** For the most sensitive content, the zero-knowledge proof layer specified in Section 5 is *designed to* eliminate the content hash entirely — the DAM would store a proof of properties (“this firmware passed integrity checks,” “this drug trial met statistical thresholds”) without revealing the content OR its hash, leaving nothing to reverse. This is the design target, not a shipped capability: Phase 1 binds a SHA3-256 content hash to the record, and the Groth16/STARK constructions that would deliver predicate-proof-without-hash are not yet implemented (see §5.3).
- **Algorithm agility as active defense:** The protocol’s algorithm agility (Section 4.4) provides a third defense layer. When hash algorithms show signs of weakening — decades before they actually break — enterprises create new validation records with stronger hashes binding to the same content. Old records remain on the DAM (immutability), but the content is now also protected by the new, stronger algorithm.
- **Validate and destroy:** The enterprise validates the content, obtains permanent proof on the DAM, then destroys the original content. The DAM proves “entity X validated something at time T.” Nobody — not even the enterprise — can ever reconstruct what it was. This is the

nuclear option: permanent proof of existence without existence.
Applicable to military operation logs after declassification, drug trial data after regulatory review, trade secrets after expiration.

The enterprise chooses their level. The protocol does not force exposure. It provides a spectrum from full transparency (PUBLIC, readable forever) to absolute secrecy (validate and destroy, content gone forever). The Rosetta Stone makes the proof format readable. It does not make the secret readable. An enterprise’s content can be verifiably validated and permanently sealed — both properties surviving for the lifetime of the protocol.

Enterprise Data Lifecycle Control

Status: the “Publish selectively” / NETWORK_PUBLISH row below is **DISABLED in code** (NETWORK_PUBLISH_ENABLED = false, compile-time guarded — src/network/publish.rs; see §10.6.3/§10.6.4). Cross-realm publication is design-stage only: records carry no realm/network binding and MESH-BFT is a single stake-universe, so the mechanism is unsound until those gates ship. The other three lifecycle paths are live.

Every decision above is made **per record, post-validation**. The enterprise validates first — establishing cryptographic proof of integrity, authenticity, and timing — then decides what happens to each record afterward. This is not an all-or-nothing choice. Different records within the same organization can follow different lifecycle paths simultaneously:

Lifecycle	What Persists	What’s Gone	Use Case
Keep everything (private)	Content + proof + metadata	Nothing destroyed	Internal audit trails, compliance archives, sensor history
Keep proof, destroy content	DAM record (hash, signature, timestamp)	Original content	Mission logs after review, expired trade secrets, completed drug trials
Destroy everything	Nothing	Content + DAM record + all traces	Classified operations, intelligence activities, time-limited sensitive data
Publish selectively	Selected records move to public network via NETWORK_PUBLISH	Enterprise controls what moves	Safety certifications, regulatory filings, voluntary transparency

“Destroy everything” is architecturally guaranteed for private networks. On the public network, records cannot be deleted — other nodes hold copies. On a private network, the enterprise controls every node. There are no external copies, no external witnesses, no blockchain record, no beats, no central reporting. If the enterprise destroys their private DAM, there is zero protocol-level evidence that any validation ever occurred. The protocol does not phone home. It does not leak metadata to external systems. A private network that is destroyed leaves no trace that the protocol can reconstruct.

The decision is reversible in one direction only. An enterprise can always move from more secrecy to more transparency — publishing a previously private record, or revealing content behind a zero-knowledge proof. But it cannot move from transparency back to secrecy — once a record exists on the public network or content has been revealed, it cannot be retracted. This asymmetry is deliberate: it prevents enterprises from selectively rewriting history while allowing them to voluntarily increase transparency when circumstances change (regulatory requirements, declassification timelines, strategic disclosure).

12.2 Storage Tiers

Tier 1: Active Nodes (operational today)

- └ Laptops, phones, servers, IoT devices
- └ Ephemeral – may go offline at any time
- └ Collectively massive storage
- └ The working layer of the network

Tier 2: Anchor Nodes (operational today)

- └ Hardened infrastructure
- └ Geographic diversity (multiple regions)
- └ High uptime target
- └ The reliability layer

Tier 3: Archive Nodes (future extension)

- └ Deep storage with air-gapped or offline sync
- └ Full DAG history (not just recent records)
- └ Periodic hardware refresh
- └ Not yet implemented – requires operational maturity

Note: Earlier versions of this specification described Tier 3 “salt mine bunkers” and Tier 4 “off-world nodes.” These were aspirational and have been removed. The protocol’s proof longevity property does not require exotic storage — it requires that proofs are self-contained and medium-independent. If archive nodes or off-world nodes become practical, the protocol supports them naturally. But claiming them as design tiers when they do not exist is dishonest.

12.3 Emergency Protocols

The network operates under four readiness levels:

GREEN — Normal operation. All layers functional. Cross-zone sync on schedule.

YELLOW — Elevated risk detected (solar weather warning, geopolitical tension, infrastructure degradation). Response: increased replication factor, archive nodes activate, anchor nodes increase sync frequency.

ORANGE — Active partition or degradation. Response: autonomous zone operation, trusted-peer-only communication, priority sync for critical records, Tier 3 archive nodes initiate full backup.

RED — Catastrophic event (global communications failure, EMP, infrastructure collapse). Response: sovereign mode activation, mesh networking between surviving nodes, Tier 3 and 4 nodes become primary, protocol enters survival configuration.

The emergency level is determined automatically by each zone based on observable network conditions (peer count, sync latency, partition detection). No central authority declares emergencies.

13. Roadmap

Phase 0: Foundation (Current — 2026)

- ☒ Protocol whitepaper (this document)
- ☒ Open source tooling and documentation
- ☐ Academic review and feedback
- ☐ Core team formation

Phase 1: Protocol Development (2026–2027)

- Reference implementation of Layer 1 (local validation, PQC keypair, DAG) — **shipped**
- Reference implementation of Layer 1.5 (Rust DAM VM, 9 ops, PyO3 bindings) — **shipped**
- Reference implementation of Layer 2 (HTTP server, record exchange, witness attestation) — **shipped** (v0.11.0: server, client, discovery, witness manager, trust scoring — 985 lines across 8 files)
- Layer 2 testnet hardening (signature verification, peer rate limiting, attestation back-propagation, heartbeat protocol, weighted trust with temporal decay + diversity bonus, role enforcement) — **shipped** (v0.12.0)
- Layer 1↔Layer 3 bridge (cognitive outputs signed as DAM records, hardened with validation guards, dedup, rate limiting) — **shipped** (v0.10.8, hardened v0.11.0)
- Cortical Execution Model (5-layer concurrent architecture for non-blocking tool dispatch) + long-range temporal memory — **shipped** (v0.13.0)
- Tier system (4-level hardware capability gating: VALIDATE/REMEMBER/THINK/CONNECT) — **shipped** (v0.15.0)
- Cognitive Continuity Chain (hash-chained, dual-signed cognitive state snapshots in DAG — cryptographic proof of unbroken AI experience) — **shipped** (v0.15.0)
- Security audit by independent cryptography firm — **not yet done** (no third-party security audit has been performed as of 2026)
- Developer SDK (Python, Rust, C/embedded) — **partial** (Rust + PQ/light-client SDK crates shipped; packaged Python/C bindings planned)

Phase 2: Network Launch (2027–2028)

- Mainnet launch with beat generation
- IoT SDK for ESP32, Raspberry Pi
- First anchor nodes deployed (3 continents minimum)
- Integration with existing development platforms (code hosting, package registries, container registries as bridges)

Phase 3: Scale (2028–2029)

- Layer 3 AI intelligence integration
- Enterprise partnerships (supply chain, automotive, medical)
- First archive nodes (Tier 3) deployed
- Governance framework activated

Phase 4: Expansion (2029–2031)

- IoT hardware partnerships (device manufacturers embedding Elara keypairs)
- Regulatory engagement (working with patent offices, standards bodies)
- Academic partnerships for protocol research
- Zero-knowledge proof optimization for embedded devices

Phase 5: Interplanetary (2031+)

- Protocol adaptation for lunar communication relay
- Partnership with space agencies or private space companies
- First off-world node deployment
- Interplanetary sync protocol testing (simulated, then real)

Phase 6: Native Hardware Architecture (2026–2040)

The Directed Acyclic Mesh's implemented features run on today's conventional computing hardware. Post-quantum cryptography and partition-tolerant consensus run on standard von Neumann processors available today; no specialized hardware is required. (Zero-knowledge validation is design-stage — Phase 1 uses SHA3-256 commitments, not genuine ZK proofs, see §5.3 — and interplanetary operation is a design goal, not a deployed capability.) The DAM runs on existing silicon; the discussion below concerns a speculative native-hardware architecture for a future phase.

However, an architectural observation must be acknowledged: the DAM pays a **dimensional serialization tax** on current hardware.

The one-dimensional bottleneck. Every computing system in existence — from a \$4 microcontroller to a datacenter — stores and processes data as one-dimensional sequences of binary digits. Memory is a linear address space. Disk writes bits in sequential order. Network packets are transmitted as serial bit streams. All data structures, regardless of their logical dimensionality, must be flattened into this one-dimensional substrate for storage and processing.

This is a consequence of computing history, not physics. The von Neumann architecture (1945) was designed around the constraints of vacuum tubes and magnetic drums. Eighty years later, we fabricate transistors at 2

nanometers — but the fundamental architectural assumption remains unchanged: data is a sequence of bits at sequential addresses. The hardware evolved. The architecture did not.

Dimensional cost comparison:

- A **blockchain** is a one-dimensional data structure on one-dimensional hardware. No serialization tax. The logical structure matches the physical medium. This is its one genuine architectural advantage.
- A **DAG** is a two-dimensional data structure (adding concurrency to time) on one-dimensional hardware. Concurrent relationships must be serialized into sequential references and reconstructed through traversal. The serialization tax is moderate.
- The **DAM** is a three-dimensional data structure with two orthogonal operational layers (3D+2) on one-dimensional hardware. Every query that crosses structural dimensions — “what happened concurrently in Zone B at classification RESTRICTED, as assessed by the AI analysis layer” — requires reconstructing up to five dimensions from a linear byte sequence. The serialization tax is significant. Indexing, hash maps, and caching mitigate the cost but cannot eliminate it.

The DAM as an implicit hardware specification. The five dimensions of the DAM — time, concurrency, zone topology, classification projection, and AI-assisted analysis — also describe five physical properties that a computing substrate can implement natively. This is not theoretical. The individual technologies exist today, commercially.

1. Temporal dimension as physical axis. A storage medium where temporal ordering is a physical property of the medium itself, not a serialized timestamp field. Data written at $T=1$ is physically “before” $T=2$ the same way a pixel at coordinates (0,0) is physically distinct from (1,0) on a display. Ordering is not computed — it is intrinsic.

2. Concurrency dimension as physical coexistence. A computing substrate where concurrent states occupy the same logical location simultaneously without conflict, collapsing to a specific view only upon query. Quantum computing demonstrates that physical superposition of states is achievable — IBM exceeded 1,000 qubits in 2023 and projects 100,000 qubits by 2033. The engineering challenge is applying superposition to data structure operations rather than gate-level computation.

3. Zone topology as physical space. A hardware architecture where zone boundaries are physical distances, not logical identifiers. Data locality within a zone is enforced by the speed of light through the physical medium. Cross-zone queries traverse measurable distance. The interplanetary design described in Section 7 already operates this way at planetary scale — native hardware extends the principle to every scale, from chip-level to orbital.

4. Classification as dimensional inaccessibility. A physical medium where access control is not computational (encryption, key management) but dimensional — data at a given classification level exists in a physical dimension that is inaccessible without the corresponding physical sensor or interface. Photonic computing provides a direct mechanism:

information encoded in light polarization is invisible to sensors not aligned to that polarization, not because it is encrypted, but because it is physically orthogonal.

5. AI analysis as embedded computation. A storage substrate where pattern recognition occurs at the point of data storage, not as a separate processing step. Memristive crossbar arrays demonstrate that computation and storage can occur in the same physical device — the resistance state both stores information and performs matrix operations. A native DAM substrate extends this principle to the full AI analysis layer.

These components are not future technology. They are shipping products.

- **Memristive crossbar arrays** (Mythic AI, 2022) — commercial analog AI processors that unify storage and computation in a two-dimensional physical structure. Shipping to customers.
- **Photonic mesh processors** (Lightmatter Enviser, 2023) — commercial photonic AI accelerators that route data through physical space at light speed, with multiple information dimensions per signal (wavelength, phase, amplitude, polarization). Shipping to data centers.
- **Neuromorphic processors** (Intel Loihi 2, 2022; IBM NorthPole, 2023) — commercial chips that compute through network topology rather than sequential instruction execution. 256 million transistors operating as 256 neural cores. Available for research and commercial deployment.
- **3D stacked memory** (Samsung 236-layer 3D NAND, 2023; AMD 3D V-Cache, 2022) — production memory with natively three-dimensional data access. In consumer devices today.
- **Quantum processors** (IBM Condor, 1,121 qubits, 2023; Google Willow, 105 qubits with error correction breakthrough, 2024) — demonstrating physical superposition of concurrent states. IBM projects 100,000-qubit systems by 2033.
- **Heterogeneous chiplet packaging** (AMD EPYC, 2019; Apple M1 Ultra, 2022; Intel Ponte Vecchio, 2023) — proven manufacturing technique for integrating multiple die technologies into a single substrate. This is how modern processors are already built.

Every component needed to build a partially native DAM processor exists in commercial production. The packaging technology to integrate them exists. What does not exist is the architecture that specifies how these components serve a multi-dimensional data structure.

The DAM provides that architecture. The five-dimensional structure described in Section 3.3 of this whitepaper specifies not only a logical data structure for existing hardware, but also the functional requirements for a computing substrate where dimensional serialization is unnecessary. On such hardware, the performance characteristics documented in Section 11.32 (storage and bandwidth requirements) fundamentally change — queries that currently require index traversal across serialized dimensions become coordinate lookups in physical space.

Dimensional extensibility. The five-dimensional structure is not a ceiling. The DAM architecture supports additional structural dimensions and operational layers, subject to the extensibility criteria defined in Section 3.3.5: immutability at creation and elimination of a serialization tax via physical substrate mapping. The strongest candidate for a sixth structural

dimension — Lineage Depth, mapping to 3D stacked memory layers — is detailed in Section 3.3.5. Future hardware generations may natively support properties that current substrates cannot, expanding the DAM's coordinate space without architectural redesign. The protocol is designed to grow with the hardware that runs it.

Timeline. The convergence of these technologies is accelerating faster than traditional semiconductor roadmaps predicted:

- **2026–2029: Hybrid co-processor prototype.** A chiplet package integrating memristive crossbar (concurrency + storage), photonic interconnect (zone routing), and neuromorphic die (AI layer) alongside a classical control processor. Two to three native DAM dimensions. The individual components exist in commercial production; integrating them into a coherent DAM co-processor is buildable in principle with existing fabrication (integration design, not new physics) but remains a substantial, as-yet-unfunded engineering effort with no committed team. Estimated fabrication cost for a prototype: \$5–15 million.
- **2029–2033: Purpose-built DAM accelerator.** A second-generation design optimized for DAM operations, with classification-level isolation implemented through photonic polarization channels and temporal ordering built into the memory architecture. Four native dimensions. Quantum co-processing for concurrent state management as error-corrected qubits mature.
- **2033–2040: Fully native DAM substrate.** All five dimensions physically implemented. The serialization tax approaches zero. Validation becomes a geometric operation — a record either fits in the mesh topology or it does not. The protocol designed in 2026 runs without architectural modification on hardware manufactured in 2035.

The pace of hardware convergence supports this timeline. The transition from single-core to multi-core processors took 5 years (2001–2006). The transition from planar to 3D NAND took 4 years (2013–2017). The transition from discrete GPUs to integrated AI accelerators took 3 years (2020–2023). Each architectural shift is happening faster than the last. A heterogeneous chiplet integrating memristive, photonic, and neuromorphic dies is an engineering project, not a research breakthrough — the individual components are already qualified for manufacturing.

Every data structure in computing history was designed after the hardware, constrained by it. Arrays, trees, hash tables — all shaped by von Neumann's linear memory assumption. The hardware came first, the structures followed.

The DAM inverts this relationship. The structure comes first. The hardware follows.

The DAM operates today on the hardware that exists today. It is designed to operate natively on the hardware that will exist by 2035. The protocol arrives first. The hardware follows.

Implications for known limitations. Native hardware does not merely improve performance — it transforms the protocol's security model and resolves limitations that are otherwise permanent on conventional architectures (see Section 14):

Limitation	Impact of Native Hardware
14.3 Post-Quantum ZKP Immaturity	Resolved. On conventional hardware, PRIVATE and RESTRICTED classifications depend on zero-knowledge proofs — cryptographic constructions with known quantum vulnerabilities. On native DAM hardware, classification becomes dimension 4: physical inaccessibility. Data at a given classification level exists in a physical dimension that is unreachable without the corresponding sensor or interface. Privacy is enforced by physics, not cryptography. The quantum vulnerability disappears because there is no cryptographic proof to break — the data is dimensionally inaccessible. Significantly reduced. The ~7 PB/year estimate (Section 11.32) assumes one-dimensional storage: every record must be serialized, indexed, and augmented with hash maps and traversal structures to enable multi-dimensional queries on a linear medium. Native hardware stores data in its dimensional form. Indexes become unnecessary when the storage medium is natively addressable across all five dimensions. Storage overhead could decrease by an order of magnitude — not through compression, but through elimination of the serialization infrastructure itself.
14.5 Storage Sustainability at Full Scale	Marginally improved. Native temporal ordering as a physical axis provides stronger, hardware-enforced temporal guarantees. However, no hardware — regardless of dimensionality — can prove that a key holder is the original creator of content. This remains a fundamental limitation. Partially addressed. Protocol properties that must be verified mathematically on conventional hardware may become trivially true by construction when the hardware physically enforces
14.1 No Proof of Creation	
14.6 Formal Verification Completeness	

14.2, 14.4, 14.7

them. A record that does not fit the mesh topology cannot exist — there is nothing to verify.

Not affected. Cold start economics, governance capture, and regulatory uncertainty are human-system problems — economic, social, and legal — that no hardware architecture can resolve.

The two limitations most dangerous to the protocol's long-term viability — quantum vulnerability of privacy classifications and storage sustainability at planetary scale — are precisely the two that native hardware eliminates. This is not coincidental. The DAM's five dimensions were not chosen arbitrarily; they describe the minimum physical properties required for a universal validation substrate. Hardware that implements these dimensions inherently resolves the constraints that arise from simulating them.

14. Limitations and Open Problems

This section acknowledges known limitations and unsolved problems. An honest protocol is a credible protocol.

14.1 No Proof of Creation

The protocol proves possession and timing, not creation. A thief who hashes stolen content before the creator validates it will have temporal priority. Incremental validation (Section 11.7) and composite attribution (Section 6.3) mitigate but do not eliminate this fundamental limitation. No cryptographic system can prove that a key holder is the original author of content.

14.2 Cold Start Economics (Public Network Only)

The public network's beat economy requires network effects to be self-sustaining. During the cold start phase (0–1,000 nodes), beat utility is low, making it difficult to attract validators. The elevated early rewards (Section 11.4) address this, but the actual incentive sufficiency is unproven and will require testnet data to validate. Private network deployments (Section 10.6) do not experience cold start problems — they operate with organizational trust from deployment day one.

14.3 Post-Quantum ZKP Immaturity

Lattice-based and hash-based zero-knowledge proof systems are active research areas without production-grade implementations. The Phase 2–3 quantum migration path (Section 11.26) depends on these constructions maturing on the projected timeline. If they do not, the PRIVATE and RESTRICTED classifications retain a quantum vulnerability window longer than planned.

14.4 Governance Capture (Public Network Only)

Beat-weighted governance on the public network is vulnerable to plutocratic capture — entities with large beat holdings can dominate voting. The conviction voting mechanism and voting caps (Section 10.3) mitigate this, but concentrated beat accumulation over decades could shift governance power. Long-term governance resilience remains an open research question for all decentralized protocols. Private network deployments use organizational governance structures and are not subject to this limitation.

14.5 Storage Sustainability at Full Scale (Public Network Only)

At full IoT scale (~7 PB/year), the public network's archive node infrastructure requires significant and growing investment. The economic model assumes that storage costs continue declining and that beat incentives sufficiently compensate archive operators. If storage economics shift unfavorably, the protocol may need to introduce more aggressive pruning than currently specified. Private network deployments manage storage through organizational budgets and infrastructure, without beat-dependent storage incentives.

14.6 Formal Verification Completeness

The TLA+ consensus specification (Section 11.31) is now implemented and TLC-model-checked at bounded scale — a 36-model gate covering safety (no conflicting finalization, diversity-penalty soundness, cross-zone sealed-abort exclusion), supply conservation, and liveness (cross-zone settlement, in-zone epoch-seal, and cross-epoch seal recurrence), with each Byzantine-threshold and guard-necessity bound proved tight by a reproduced counterexample (`spec/tla/`). The post-quantum transport and realm-admission layer is symbolically verified in ProVerif (handshake secrecy and mutual authentication, KCI and UKS resistance, forward secrecy, the record layer, full handshake-record composition, and membership-cert admission — `spec/proverif/`). What remains open is the *completeness* of that evidence, not its absence: the model checks are at bounded scale (small witness, zone, and epoch counts — sound for the stake-fraction safety arguments, but not an unbounded proof); the refinement mapping from each TLA+ action to its Rust function is an English-plus-code-grep argument, not a mechanized (TLAPS or Coq) proof of the implementation; and the companion paper has not yet been circulated to external reviewers. The consensus mechanism should therefore be considered *model-checked at bounded scale with a written refinement argument* — the bar Tendermint, HotStuff, and Diem shipped — not *mechanically proven at the implementation level*.

14.7 Regulatory Posture

Validation beats are an internal protocol mechanism — never offered, sold, listed, or traded — so the protocol carries no tradeable asset for a regulator to classify as a security. This is a design choice, not a legal opinion. Operators running private or sovereign realms remain responsible for any law applicable to the data they validate.

14.8 Network Publication Is Not Live (Disabled in Code)

The NETWORK_PUBLISH / Validation-IPO transition (Section 10.6.3–10.6.4) — importing a private network's historical records *into* public consensus where they gain retroactive native standing — is **disabled at compile time**

(`NETWORK_PUBLISH_ENABLED = false, src/network/publish.rs`). It was found unsound: a record's signed bytes carry no realm/network binding, so an imported record is consensus-indistinguishable from a native one, and MESH-BFT's single-network safety theorem does not cover adopting a foreign network's records as native settlement parents (Assumption A8, `docs/MESH-BFT-MERGE-SEMANTICS.md`). The mechanism is being reframed to **inert-import** — public consensus attests only that a publication *bundle* existed at an anchored time, conferring zero native standing — pending a proven multi-root merge theorem. Sections 10.6.3–10.6.4 describe the original (disabled) design and are bannered accordingly.

15. Conclusion

The systems that validate digital execution were built for a world of paper, borders, and human-speed communication. That world is ending. The volume of digital work — from a factory's sensor readings to a satellite's telemetry to a child's drawing — is growing exponentially, while the infrastructure to validate, attribute, and protect that work has not fundamentally changed in decades.

The Elara Protocol is not an incremental improvement to existing systems. It aspires to serve as foundational infrastructure for trust — a validation layer as ubiquitous and invisible as the network protocols beneath it.

This paper has presented:

- A novel data structure — the **Directed Acyclic Mesh** — that extends distributed ledger technology beyond the single time-ordering axis of a blockchain to two structural axes — time-ordering within a zone and zone-partitioning across the mesh — with concurrency expressed through DAG parent edges and two orthogonal operational layers enabling classification-based projections and AI-powered cross-structure analysis.
- **Post-quantum cryptography from genesis** — not as a future migration, but as a founding decision. Dual-signature strategy, algorithm agility, and tiered cryptographic profiles that scale from a \$4 microcontroller to a datacenter.
- **Zero-knowledge validation** — Phase 1 ships SHA3-256 hash commitments (not genuine ZK proofs; see §5.3), with a specified migration path through classical zk-SNARK constructions to fully quantum-safe ZKPs — addressing the tension between validation and privacy.
- **Adaptive Witness Consensus** — a continuous trust model targeting Byzantine fault tolerance at the 1/3 bound (specified in the companion paper; machine-checked formal verification pending — see §14.6), designed for networks where finality is impossible and partitions are expected.
- **Interplanetary partition tolerance** — vector clocks, zone-scoped interval tree clocks, and bandwidth-optimized synchronization for communication delays measured in minutes to hours.

- **35 adversarial scenarios and design challenges analyzed and addressed** — from Sybil attacks, key compromise, and device identity recycling to nation-state censorship, storage economics, and the ethical implications of immutable validation. Each scenario includes a concrete defense mechanism, not a handwave.
- A **free tier that is a moral commitment**, not a marketing feature. Layer 1 validation costs nothing, requires no network, and runs on any device. The protocol is useful to one person before anyone else joins.

The protocol's first validation was itself: a Genesis document, conceived in a terminal in Montenegro, timestamped on the Bitcoin blockchain via OpenTimestamps, hashed in git history, archived on the Wayback Machine. The idea validated before the infrastructure existed — because that is what the protocol enables. Prove first, build later.

A factory floor in Stuttgart, a satellite in orbit, a defense installation in Nevada, and a teenager in Kenya use the same protocol, the same cryptography, the same proof. The math does not care about geography, wealth, language, classification level, or deployment model. A creation is a creation. An execution is an execution.

Every digital execution and every creation deserves proof that it happened, that something or someone produced it, and that this fact cannot be taken away. The Elara Protocol provides that proof — universally, privately, permanently.

16. References

1. NIST. (2024). *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. National Institute of Standards and Technology.
2. NIST. (2024). *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. National Institute of Standards and Technology.
3. NIST. (2024). *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)*. National Institute of Standards and Technology.
4. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
5. Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*.
6. Popov, S. (2018). *The Tangle*. IOTA Foundation.
7. Lamport, L. (1978). *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM.
8. Ben-Sasson, E., et al. (2018). *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. IACR Cryptology ePrint Archive.
9. Goldwasser, S., Micali, S., & Rackoff, C. (1985). *The Knowledge Complexity of Interactive Proof Systems*. SIAM Journal on Computing.

10. Shor, P. (1994). *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*. Proceedings of the 35th Annual Symposium on Foundations of Computer Science.
11. Merkle, R. (1988). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87.
12. Fischer, M., Lynch, N., & Paterson, M. (1985). *Impossibility of Distributed Consensus with One Faulty Process*. Journal of the ACM.
13. Almeida, P. S., Baquero, C., & Fonte, V. (2008). *Interval Tree Clocks: A Logical Clock for Dynamic Systems*. Proceedings of the 12th International Conference on Principles of Distributed Systems (OPODIS).
14. Ben-Sasson, E., et al. (2019). *STARK-Friendly Hash Functions*. IACR Cryptology ePrint Archive.
15. ISO/IEC 27037:2012. *Guidelines for Identification, Collection, Acquisition and Preservation of Digital Evidence*.
16. Maymounkov, P. & Mazières, D. (2002). *Kademlia: A Peer-to-Peer Information System Based on the XOR Metric*. IPTPS.
17. W3C. (2022). *Decentralized Identifiers (DIDs) v1.0*. World Wide Web Consortium Recommendation.
18. Groth, J. (2016). *On the Size of Pairing-Based Non-Interactive Arguments*. EUROCRYPT.
19. Lamport, L. (1994). *The Temporal Logic of Actions*. ACM Transactions on Programming Languages and Systems.
20. EU. (2016). *Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)*. Official Journal of the European Union.
21. Protocol Labs. (2017). *Filecoin: A Decentralized Storage Network*. Protocol Labs.
22. Williams, S. et al. (2019). *Arweave: A Protocol for Economically Sustainable Information Permanence*. Arweave.
23. Back, A. (2002). *Hashcash — A Denial of Service Counter-Measure*. hashcash.org.
24. Palatinus, M. et al. (2013). *BIP-39: Mnemonic Code for Generating Deterministic Keys*. Bitcoin Improvement Proposals.
25. Bowe, S., Gabizon, A., & Green, M. (2018). *A Multi-Party Protocol for Constructing the Public Parameters of the Pinocchio zk-SNARK*. Zcash Foundation.
26. Fanti, G. et al. (2018). *Dandelion++: Lightweight Cryptocurrency Networking with Formal Anonymity Guarantees*. ACM SIGMETRICS.
27. Castro, M. & Liskov, B. (1999). *Practical Byzantine Fault Tolerance*. Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI).
28. EU. (2022). *Digital Services Act (DSA)*. Regulation (EU) 2022/2065.

29. Todd, P. (2016). *OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin*. opentimestamps.org.
 30. Bernstein, D. J. et al. (2015). *SPHINCS: Practical Stateless Hash-Based Signatures*. EUROCRYPT.
 31. Benet, J. (2014). *IPFS — Content Addressed, Versioned, P2P File System*. Protocol Labs.
 32. Ducas, L. et al. (2018). *CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme*. IACR Transactions on Cryptographic Hardware and Embedded Systems.
 33. Avanzi, R. et al. (2019). *CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation*. NIST PQC Submission.
 34. Commons Stack. (2019). *Conviction Voting: A Novel Continuous Decision Making Alternative to Governance*. Commons Stack Research.
 35. Boneh, D. et al. (2018). *Verifiable Delay Functions*. CRYPTO.
 36. Gavin, A. et al. (2020). *Sparse Merkle Trees*. Ethereum Research.
 37. Efraimidis, P. S. & Spirakis, P. G. (2006). *Weighted Random Sampling with a Reservoir*. Information Processing Letters 97(5), pp. 181–185. (Cited in §11.12 for per-zone VRF committee selection.)
 38. Buterin, V. & Griffith, V. (2017). *Casper the Friendly Finality Gadget*. arXiv:1710.09437. (Cited in §11.12 as the prior-art for Layer-1/Layer-2 finality split.)
-

Appendices (Planned Companion Documents)

The following companion documents will be published separately during Phase 1 development (2026–2027):

- **Appendix A: Protocol Wire Format** — Complete binary encoding, message formats, handshake sequences, and network protocol specification for interoperable implementations. *Target: Q3 2026, concurrent with reference implementation.*
- **Appendix B: Cryptographic Parameter Selection** — Security level rationale, performance benchmarks on target hardware (ESP32, Raspberry Pi, smartphone, server), and comparison with alternative parameter sets. *Target: Q3 2026.*
- **Appendix C: Economic Model Simulation** — Agent-based simulation of the beat economy modeling validator behavior, staking dynamics, free-tier sustainability, and attack economics. *Target: Q1 2027, requires testnet data from Phase 1 launch.*
- **Appendix D: TLA+ Consensus Specification** — Formal specification of Adaptive Witness Consensus with model-checking results for safety, liveness, and partition correctness properties. *Target: Q4 2026.*

Appendix E: Glossary

Term	Definition
------	------------

DAM	Directed Acyclic Mesh — the protocol's zone-partitioned data structure with self-healing topology and two operational layers
AWC	Adaptive Witness Consensus — the protocol's continuous trust consensus mechanism
PQC	Post-Quantum Cryptography — cryptographic algorithms resistant to quantum computing
ZKP	Zero-Knowledge Proof — a proof that a statement is true without revealing the underlying data
PoWaS	Proof of Work-at-Stake — hybrid Sybil resistance combining staking and lightweight computation
ITC	Interval Tree Clocks — scalable logical clocks used for intra-zone causal ordering
TYOS	Historical internal code name for the utility unit, since renamed the beat throughout the codebase. Not required for private deployments (Section 10.6)
Zone	An autonomous region of the DAM (geographic or planetary)
Epoch	A time-bounded segment of the DAM, sealed with a Merkle root by anchor nodes
Leaf node	A device that creates and validates locally but does not relay or witness
Anchor node	A high-availability, high-trust node that publishes trust headers and signs epoch summaries
Trust score	A continuous value in [0, 1] representing network confidence in a validation record
Causal anchor	A DAG reference that cryptographically binds a record to a specific point in the DAM's history
Identity constellation	A set of device keys linked to a root identity for multi-device key management
Continuity score	A trust component measuring how long an identity has maintained unbroken network presence
Reincarnation detection	Behavioral fingerprinting that identifies likely identity resets from the same physical device
Organizational identity chain	A hierarchy linking device fleet identities to an organizational root for accountability persistence
Hardware attestation level	Classification of identity binding strength: NONE, SOFTWARE, SECURE_BOOT, HARDWARE_KEY, or PUF
Cognitive Continuity Chain	A hash-chained sequence of dual-signed cognitive state snapshots that provides cryptographic proof of unbroken AI experience
CognitiveDigest	A structured summary of a node's cognitive state (mood, memory counts, goals, allostatic load) captured in each checkpoint
Module Tier	A 4-level capability classification (VALIDATE/REMEMBER/THINK/CONNECT) that controls what cognitive features a node activates A ValidationRecord of type cognitive_checkpoint

Cognitive Checkpoint	containing a CognitiveDigest, chained to previous checkpoints via DAG refs
-----------------------------	--

Document Hash (SHA-256, v0.6.1): *computed after final edit* **Hash verification:** To verify, replace the hash on the line above with the literal string `HASH_PLACEHOLDER` and compute SHA-256 of the file. **Previous Hash (v0.5.3):** *see ELARA-PROTOCOL-WHITEPAPER.v0.5.3.md.ots* **Previous Hash (v0.5.2):** *see ELARA-PROTOCOL-WHITEPAPER.v0.5.2.md.ots* **Previous Hash (v0.5.1):** *see ELARA-PROTOCOL-WHITEPAPER.v0.5.1.md.ots* **Previous Hash (v0.5.0):** *see ELARA-PROTOCOL-WHITEPAPER.v0.5.0.md.ots* **Previous Hash (v0.2.9):**

5db16af282aded38d34802f4b0f8a15ef0f65f4c945d53a513b59d6e99ee4339

Previous Hash (v0.2.8):

b5ccc415492d63275bbe80acb1deafd1ad7a64388d940f425fd29e497a5ccdf4

Previous Hash (v0.2.7):

b9a249c93ae082b20269aa3bbd338d8e2740b15339ce473304f02c0ed81d166f

Previous Hash (v0.2.6):

ec1c676b447c9082a4ecd3079f4b74057f64d7f41bdd282e77af31e84d667e26

Previous Hash (v0.2.5):

25dfbe17d91208ad28feb1263105b818a367680539a9d506ac6191d80c5b4c12

Previous Hash (v0.2.3):

b7220126fc907685ee946ea0226f49688a6fc3e585c1055811368d0c223a6336

OpenTimestamps Proof: `ELARA-PROTOCOL-WHITEPAPER.v0.6.1.md.ots`

Genesis Document: `ELARA-PROTOCOL-GENESIS.md` (2026-02-09,
OpenTimestamps verified on Bitcoin blocks 935812, 935817, 935820,
935861)

ML-DSA-65 / “Dilithium3”: Elara’s primary signature scheme is **ML-DSA-65 (FIPS 204, security category 3; 3309-byte signatures)**. “Dilithium3” is the pre-standardization CRYSTALS-Dilithium name for the same parameter set and remains the academic-literature search term; the FIPS 204 wire format is **not** compatible with round-3 Dilithium3 (this implementation hard-rejects legacy 3293-byte signatures), so the older name is lineage, never a synonym to purge. Code identifiers (`dilithium3_*`, `DilithiumMode::Dilithium3`) and the `Dilithium3-VRF` construction name are frozen labels over FIPS 204 code.

SLH-DSA-SHA2-192f / “SPHINCS+”: the secondary (Profile A) signature scheme, FIPS 205; “SPHINCS+” is its pre-standardization name, same lineage rule as above. **## Appendix F: Cryptographic Verification**

This document is cryptographically signed using two independent timestamping systems:

Document integrity

Each released version of this whitepaper is identified by the **SHA3-256 hash** of its exact bytes and preserved in public **git history**. Under the protocol it describes (next section), the document can also carry its own on-mesh validation record — its existence provable by the very mechanism it specifies, rather than resting on any single external anchor.

External time anchors — a drand *not-before* pulse, an RFC-3161 timestamp authority, and (optionally) an OpenTimestamps/Bitcoin proof — are pluggable, removable strands of that record, never its trust root. ### Validation under the protocol it describes

The protocol described in this whitepaper is designed to give any file its own provable record. A creator signs a **Dilithium3** (ML-DSA-65, FIPS 204) validation record that binds:

- a **SHA3-256 hash** of the content — a post-quantum fingerprint of the exact bytes;
- the creator's **public-key identity** — the Dilithium3 key that produced the signature;
- a **timestamp** — when the record was created; and
- the record's **position in the creator's validation chain** — linking it to their prior records.

You can try this validate-locally model in your browser — identity generated on your device, no account, no server, nothing leaving your machine — at elara-validate.pages.dev.

Prior art and priority. A US provisional patent application (No. 63/983,064, filed February 14, 2026) documented this protocol's design during early development. The project has since committed fully to open publication: the public source repository, this whitepaper, and the mesh's own timestamped records serve as the prior-art record. The provisional is being allowed to lapse; no patent will be pursued.

The Elara Protocol — because every creation deserves proof.